# Computer Forensic Paper-II — Rapid Revision, Importance Ratings & Model Answers

**NPSC CTSE 2026 · Elective: Computer Forensic (Scientific Assistant, Police) · Technical Paper-II (descriptive, 100 marks, 3 h) · Wed 14 Oct 2026, 09:00 IST**

## If you have only 20 minutes: read these first

1. **Search & seizure of digital evidence:** the step-by-step procedure, the first responder's role and toolkit, and live vs dead-box (when to pull the plug).
2. **Order of volatility** (RFC 3227): registers/cache → RAM → network state/processes → disk → remote logs → archival media.
3. **Imaging & hashing:** bit-stream image, write blocker, imaging vs cloning, MD5/SHA-1/SHA-256 and verifying the hash.
4. **Forms of cyber crime:** computer as target/tool/incidental. ATM, UPI and banking frauds. Social-media crimes.
5. **Packet sniffing and spoofing** (IP, ARP, DNS, email), plus web security (SQLi, XSS, CSRF).
6. **Email header analysis:** read `Received:` bottom-up, check Return-Path/Message-ID, and use SPF/DKIM/DMARC for spoofing.
7. **Browser artifacts:** cookies, cache, history, bookmarks, session data and plugins.
8. **File signatures** (JPEG FF D8 FF, PDF %PDF, ZIP/DOCX PK, EXE MZ), **recovery of deleted/hidden/altered files**, and **steganography**.
9. **SQL injection & database security** (GRANT/REVOKE, triggers for auditing) and **transactions/recovery**.
10. **Mobile:** cells and frequency reuse, GSM architecture, 1G–5G, IMEI vs IMSI.

## How the importance ratings work

`10` sure to come · `9` very likely · `8` likely · `7` good chance · `6` / `5` moderate · `4` – `1` background only

**Why these ratings:** no Computer Forensic CTSE paper has been published yet. The ratings follow how NPSC set the 2025 CS papers, which **copied syllabus phrases word for word** as "explain / differentiate / describe" questions. **Units 1, 2, 7 and 8** (cyber crime, browsers/email/cloud, mobile, digital evidence) are the forensic-specific units, so they score highest. **Units 3–6** (SE, C/C++/web, DBMS/SQL, data structures) are CS core you already know well.

## Exam-hall plan (Sec A any 8 of 10 × 5 · Sec B any 3 of 5 × 10 · Sec C any 2 of 4 × 15)

- **Answer the forensic questions first,** with diagrams: the seizure flowchart, the volatility pyramid, the email path and the GSM architecture.
- **For every procedure answer:** definition → numbered steps → diagram → precautions → legal/court significance (chain of custody, hash, §65B IEA / §63 BSA certificate) → conclusion.
- **Time budget:** 5-mark ≈ 8 min · 10-mark ≈ 15 min · 15-mark ≈ 25 min · keep 10 min to review.

**Contents (forensic units first):** Unit 1 Cyber Crime · Unit 8 Digital Evidences · Unit 2 Browsers, Email, VM & Cloud · Unit 7 Mobile Computing · Unit 3 Software Engineering · Unit 5 Database & SQL · Unit 4 C, OOP, C++ & Web Programming · Unit 6 Data & File Structures. Each pair of units ends with its likely questions and model answers.

# Unit 1 — Cyber Crime

## Forms of cyber crime — classification

Importance 10/10 · sure to come · opening syllabus phrase "forms of cyber crime" — near-certain 5/10-mark question

In simple words: cyber crime = any crime where a computer/network is the target, the weapon, or just the place the evidence is stored.

**(a) By role of the computer**

| Class | Computer is… | Examples |
|---|---|---|
| **Computer as target** | the victim | Hacking/unauthorised access, DoS/DDoS, website defacement, ransomware, data theft, virus/worm release |
| **Computer as tool/instrument** | the weapon | Online fraud, phishing, identity theft, forgery, cyber-stalking, CSAM distribution, IPR piracy |
| **Computer as incidental** | a store/record | Drug dealer's client spreadsheet, murderer's search history, fraudster's chat logs |
| **Prevalence of computers** | the environment | Software piracy, copyright/trademark violation, counterfeit hardware |

**(b) By victim**

| Against | Offences | Indian legal hook (⚠️ verify each section) |
|---|---|---|
| **Person** | Cyber-stalking, bullying, defamation, identity theft, phishing, email spoofing, morphing, sextortion | IT Act §66C (identity theft), §66D (cheating by personation), §66E (privacy) |
| **Property** | Online/UPI/card fraud, IP theft, piracy, data theft, viruses, ransomware | IT Act §43 (damage, civil), §66, §65 (source-code tampering) |
| **Government / organisation** | Hacking govt systems, attacks on critical infrastructure, espionage, cyber-terrorism, defacement | IT Act §66F (cyber terrorism), §70 (protected systems) |
| **Society** | CSAM, obscenity, online gambling, mass scams, fake news | IT Act §67, §67A, §67B |

Exam tip: give **both** tables for "classify cyber crime"; for 10 marks add one example each + IT Act sections (write "⚠️ verify" mentally — cite concept + section, never invent).

## Internal vs external attacks

Importance 9/10 · very likely · classic "differentiate" sub-question

In simple words: insiders already have the keys; outsiders must break in.

| Aspect | Internal (insider) | External (outsider) |
|---|---|---|
| Who | Employee, contractor, ex-employee, privileged user | Hacker, criminal group, competitor, state actor, script kiddie |
| Access | **Already legitimate**; knows systems and value of data | Must **gain** access — exploits, phishing, brute force, credential stuffing |
| Detection | **Hard** — looks like authorised work | Easier — firewall/IDS log the intrusion |
| Motive | Grievance, greed, espionage, negligence | Money, ideology, notoriety, espionage |
| Evidence | Access beyond role, off-hours activity, mass download/USB exfiltration, deleted logs | Firewall/IDS logs, malware artifacts, foreign IPs, new accounts/backdoors |
| Defence | Least privilege, separation of duties, DLP, logging & review, exit controls | Firewall, IDS/IPS, patching, MFA, segmentation |

Exam tip: insiders are the harder problem because they **defeat perimeter defences by design**.

## Crimes related to social media

**Importance 8/10 · likely** · syllabus phrase; everyday relevance

In simple words: abuse of social platforms to impersonate, harass, extort, mislead or defraud.

| Crime | What happens | Artifacts |
|---|---|---|
| Fake profile / impersonation | Clone identity to defame/defraud contacts | Account creation IP, device, registered phone/email (from platform) |
| Cyber-stalking / harassment / bullying | Repeated targeted abuse | Message logs, timestamps, account attribution |
| Sextortion / morphing / revenge porn | Coercion with real or fabricated intimate images | IT Act §66E/§67A; EXIF, upload logs |
| Fake news / incitement | Coordinated disinformation | Forwarding chains, group metadata, first originator |
| Financial scams | Romance, investment, job, fake giveaway | Chat logs, UPI/wallet trail, mule accounts |
| Account takeover | Credential theft, SIM-swap hijack | Login IP history, session tokens, recovery-email change |

**Obtaining evidence:** (1) documented open-source capture (screenshot + URL + timestamp + **hash**); (2) legal process to the platform (law-enforcement portal / MLAT for foreign providers); (3) device forensics of the app's local databases. IT Rules 2021 first-originator obligations ⚠️ verify current status.

## ATM and banking frauds

**Importance 9/10 · very likely** · syllabus phrase; mechanism + evidence pairs score well

In simple words: most banking fraud succeeds by tricking the user into giving OTP/PIN/CVV, not by "hacking" the bank.

| Fraud | Mechanism | Evidence / detection |
|---|---|---|
| **Skimming** | Hidden reader on ATM/POS slot copies **magnetic-stripe** data; pinhole camera / overlay keypad gets PIN | Recovered skimmer, CCTV, location anomalies |
| **Cloning** | Skimmed data written to blank card; used where magstripe fallback allowed. **EMV chip resists cloning** (per-transaction cryptogram) | Same card used in two places; fallback flag |
| Card trapping (Lebanese loop) / shimming | Card trapped in slot / thin device reads chip data | Physical device, CCTV |
| **Phishing** | Fake email/website harvests card, CVV, net-banking login, OTP | Look-alike domain, email headers, hosting records |
| **Vishing** | **Voice** call posing as bank/RBI/KYC extracts OTP/PIN | CDRs, spoofed caller ID, mule trail |
| **Smishing** | **SMS** with malicious link / fake helpline | Sender ID, URL analysis |
| **SIM swap** | Duplicate SIM obtained → OTPs intercepted → account drained | Telco SIM-change logs, IMSI change vs transaction timing |
| **UPI frauds** | **Collect request** that actually debits; **QR scam** (scanning a QR **pays**, never receives); fake apps/handles; remote-access apps (AnyDesk) steal UPI PIN; fake customer care | UTR/RRN, VPA, PSP logs, device ID, mule account KYC |
| Money mules | Recruited accounts layer stolen money | Account-graph analysis |

**Reporting:** call **1930** helpline / **cybercrime.gov.in** (I4C) immediately — "golden hour" freeze of the money trail. Never share OTP/PIN/CVV.

## Data privacy issues

Importance 7/10 · good chance · syllabus phrase; law part must be cautious

In simple words: harm when personal data is collected, used, shared or leaked beyond consent.

- **Issues:** data breach/leakage · unauthorised profiling/tracking · function creep (reuse for another purpose) · surveillance without lawful basis · consent & purpose limitation · data minimisation & retention · cross-border transfer.
- Also a constraint on investigators: do not trawl personal data beyond warrant scope.
- **Indian framework (⚠️ verify sections):** IT Act **§43A** (body corporate failing to protect sensitive personal data; SPDI Rules 2011) · **§72/§72A** (breach of confidentiality/disclosure) · **§66E** (privacy violation) · **DPDP Act 2023** — Data Principal / Data Fiduciary / Data Processor, consent, purpose limitation, **Data Protection Board of India**, penalties (⚠️ rules' operative status — awareness level only, no section numbers) · *K.S. Puttaswamy v. Union of India (2017)*: privacy a fundamental right under Art. 21 (⚠️ verify spelling).

## Packet sniffing

Importance 8/10 · likely · syllabus phrase; dual use (attack and evidence tool)

In simple words: a sniffer captures every packet it can see on the network and reads it.

- NIC put in **promiscuous mode** (wired) or **monitor mode** (Wi-Fi).
- **Passive sniffing:** on a **hub** every port sees all frames — just listen.
- **Active sniffing:** on a **switch** the attacker must defeat it — **ARP spoofing/poisoning**, **MAC flooding** (overflow CAM table → switch broadcasts); legitimate route = **port mirroring/SPAN**.
- **Tools:** Wireshark, tcpdump, tshark, Zeek.
- **Forensic use:** a **PCAP** is the network equivalent of a disk image.

- **Defence:** encryption (TLS/HTTPS, SSH, VPN); switched networks, port security, **Dynamic ARP Inspection**.
- Exam line: encryption hides payload but not **metadata** (who, when, how much — traffic analysis).

## Spoofing

**Importance 9/10 · very likely** · syllabus phrase; table answer

In simple words: forging an identity field so traffic/messages appear to come from someone else.

| Type | What is forged | Abuse | Defence |
|---|---|---|---|
| **IP** | Source IP in header (L3) | DDoS reflection/amplification, hide origin, bypass IP trust | Ingress/egress filtering (BCP 38), reverse-path checks |
| **MAC** | NIC MAC address | Bypass MAC filtering, impersonate device | Port security, DHCP snooping, 802.1X |
| **ARP** | IP↔MAC mapping in ARP caches (L2, LAN) | **MITM**, sniffing on a switch, session hijack | Static ARP, Dynamic ARP Inspection, arpwatch |
| **DNS** | Name→IP answer (cache poisoning) | **Pharming** — redirect to fake site | **DNSSEC**, randomised ports/IDs |
| **Email** | From / envelope sender | Phishing, BEC, malware | **SPF, DKIM, DMARC** |
| **Caller-ID / SMS sender** | Displayed number / sender ID | Vishing, smishing | Telco controls, awareness |
| Website/URL (typosquatting, homograph) | Apparent domain | Phishing | Certificate check, punycode display |

## Web security — SQLi, XSS, CSRF, HTTPS

**Importance 8/10 · likely** · syllabus phrase "web security"

In simple words: web attacks trick the server (SQLi) or the user's browser (XSS, CSRF) into doing what the attacker wants.

| Attack | How | Forensic footprint | Defence |
|---|---|---|---|
| **SQL injection** | Unsanitised input concatenated into SQL; `' OR '1'='1` bypasses login; `UNION SELECT` exfiltrates; blind/time-based | Access logs with `%27`, `UNION`, `OR 1=1`, `--`, `information_schema`; spikes of 500 errors | **Parameterised queries**, input validation, least-privilege DB account, WAF |
| **XSS** | Inject script that runs in another user's browser — **stored**, **reflected**, **DOM-based**; steals cookies | `<script>`, `onerror=`, `javascript:` in content/logs; outbound calls to attacker | Output encoding, CSP, `HttpOnly` cookies |
| **CSRF** | Logged-in victim tricked into an unintended request; browser auto-sends session cookie | Valid authenticated request with **foreign Referer/Origin** | Anti-CSRF token, `SameSite` cookies, re-authentication |
| Session hijacking | Steal/fix session ID (sniffing, XSS, fixation) | Same session ID from two IPs/user-agents | TLS, `Secure` + `HttpOnly`, rotate ID on login |

- **XSS vs CSRF:** XSS abuses the site's trust in the **browser** (runs script); CSRF abuses the site's trust in the **authenticated user** (rides the session).
- **HTTPS** = HTTP over TLS: encryption (defeats sniffing), server authentication by certificate (defeats spoofed sites/MITM), integrity. Does **not** stop SQLi/XSS (application bugs).

## First responder — role and toolkit

**Importance 10/10 · sure to come** · unique forensic syllabus phrase

In simple words: the first trained person at a digital crime scene — job is **not to solve the case but not to destroy it**.

**Duties:** (1) secure & isolate scene; (2) **assess power state** (on/off/sleep); (3) don't touch keyboard/mouse except documented capture; (4) **photograph** screen, cabling, serials, room, sticky-note passwords; (5) capture **volatile evidence** if trained (RAM, processes, connections); (6) identify all devices incl. NAS, phones, cloud sessions; (7) **prevent remote wipe** — network cable out / Faraday bag; (8) start **chain of custody** — label, seal, log; (9) never run programs or open files "to check"; (10) hand over with complete notes.

| Toolkit category | Items |
|---|---|
| Documentation | Notebook/forms, chain-of-custody forms, evidence labels, **camera**, markers |
| Packaging | Anti-static bags, **Faraday bags/boxes**, evidence tape, boxes, cable ties |
| Acquisition hardware | **Hardware write blockers** (SATA/IDE/USB/NVMe), forensic duplicator, **sterile (wiped, verified) target drives**, adapters |
| Live capture | Trusted IR USB with statically-linked tools (memory capture, process/network listing), clean external drive |
| Software | Bootable forensic Linux live CD, dd/dcfldd, FTK Imager, hashing tools |
| Misc | Torch, screwdrivers, gloves, tamper-evident seals, power strip, **UPS/portable power** |

Guiding doctrine: ACPO — don't change data; if you must, be competent and document why (see Unit 8).

## Procedure for search & seizure of digital evidence

**Importance 10/10 · sure to come** · verbatim syllabus phrase — strongest 10/15-mark candidate

In simple words: a lawful, witnessed, documented drill that grabs the most fragile data first and seals everything with hashes.

1. **Plan** — target, likely devices, encryption risk; obtain proper legal authority.
2. **Secure the scene**, restrict access, document state on arrival.
3. **Photograph** everything before touching.
4. **Assess power state** → apply live / dead-box logic.
5. **Capture volatile evidence** (if ON and trained) in **order of volatility** (RFC 3227).
6. **Isolate from networks** (cable out / Faraday) — stop remote wipe.
7. **Label and seize** all storage: phones, USBs, SD cards, routers, NAS, cameras, notes/passwords, chargers, cables.
8. **Image on-site or seize for lab** using **write blockers**; compute **hashes** at acquisition.
9. **Chain of custody** — seal exhibits, record seal numbers and hashes.
10. **Transport** safely (anti-static, cushioned, away from magnets/heat) to lab.

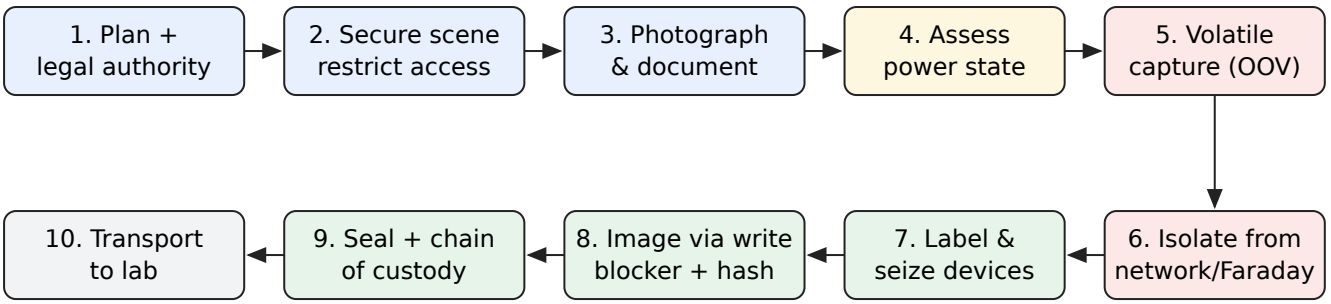



*Fig: Search and seizure procedure for digital evidence (10 steps)*

**Legal hooks (⚠️ verify numbers; cite concept + both codes):** search/seizure powers under **BNSS 2023** (replaced CrPC; seizure witnessed — *panchnama*) · IT Act **§80** (police power to enter/search public place, arrest without warrant) · admissibility needs the **§65B Indian Evidence Act 1872 / §63 Bharatiya Sakshya Adhiniyam 2023 certificate** · BNSS mandatory forensics for offences punishable ≥ 7 years (commonly cited §176(3)). Golden rule: seizure must be **lawful, witnessed, documented**.

## Search & seizure of volatile and non-volatile evidence (live vs dead-box)

**Importance 10/10 · sure to come** · verbatim syllabus phrase; the decision logic earns the marks

In simple words: if the machine is off, keep it off and image the disk; if it is on, grab RAM first, then decide how to power down.

| Aspect | Volatile evidence — live acquisition | Non-volatile — dead-box (static) |
|---|---|---|
| System | Running | Powered off |
| Captures | RAM, processes, network connections, **unlocked encrypted volumes**, cloud sessions | Full bit-stream disk image |
| Integrity | Harder — running OS changes state; rootkit may lie | Cleaner, repeatable, write-blocked |
| Risk | Every action alters some state (justify per ACPO P2) | All volatile evidence lost |

| Power-down option | Pros | Cons |
|---|---|---|
| **Pull the plug** | Freezes disk; defeats shutdown wipers/scripts | Possible FS corruption; loses RAM; re-locks encrypted disk |
| **Graceful shutdown** | Clean file system | Shutdown scripts may wipe temp/pagefile, rotate logs, trigger logic bombs; changes timestamps |
| **Live capture first, then decide** | Preserves most volatile evidence | Needs trained responder + trusted tools |

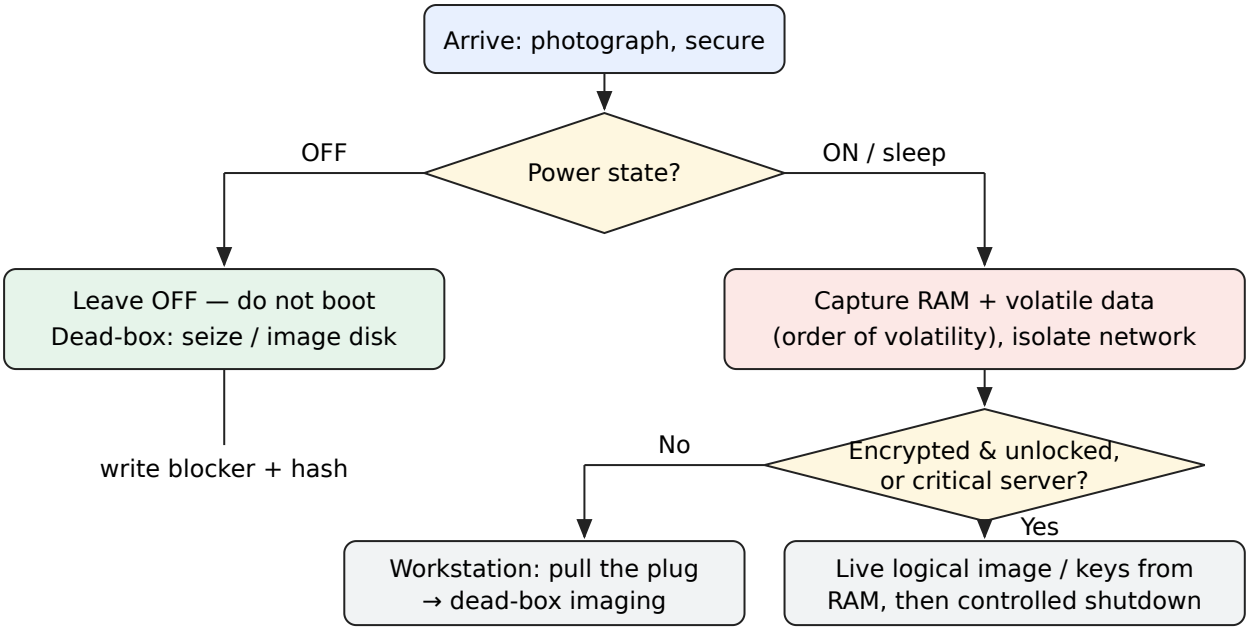



*Fig: Live vs dead-box decision for volatile and non-volatile evidence*

Memorise: *"Off → keep off and image dead-box. On → capture volatile memory first (especially if disk encrypted and unlocked), then pull the plug for a workstation at risk of shutdown wipers, or controlled/live acquisition for a server or unlocked encrypted volume."* Sleeping/hibernating = treat as ON (hiberfil.sys may hold a RAM image).

## Imaging and hashing digital evidence

**Importance 10/10 · sure to come** · verbatim syllabus phrase; integrity spine of every forensic answer

In simple words: make an exact sector-by-sector copy without writing to the original, and prove it is identical with a hash "seal".

- **Bit-stream (physical) image:** every sector — allocated + **unallocated + slack + HPA/DCO** → deleted files recoverable. **Logical copy:** only live files → **no deleted data**.
- **Write blocker:** reads allowed, **all writes blocked**. **Hardware** (between drive and workstation; gold standard) vs **software** (OS/driver read-only; depends on OS behaving). Windows writes to any disk it mounts.

| Format | Features |
|---|---|
| **dd / raw** | Pure bit-for-bit; **no metadata, no compression, no embedded hash** (store hash separately); universally readable |
| **E01 (EnCase / Expert Witness Format)** | **Compression**, **embedded case metadata**, **CRC per block + stored hash → self-verifying**; segments .E01, .E02… |
| AFF/AFF4 | Open format; compression, metadata, very large images |

| Hash | Digest | Status |
|---|---|---|
| **MD5** | **128-bit** (32 hex) | Collision-broken; still fine vs accidental change |
| **SHA-1** | **160-bit** (40 hex) | Collision-broken (SHAttered 2017) |
| **SHA-256** | **256-bit** (64 hex) | Recommended (SHA-2) |

**Workflow:** (1) attach via write blocker, hash **source**; (2) acquire bit-stream image; (3) hash **image** — must equal source → verified; (4) record both hashes in chain of custody; (5) analyse a **working copy**; re-hash before/after. Best practice: **two algorithms (MD5 + SHA-256)**. ⚠️ SSD garbage collection can legitimately change a live SSD's hash between reads — document and explain any mismatch.

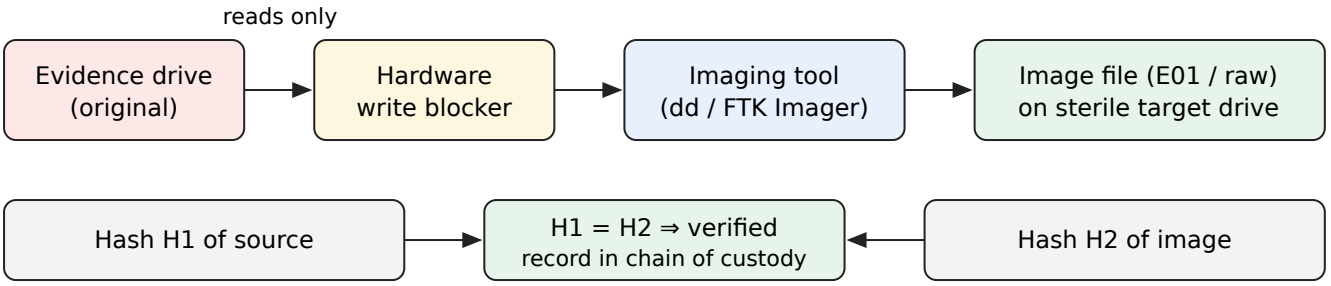



*Fig: Forensic imaging through a write blocker with hash verification*

## Analysing and recovering deleted, hidden and altered files

**Importance 9/10 · very likely** · verbatim syllabus phrase

In simple words: "delete" only updates bookkeeping — the bytes stay until overwritten, so we can get them back.

- **Deletion mechanics:** FAT sets first filename byte to **0xE5** and frees the chain; NTFS marks the MFT record unallocated; ext3/4 zero block pointers.
- **Recovery regimes:** (1) **metadata undelete** (directory/MFT entry survives); (2) **file carving** — ignore FS, scan unallocated space for **header/footer signatures** (JPEG `FF D8 FF` … `FF D9`); limits: fragmentation, **no name/path/timestamps**, overwritten data gone, **TRIM SSDs**; tools PhotoRec, Scalpel, Foremost, Autopsy; (3) **journal/shadow copies** — `$LogFile`, `$UsnJrnl`, Volume Shadow Copies.
- **Hidden data:** Hidden/System attribute, Linux dot-files · **NTFS Alternate Data Streams (ADS)** — invisible in normal listing, lost when copied to FAT · **slack space** (file/RAM slack) and unallocated · **HPA/DCO** · partition gaps, `$BadClus` · **steganography** · **renamed extensions** (caught by signature analysis).
- **Altered files:** hash mismatch vs known-good · **timestamp anomalies / timestomping** — NTFS `$STANDARD_INFORMATION` vs `$FILE_NAME` MACE mismatch, created later than modified, inconsistent with `$UsnJrnl`/event logs · EXIF inconsistencies (editing software field) · data appended after footer (`%%EOF`).

## Likely questions with model answers (Unit 1)

**Q. Classify cyber crimes with examples.** *(5 marks · Likelihood 9/10)*
Cyber crime is any offence in which a computer or network is the target, the instrument, or the store of evidence. **By role of the computer:** (i) *computer as target* — hacking, DoS/DDoS, ransomware, defacement; (ii) *computer as tool* — phishing, online fraud, identity theft, cyber-stalking, CSAM distribution; (iii) *computer as incidental* — a fraudster's chat logs or a dealer's client spreadsheet; (iv) crimes from the prevalence of computers — piracy, counterfeit hardware. **By victim:** (i) *against person* — stalking, bullying, morphing, sextortion (IT Act §66C/§66D/§66E); (ii) *against property* — card/UPI fraud, data theft, IP theft (§43, §66); (iii) *against government* — attacks on critical infrastructure, cyber-terrorism (§66F); (iv) *against society* — obscenity, CSAM, fake news (§67/§67A/§67B).

**Q. Differentiate internal and external attacks.** *(5 marks · Likelihood 8/10)*
An **internal attack** is by an insider — employee, contractor or ex-employee — who already holds legitimate access and knows where valuable data lies; an **external attack** is by an outsider who must first gain access through exploits, phishing or brute force. Insider activity looks authorised, so detection is hard; outsider intrusions are usually logged by firewalls and IDS. Insider motives are grievance, greed or negligence; outsiders seek money, ideology or notoriety. Insider evidence:

access beyond role, off-hours activity, mass downloads/USB exfiltration, deleted logs; outsider evidence: IDS/firewall logs, malware, foreign IPs, new backdoor accounts. Defences: least privilege, separation of duties, DLP and log review (internal) vs firewall, IDS/IPS, patching and MFA (external). Insiders are harder because they bypass perimeter defences by design.

**Q. Explain ATM and banking frauds and how they are reported.** *(5 marks · Likelihood 8/10)*
**Skimming:** a hidden reader on the ATM slot copies magnetic-stripe data while a pinhole camera/overlay keypad captures the PIN. **Cloning:** that data is written to a blank card; EMV chips resist cloning because each transaction carries a fresh cryptogram. **Phishing / vishing / smishing:** fake email-website / phone call / SMS that extracts card details, CVV, OTP. **SIM swap:** criminal gets a duplicate SIM, intercepts OTPs and drains accounts — traced via telco SIM-change logs. **UPI frauds:** fake collect requests that debit, QR codes (scanning pays, never receives), remote-access apps stealing the UPI PIN, fake customer care. Evidence: CCTV, transaction logs, UTR/RRN, VPA, mule-account KYC, CDRs. Victims must call **1930** or file at **cybercrime.gov.in** at once so the money can be frozen; never share OTP/PIN/CVV.

**Q. What is packet sniffing? Distinguish passive and active sniffing.** *(5 marks · Likelihood 7/10)*
Packet sniffing is capturing and decoding network packets by putting a NIC into promiscuous (or Wi-Fi monitor) mode so it accepts all frames, not just its own. **Passive sniffing** works on hubs/shared media — every port sees all traffic, so the attacker just listens and is hard to detect. **Active sniffing** is needed on switches, which forward frames only to the destination port; the attacker injects traffic — **ARP poisoning** to become a man-in-the-middle, or **MAC flooding** to overflow the CAM table so the switch broadcasts. Tools: Wireshark, tcpdump, Zeek. Legitimately, investigators use sniffing to capture PCAP evidence. Defences: TLS/SSH/VPN encryption, port security, Dynamic ARP Inspection. Encryption hides content but not metadata.

**Q. Explain types of spoofing.** *(5 marks · Likelihood 8/10)*
Spoofing is forging an identity field so traffic appears to come from someone else. **IP spoofing** forges the source IP (L3) for DDoS reflection or hiding origin — countered by ingress filtering (BCP 38). **MAC spoofing** forges the NIC address to bypass MAC filters — port security/802.1X. **ARP spoofing** poisons IP↔MAC caches on a LAN for MITM — Dynamic ARP Inspection, static ARP. **DNS spoofing/cache poisoning** returns a false IP to redirect users to fake sites (pharming) — DNSSEC. **Email spoofing** forges the From/envelope sender for phishing/BEC — SPF, DKIM, DMARC. **Caller-ID/SMS spoofing** fakes the displayed number for vishing/smishing. Forensically, compare forged fields with fields recorded by the receiving system (e.g. Received headers).

**Q. Role of the first responder and contents of the toolkit.** *(5 marks · Likelihood 9/10)*
The first responder is the first trained person at a digital crime scene; the role is to preserve, not analyse. Duties: secure and isolate the scene; assess power state; avoid touching keyboard/mouse; photograph screen, cabling and surroundings; capture volatile data if trained; identify all devices including phones, NAS and cloud sessions; isolate from network/Faraday-bag phones to prevent remote wipe; label, seal and begin chain of custody; never open files on the suspect system; hand over with notes. **Toolkit:** camera, forms, chain-of-custody sheets, labels; anti-static and Faraday bags, evidence tape; hardware write blockers, forensic duplicator, sterile wiped target drives; trusted IR USB with memory-capture tools; bootable forensic OS, FTK Imager/dd, hashing tools; torch, screwdrivers, gloves, UPS/portable power.

**Q. Explain SQL injection, XSS and CSRF with prevention.** *(10 marks · Likelihood 7/10)*
**SQL injection** occurs when user input is concatenated into a SQL statement, so input becomes code. Entering `' OR '1'='1` in a login field yields `WHERE user='' OR '1'='1'`, always true, bypassing authentication; `UNION SELECT` extracts other tables; blind/time-based variants infer data from response timing. Footprint: web logs containing `%27`, `UNION`, `--`, `information_schema`, bursts of HTTP 500. Prevention: parameterised queries/prepared statements, input validation, least-privilege DB

accounts, WAF.
**Cross-site scripting (XSS)** injects client-side script that runs in other users' browsers under the trusted site's origin. *Stored* XSS persists on the server (e.g. a comment); *reflected* XSS bounces from a crafted link; *DOM-based* XSS abuses client-side JavaScript sinks. It steals cookies/session tokens, logs keystrokes or defaces pages. Footprint: `<script>`, `onerror=`, `javascript:` in content or requests, outbound calls to attacker servers. Prevention: context-aware output encoding, Content-Security-Policy, `HttpOnly` cookies, input validation.
**Cross-site request forgery (CSRF)** tricks a victim who is already logged in into sending an unintended state-changing request (e.g. a hidden auto-submitting transfer form on a malicious page); the browser attaches the session cookie automatically, so the server cannot distinguish it. Footprint: authenticated request with a foreign Referer/Origin. Prevention: anti-CSRF tokens, `SameSite` cookies, re-authentication for sensitive actions.
**Comparison:** XSS abuses the site's trust in the browser; CSRF abuses the site's trust in the authenticated user. XSS can defeat CSRF tokens.
**HTTPS** (HTTP over TLS) adds encryption, server authentication via certificates and integrity — defeating sniffing, session theft over the wire and spoofed sites — but it does not fix application bugs like SQLi or XSS. Session cookies should be `Secure`, `HttpOnly`, `SameSite`, with the ID rotated at login.

**Q. Explain search and seizure of volatile and non-volatile evidence. When would you pull the plug?** *(10 marks · Likelihood 9/10)*
**Volatile evidence** is lost when power is removed or with time: CPU registers/cache, ARP/routing/process tables, live network connections, RAM (running processes, open files, clipboard, encryption keys, fileless malware), swap/temp files. **Non-volatile evidence** persists: hard disks, SSDs, USBs, memory cards, optical and backup media.
**Procedure:** secure and photograph the scene; assess power state. **If OFF:** keep it off — booting alters hundreds of files — seize the device and image it in the lab through a write blocker (dead-box/static acquisition). **If ON/sleeping:** a trained responder captures volatile data first in RFC 3227 order of volatility using a trusted IR USB (memory dump, process and connection lists), documents every action (ACPO Principle 2), and isolates the system from the network to prevent remote wipe.
**Power-down choice:** *Pull the plug* freezes the disk and defeats shutdown wipers but loses RAM and may corrupt the FS; *graceful shutdown* keeps the FS clean but runs shutdown scripts that may clear pagefile, rotate logs or fire logic bombs. A desktop workstation at risk of anti-forensic scripts → pull the plug after RAM capture. A server/database/RAID → controlled shutdown or live imaging.
**Unlocked encrypted volume** → do a live logical acquisition or extract keys from RAM before power-off, else access may be lost forever.
**Comparison:** live acquisition captures RAM, network state and decrypted volumes but alters system state and a rootkit may hide data; dead-box imaging is clean and repeatable but loses all volatile data. Modern practice combines both: RAM live first, then dead-box disk imaging.
**Then:** label and seize all media (phones in Faraday bags), hash at acquisition, seal and record in chain of custody, transport away from magnets/heat, and ensure the §65B IEA / §63 BSA certificate accompanies the record in court. (Diagram: live vs dead-box decision flow above.)

**Q. Procedure for search and seizure of digital evidence, including volatile evidence, imaging and hashing.** *(15 marks · Likelihood 9/10)*
**Introduction.** Digital evidence is latent, fragile and easily altered; one careless boot changes it. Search and seizure must therefore be lawful, witnessed and documented, follow the ACPO principles (no change to data; competence when original must be touched; audit trail; responsibility of officer in charge) and end in a §65B IEA / §63 BSA certificate for admissibility. Powers flow from BNSS 2023 and IT Act (e.g. §80) — ⚠️ cite section numbers only after verifying.
**1. Planning and authority.** Identify the target, expected devices, encryption and remote-wipe risk; obtain the warrant/authority; assemble the first-responder toolkit (write blockers, sterile drives,

Faraday bags, camera, forms, IR USB, UPS).
**2. Securing the scene.** Remove persons from keyboards and power; log entry/exit; photograph screens, cabling, serial numbers, peripherals and nearby notes/passwords; sketch the layout.
**3. Assess power state and capture volatile evidence.** If off, leave off. If on, capture in **order of volatility (RFC 3227)**: registers/cache → routing/ARP/process tables and connections → RAM → swap/temp files → disk → remote logs → physical topology → archival media. Use trusted, statically-linked tools; record every command and time. Isolate from network to stop remote wiping. Decide power-down: pull plug for a workstation at risk of shutdown wipers; controlled shutdown/live imaging for servers or unlocked encrypted volumes.
**4. Identification and seizure.** Label and seize all storage — computers, phones (Faraday bag), USBs, SD cards, external drives, routers, NAS, cameras, notes, chargers. Prepare a seizure memo/panchnama signed by independent witnesses.
**5. Imaging.** Connect the evidence drive through a **hardware write blocker**. Hash the source (MD5 + SHA-256). Create a **bit-stream image** (copies allocated, unallocated, slack, HPA/DCO — enabling deleted-file recovery) — not a logical copy. Prefer **E01** (compression, case metadata, CRC + embedded hash, self-verifying) or raw **dd** (no metadata; store hash separately). Imaging (to a file) is preferred over cloning (disk-to-disk).
**6. Hashing and verification.** Hash the image; source hash = image hash proves an exact copy. Record hashes, tool versions, examiner and times in notes and chain-of-custody forms. All analysis on a working copy; re-hash before and after. Hash change = data changed (unless explained, e.g. SSD garbage collection).
**7. Packaging, chain of custody and transport.** Anti-static bags, tamper-evident seals with seal numbers, custody log of every transfer; transport away from heat and magnets.
**8. Lab analysis and reporting.** Recover deleted/hidden files (carving, slack, ADS), check timestamps, prepare report and §65B/§63 certificate.
**Conclusion.** Volatile-first capture, write-blocked bit-stream imaging and hash verification together ensure the evidence presented in court is exactly what was seized — authentic, complete and reliable. (Diagrams: search & seizure flow; live vs dead-box decision; imaging + hashing.)

# Unit 8 — Digital Evidences

## Digital evidence and storage devices

**Importance 8/10 · likely** · opening phrase of the unit; characteristics list is a common 5-marker

In simple words: digital evidence is any information of probative value stored or transmitted in binary form — a silent witness that needs tools to be seen.

- **Characteristics:** latent · fragile/volatile · easily duplicated (bit-for-bit) · alterable & time-sensitive · crosses jurisdictions · voluminous · machine/format dependent.
- **Admissibility needs:** authentic (hash + chain of custody), reliable, complete, lawfully obtained + **§65B IEA / §63 BSA certificate** (⚠️ verify BSA sub-section).
- **Storage devices:** HDD, SSD, USB flash, SD/microSD, eMMC/UFS (phones), optical (CD/DVD/Blu-ray), magnetic tape (LTO), RAM, firmware/NVRAM, RAID/NAS/SAN, cloud. SSD TRIM can destroy deleted data on its own.

## Order of volatility

**Importance 10/10 · sure to come** · unique forensic phrase; list + RFC 3227 citation

In simple words: collect what disappears fastest first, because collecting the stable stuff (or time itself) destroys the fragile stuff.

Source: **IETF RFC 3227 "Guidelines for Evidence Collection and Archiving"**. Most → least volatile:

1. CPU registers, cache
2. Routing table, ARP cache, process table, kernel statistics, network connections
3. Main memory (RAM) — keys, processes, fileless malware
4. Temporary file systems (swap/pagefile, /tmp)
5. Disk / non-volatile storage
6. Remote logging and monitoring data
7. Physical configuration, network topology
8. Archival media (backups, tapes)

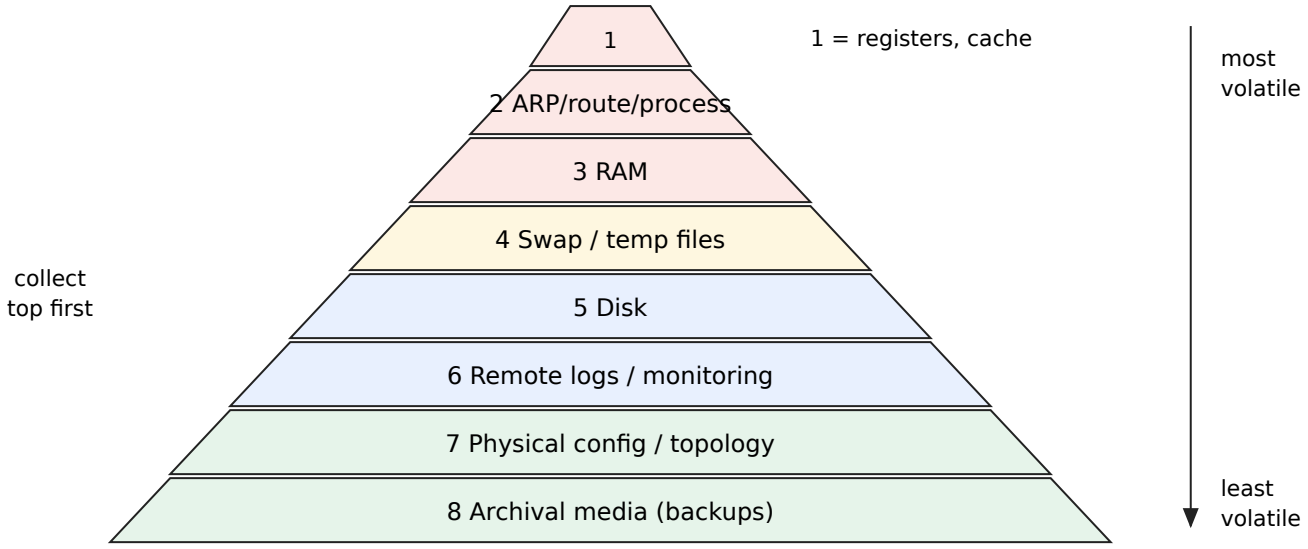



*Fig: Order of volatility pyramid (RFC 3227) — collect from the top down*

## Cybercrime and types (cross-reference)

**Importance 5/10 · possible** · repeated in Unit 8 syllabus; answer from Unit 1

In simple words: same classification as Unit 1 — target / tool / incidental; against person / property / government / society. Link each type to its evidence: hacking → logs and malware; fraud → transaction trails; stalking → messages and account attribution.

## Cyber forensics — definition and basic rules (ACPO principles)

**Importance 10/10 · sure to come** · "basic rules for cyber forensics" — near-certain

In simple words: cyber forensics = identifying, preserving, collecting, examining, analysing and presenting digital evidence in a way a court will accept.

| # | ACPO principle (UK Association of Chief Police Officers) | In practice |
|---|---|---|
| 1 | **No action should change data** that may be relied on in court | Write blockers; work on images; never boot evidence |
| 2 | If original data **must** be accessed, the person must be **competent** and able to **explain relevance and implications** | Justified live acquisition, documented |
| 3 | An **audit trail** of all processes must be kept; an **independent third party** should reproduce the result | Notes, tool versions, hashes → reproducibility |
| 4 | The **person in charge** has overall responsibility for compliance | Accountability |

Process model: **Identification → Preservation → Collection → Examination → Analysis → Presentation** (documentation & chain of custody throughout; NIST SP 800-86 condenses to

Collection, Examination, Analysis, Reporting ⚠️ verify doc number). Also name **RFC 3227**, **ISO/IEC 27037** (⚠️ verify).

## Collection and preservation of digital evidence

**Importance 8/10 · likely** · syllabus phrase

In simple words: grab it without changing it, and keep proof it never changed.

- **Collection:** follow order of volatility; live capture of RAM if ON; seize all media; bit-stream imaging via write blocker; hash at acquisition.
- **Preservation:** isolate scene & network (Faraday for phones), write blockers, imaging with hashing, analysis only on copies, tamper-evident seals, anti-static packaging, **chain of custody** (who, when, where, why for every transfer), secure storage away from magnets/heat.

## Data acquisition methods

**Importance 9/10 · very likely** · syllabus phrase; two tables

In simple words: how much you copy (physical/logical/sparse) and whether the system is running (live/static).

| Method | Captures | Trade-off |
|---|---|---|
| **Physical (bit-stream)** | Every sector incl. unallocated, slack, HPA/DCO | Forensic default; recovers deleted data; largest/slowest |
| **Logical** | Only active files visible to FS | Fast/small; **no deleted data** |
| **Sparse** | Selected data + relevant deleted/unallocated fragments | Middle ground for huge systems |
| Targeted / triage | Specific artifacts on scene | Quick prioritisation |

| | Live (dynamic) | Static (dead-box) |
|---|---|---|
| System | Running | Powered off |
| Gets | RAM, processes, network state, unlocked encrypted volumes, cloud sessions | Full disk image, repeatable |
| Risk | Alters state (justify ACPO P2); rootkit may lie | Loses volatile evidence |

## Forensic imaging vs cloning

**Importance 8/10 · likely** · "imaging & cloning" syllabus pair — classic differentiate

In simple words: an image is a file representing the disk; a clone is a second physical disk.

| Aspect | Imaging | Cloning |
|---|---|---|
| Output | **File(s)** — raw/dd, E01, AFF | **Disk-to-disk** copy on another physical drive |
| Metadata/hash | E01 embeds metadata + hash; easy to hash & store | Harder to hash, store and catalogue |
| Compression / segments | Yes (E01) | No |
| Bootable | No (mount/analyse in tools) | Yes — usable as working/test environment |
| Evidence use | **Preferred** | Building a test system / reconstructing environment |

Both must be through a write blocker and verified by hash; target drive must be **sterile (wiped & verified)** for cloning.

## Data validation

**Importance 8/10 · likely** · syllabus phrase

In simple words: prove the copy equals the original and has stayed unchanged.

- Hash the source (through write blocker) → acquire → hash the image → **must match**.
- Re-verify before and after analysis; compute **two algorithms** (MD5 + SHA-256).
- E01 adds **CRC per block + stored hash** → self-verifying; raw dd needs separately stored hash.
- Mismatch must be investigated and explained (e.g. SSD garbage collection on a live SSD).

## Hash algorithms and hash value

**Importance 9/10 · very likely** · digest sizes are favourite MCQ + 5-mark

In simple words: a hash is a fixed-length digital fingerprint of data; change one bit and the fingerprint changes completely.

- **Properties:** one-way (pre-image resistant) · fixed-length output · deterministic · **avalanche effect** · collision resistance (hard to find two inputs with same hash) · second-pre-image resistance.

| Algorithm | Digest | Hex chars | Status |
|---|---|---|---|
| MD5 | 128-bit | 32 | Collision-broken; OK vs accidental change |
| SHA-1 | 160-bit | 40 | Collision-broken (2017) |
| SHA-256 (SHA-2) | 256-bit | 64 | Recommended |
| SHA-512 / SHA-3 | 512 / variable | — | Strong; SHA-3 = Keccak sponge |

- **Uses:** integrity verification (digital seal), de-duplication, **hash-set filtering** — NSRL known-good hashes to exclude OS files; known-bad sets to flag contraband (e.g. CSAM) without viewing.
- **Is MD5 still OK?** An engineered collision needs both files crafted; it does **not** let anyone alter seized evidence to match a given hash (second pre-image — still infeasible). Still, use dual hashes.

## File extensions and file signatures

**Importance 10/10 · sure to come** · guaranteed MCQ + "detect a file hiding its type" answer

In simple words: the extension is just part of the name (easily renamed); the signature (magic bytes in the header) reveals the true type.

| Type | Extension | Header (hex) | ASCII | Footer |
|---|---|---|---|---|
| JPEG | .jpg/.jpeg | `FF D8 FF` | ÿØÿ | `FF D9` |
| PNG | .png | `89 50 4E 47 0D 0A 1A 0A` | .PNG | IEND chunk |
| GIF | .gif | `47 49 46 38 37 61` / …`39 61` | GIF87a / GIF89a | `00 3B` |
| PDF | .pdf | `25 50 44 46` | %PDF | `%%EOF` |
| ZIP / DOCX / XLSX / PPTX / APK / JAR | .zip etc. | `50 4B 03 04` | PK.. | — |
| EXE / DLL (Windows PE) | .exe/.dll | `4D 5A` | MZ | — |
| ELF (Linux) | — | `7F 45 4C 46` | .ELF | — |
| Legacy Office (.doc/.xls/.ppt) | — | `D0 CF 11 E0 A1 B1 1A E1` | OLE | — |
| RAR | .rar | `52 61 72 21 1A 07` | Rar! | — |

**Signature analysis** compares header bytes with the extension → flags **mismatch/masquerading** (e.g. `holiday.jpg` beginning `4D 5A` is really an executable). Same signatures drive file carving.

## Windows registry overview

**Importance 7/10 · good chance** · syllabus phrase; key list scores

In simple words: the registry is Windows' configuration database — and an unintentional activity log.

- Hives: SYSTEM, SOFTWARE, SAM, SECURITY (`Windows\System32\config`), per-user NTUSER.DAT.
- **USBSTOR / MountedDevices / MountPoints2** — USB device history + which user.
- **UserAssist** — GUI program execution (ROT-13 encoded). **ShellBags** — folders browsed.
- **RecentDocs / RunMRU / TypedPaths / WordWheelQuery** — recent files, Run commands, typed paths, search terms.
- **Run / RunOnce / Services** — persistence (malware autostart).
- **TimeZoneInformation** — interpret all timestamps correctly.
- Keys carry a **LastWrite** time; values do not.

## Data recovery techniques

**Importance 7/10 · good chance** · overlaps Unit 1 recovery

In simple words: get back deleted data from metadata, raw content, or old snapshots.

- **Undelete** (metadata survives: FAT 0xE5 entry, unallocated NTFS MFT record).
- **File carving** (signature-based, no metadata; fails with fragmentation; loses names/timestamps).
- **Journal / shadow copies:** `$LogFile`, `$UsnJrnl`, **Volume Shadow Copies**.
- Slack and unallocated space analysis; deleted-partition recovery (backup GPT).
- **SSD TRIM** often destroys deleted data autonomously — recovery may be impossible.

## Windows disks, files and partitions — MBR vs GPT

**Importance 8/10 · likely** · "guaranteed compare" in source

In simple words: the partition table is the disk's map — MBR is the old 1-sector map, GPT the modern redundant one.

| Aspect | MBR | GPT |
|---|---|---|
| Firmware | Legacy BIOS | UEFI |
| Location | **LBA 0, 512 bytes**: 446 B boot code + 64 B table (4×16) + `55 AA` | Header at **LBA 1** + entries; **protective MBR at LBA 0** |
| Partitions | **4 primary** (or 3 + 1 extended → logical) | **128** typical |
| Max disk | **~2 TB** (32-bit LBA × 512 B) | ~9.4 ZB (64-bit LBA) |
| Redundancy | None | **Backup header + table at disk end**, **CRC32** |
| IDs | 1-byte type code | GUIDs |

- **Partition** = slice of the physical disk; **volume** = a formatted, mountable storage area with a file system (may span partitions, e.g. dynamic/spanned volumes).
- Windows file systems: **NTFS** (MFT, ADS, journal), FAT32/exFAT (removable media).
- Hiding places: inter-partition gaps, boot-record slack, file/RAM/volume slack, unallocated space.




MBR = sector 0 (LBA 0) = 512 bytes = 446 + 64 + 2

*Fig: Layout of the Master Boot Record (512-byte sector 0)*

## Email tracing techniques

**Importance 9/10 · very likely** · unique forensic topic; annotated header earns marks

In simple words: read the email's hidden header lines from the bottom up to find the real sending IP, then check authentication.

**Method:** (1) view full/original headers; (2) read `Received:` **bottom → top** — bottom-most = origin; (3) extract originating IP → WHOIS/geolocate; (4) compare **From vs Return-Path vs Reply-To vs Message-ID domain**; (5) check **SPF/DKIM/DMARC** results; (6) legal process to ISP/provider for subscriber; preserve with hash + §65B/§63.

| Field | Forensic use |
|---|---|
| `Received:` (one per hop) | Trace backbone; `(host [IP])` is the receiving server's own observation — harder to forge |
| `From:` | Displayed sender — easily spoofed |
| `Return-Path:` | Envelope sender (MAIL FROM); mismatch with From = suspicious |
| `Reply-To:` | Phishers point it to attacker |
| `Message-ID:` | Domain should match sender's server |
| `Authentication-Results:` / `Received-SPF:` | spf/dkim/dmarc = pass/fail |
| `X-Originating-IP:` | Some webmail records client IP |
| `Date:`, `X-Mailer:` | Timeline; client software |

| Mechanism | Checks |
|---|---|
| **SPF** | Is the sending server's IP authorised for the envelope domain? (DNS TXT list) |
| **DKIM** | Message cryptographically signed by domain and unaltered? (public key in DNS) |
| **DMARC** | Do SPF/DKIM **align with visible From** domain; policy none/quarantine/reject |

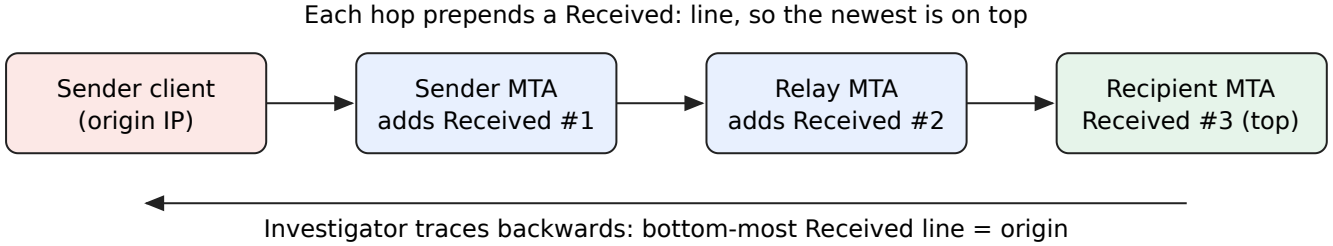



*Fig: Email delivery chain and tracing via Received headers (bottom-most = origin)*

Spoof indicators: spf/dkim/dmarc **fail** + From domain ≠ Return-Path/authenticated domain + origin IP not in sender's network.

## Social media analysis

**Importance 6/10 · moderate** · syllabus phrase; short answer

In simple words: collect posts/messages defensibly, get account data from the platform, and examine the suspect's device.

- **Open-source capture:** screenshot + URL + timestamp + **hash**, page source/web archive.
- **Legal process** to platform: subscriber info, creation IP, login history, content (law-enforcement portal / MLAT).
- **Device forensics:** app SQLite databases on phone/PC.
- Analyse: account attribution, timeline, link/network analysis of contacts, EXIF of images, forwarding chains.

## Steganography and steganalysis

Importance 9/10 · very likely · unique forensic topic

In simple words: steganography hides the **existence** of a message; cryptography hides its **meaning**.

| | Steganography | Cryptography |
|---|---|---|
| Goal | Hide existence | Hide meaning |
| Visibility | Message not noticed | Ciphertext visible but unreadable |
| Best practice | **Encrypt then hide** | |

- **Techniques: LSB substitution** in image pixels/audio samples (classic; changes invisible); DCT-coefficient embedding in JPEG; text (whitespace, word choice); network covert channels; **appending data after a valid footer** (ZIP after JPEG `FF D9`, `copy /b img.jpg + secret.zip`); metadata fields.
- **Steganalysis:** signature/known-tool artifacts · **statistical** (chi-square, RS analysis — LSB embedding disturbs pixel statistics) · file size/structure anomalies (data after footer, odd palette) · comparison with known clean original · extraction/brute force once tool identified.
- Tools: specific stego tools leave detectable signatures (known-tool detection); forensic suites (Autopsy/EnCase/FTK) help flag size/structure anomalies such as data after a footer. ⚠️ No named stego tools in source notes.

**Indian law (only as in sources, ⚠️ verify):** IT Act §43, §65, §66, §66C–F, §67/A/B; admissibility via **§65B IEA 1872 → §63 BSA 2023 certificate** (cite both); expert opinion §45 IEA → §39 BSA (⚠️ verify).

## Likely questions with model answers (Unit 8)

**Q. What is order of volatility? List it.** *(5 marks · Likelihood 9/10)*
Order of volatility is the principle that digital evidence must be collected from the most volatile (shortest-lived) to the least volatile, because collecting stable evidence — or simply the passage of time — destroys volatile evidence. It comes from **IETF RFC 3227**, "Guidelines for Evidence Collection and Archiving". Order: (1) CPU registers and cache; (2) routing table, ARP cache, process table, kernel statistics, network connections; (3) main memory (RAM) — running processes, encryption keys, fileless malware; (4) temporary file systems — swap/pagefile, /tmp; (5) disk; (6) remote logging and monitoring data; (7) physical configuration and network topology; (8) archival media/backups. Practically, the first responder dumps RAM and network state before deciding to power down, then images the disk. (Draw the pyramid.)

**Q. State the basic rules (ACPO principles) of cyber forensics.** *(5 marks · Likelihood 9/10)*
The UK ACPO Good Practice Guide for Digital Evidence gives four principles. **(1)** No action taken should change data held on a computer or storage medium that may be relied upon in court — so use write blockers and work only on verified images. **(2)** Where a person must access original data, they must be competent and able to explain the relevance and implications of their actions — e.g. justified live RAM capture. **(3)** An audit trail of all processes applied must be created and preserved, so an independent third party can repeat them and reach the same result — notes, tool versions,

hashes, chain of custody. **(4)** The person in charge of the investigation has overall responsibility for ensuring the law and these principles are followed. Together with hashing, chain of custody and the §65B IEA/§63 BSA certificate they make evidence admissible.

**Q. Differentiate forensic imaging and cloning.** *(5 marks · Likelihood 8/10)*
Both create an exact bit-for-bit copy of a source drive through a write blocker, verified by hash. **Imaging** writes the copy as a file or set of files (raw/dd, E01, AFF) on a target drive; E01 adds compression, segmenting, case metadata and embedded CRC/hash, making storage, cataloguing and verification easy, and several images fit on one drive. **Cloning** writes the copy disk-to-disk onto another physical drive, producing a bootable duplicate useful for recreating the suspect's working environment, but it is harder to hash, store and manage as an exhibit, and the target must be forensically wiped (sterile) beforehand so old data does not contaminate it. Imaging is therefore preferred for evidence; cloning for testing or environment reconstruction.

**Q. What is a hash value? Name hash algorithms with digest sizes.** *(5 marks · Likelihood 9/10)*
A hash value is the fixed-length output of a cryptographic one-way function applied to data; it acts as the data's digital fingerprint or seal. Properties: deterministic, fixed-length, one-way, collision resistant, and **avalanche effect** — changing one bit changes the digest completely. **MD5** gives 128 bits (32 hex), **SHA-1** 160 bits (40 hex), **SHA-256** 256 bits (64 hex); SHA-512 and SHA-3 are also used. MD5 and SHA-1 have practical collisions, so SHA-256 is recommended and examiners often record two hashes. Uses: verifying the image equals the source and is unchanged later; de-duplication; filtering known-good files (NSRL) and flagging known-bad files without viewing them.

**Q. What are file signatures? How do they detect a disguised file?** *(5 marks · Likelihood 9/10)*
A file **extension** is part of the filename and can be changed freely; a **file signature** (magic number) is a fixed byte sequence at the start of the file that identifies its real format. Examples: JPEG `FF D8 FF`, PNG `89 50 4E 47`, GIF `47 49 46 38` (GIF87a/89a), PDF `25 50 44 46` (%PDF), ZIP/DOCX `50 4B 03 04` (PK), EXE `4D 5A` (MZ). A suspect may rename `plan.exe` to `plan.jpg` to hide it. **Signature analysis** (in EnCase/FTK/Autopsy) reads each header and compares it with the extension; a mismatch — a .jpg starting `4D 5A` — is flagged for examination. Header/footer signatures (JPEG `FF D9`, PDF `%%EOF`) also drive file carving of deleted files.

**Q. How are emails traced? Explain header analysis and SPF/DKIM/DMARC.** *(5 marks · Likelihood 8/10)*
Open the full original headers. Each mail server that handles the message prepends a `Received:` line, so read them **bottom to top**: the bottom-most shows the originating host and IP. Look up that IP (WHOIS/geolocation) and compare with the claimed sender. Compare `From:` with `Return-Path:` (envelope sender), `Reply-To:` and the `Message-ID:` domain — mismatches indicate spoofing. Check `Authentication-Results:`: **SPF** verifies that the sending IP is authorised in the domain's DNS record; **DKIM** verifies a cryptographic signature proving the domain signed it and it was unaltered; **DMARC** checks that SPF/DKIM align with the visible From domain and sets a reject/quarantine policy. Finally obtain subscriber data from the ISP/provider by legal process and preserve the email with hashes and a §65B/§63 certificate.

**Q. Explain data acquisition methods.** *(10 marks · Likelihood 8/10)*
Data acquisition is the process of creating a forensically sound copy of digital evidence. It is classified two ways.
**By completeness:** (1) **Physical / bit-stream acquisition** copies every sector — allocated files, unallocated space, file slack, HPA/DCO — so deleted files can be recovered and carved; it is the forensic default but the slowest and largest. (2) **Logical acquisition** copies only the active files and folders the file system reports; faster and smaller, used for very large servers or authorised subsets, but cannot recover deleted data. (3) **Sparse acquisition** collects selected relevant files plus related deleted/unallocated fragments without imaging the whole device — a middle path for huge or

enterprise systems. (4) **Targeted/triage** captures specific artifacts on scene to set priorities.
**By system state:** (1) **Live (dynamic) acquisition** on a running system captures RAM, running processes, network connections, logged-in cloud sessions and unlocked encrypted volumes; essential when disks are encrypted, malware is fileless or a critical server cannot be shut down; but it alters system state (must be justified and documented under ACPO Principle 2) and a rootkit may hide data. (2) **Static (dead-box) acquisition** images a powered-off drive through a write blocker; clean and repeatable but loses all volatile data. Modern practice: RAM live first, then dead-box imaging.
**Formats and validation:** images are stored as raw/dd (no metadata), E01 (compression, metadata, CRC + embedded hash) or AFF. Every acquisition is validated by hashing source and image (MD5 + SHA-256) and recording both in the chain of custody. Choice of method depends on case scope, device size, encryption, time and legal authority.

**Q. Compare MBR and GPT; explain partitions and volumes.** *(10 marks · Likelihood 7/10)*
A partition table maps how a physical disk is divided. **MBR (Master Boot Record)** is the legacy BIOS scheme stored in sector 0 (LBA 0), 512 bytes: 446 bytes bootstrap code, a 64-byte partition table of four 16-byte entries, and the `55 AA` boot signature. It supports four primary partitions (or three primary plus one extended containing logical drives) and disks up to about 2 TB (32-bit LBA × 512-byte sectors). It has no redundancy — a corrupt MBR loses the map — and is a target for bootkits. **GPT (GUID Partition Table)** is used with UEFI: a protective MBR at LBA 0 for backward compatibility, the GPT header at LBA 1, then partition entries; typically 128 partitions, 64-bit LBA (about 9.4 ZB); a **backup header and table at the end of the disk** and **CRC32** checks give integrity and recovery; partitions are identified by GUIDs.
A **partition** is a contiguous slice of the physical disk; a **volume** is a logical storage unit formatted with a file system (NTFS, FAT32, exFAT) and given a drive letter, possibly spanning partitions/disks.
**Forensic points:** image the whole disk so MBR/GPT and gaps are captured; recover deleted partitions from the backup GPT; compare primary and backup GPT to detect tampering; examine inter-partition gaps, boot-record slack, volume slack and unallocated space for hidden data; identify partitions by GUIDs. (Draw the MBR layout.)

**Q. Explain digital evidence: order of volatility, acquisition, imaging, hashing and validation.** *(15 marks · Likelihood 9/10)*
**Digital evidence** is information of probative value stored or transmitted in binary form. It is latent, fragile, easily duplicated, easily altered, time-sensitive, voluminous and may cross jurisdictions. It lives on HDDs, SSDs, USB drives, memory cards, phones (eMMC/UFS), optical and tape media, RAM, RAID/NAS and the cloud. To be admissible it must be authentic, reliable, complete and lawfully obtained, with the **§65B IEA / §63 BSA certificate** (⚠️ verify sub-section). The ACPO principles govern handling: no change to data; competence when original must be touched; audit trail reproducible by a third party; responsibility of officer in charge.
**1. Order of volatility (RFC 3227).** Collect most volatile first: registers/cache → routing, ARP, process tables, network connections → RAM → swap/temp files → disk → remote logs → physical topology → archival media. Reason: collecting stable evidence or time itself destroys fragile evidence. So a running system's RAM is dumped before the power decision. (Draw pyramid.)
**2. Acquisition.** Physical (bit-stream: all sectors incl. unallocated, slack, HPA/DCO), logical (active files only, no deleted data) or sparse (selected data plus relevant fragments). Live acquisition captures RAM, network state and unlocked encrypted volumes but changes state; static acquisition of a powered-off disk is clean and repeatable but loses volatile data. Off → keep off; on → volatile first, then pull plug (workstation) or controlled shutdown/live imaging (server, unlocked encryption).
**3. Imaging.** Connect the evidence via a **hardware write blocker** (reads allowed, writes blocked). Create a bit-stream image to a sterile target as raw/dd (no metadata, hash stored separately) or **E01** (compression, case metadata, CRC per block + embedded hash). Imaging (file) is preferred over cloning (disk-to-disk, bootable but hard to manage). Tools: FTK Imager, dd/dcfldd, EnCase.

**4. Hashing.** A hash is a fixed-length one-way fingerprint with avalanche effect. MD5 = 128-bit, SHA-1 = 160-bit, SHA-256 = 256-bit. MD5/SHA-1 have engineered collisions, but that does not allow altering seized evidence to match a given hash; best practice records MD5 + SHA-256. Hash sets (NSRL known-good, known-bad) speed analysis.
**5. Validation.** Hash source → hash image → match proves an exact copy. Record hashes, tool versions, times and examiner in notes and chain-of-custody forms. Work on a copy; re-hash before and after analysis. Any mismatch is investigated and explained (e.g. SSD garbage collection).
**6. Preservation and presentation.** Tamper-evident seals, anti-static/Faraday packaging, custody log of every transfer, secure storage; final report with methods, hashes and findings, plus §65B/§63 certificate.
**Conclusion.** Volatility-ordered collection, write-blocked bit-stream imaging and hash validation, governed by ACPO and chain of custody, make digital evidence authentic and reproducible in court.

# Unit 2 — Web Browsers, Email, VM & Cloud Forensics

## Browser artifacts — what each proves

**Importance 9/10 · very likely** · "browser artifacts" is a core forensic syllabus phrase — near-certain 5/10-mark
In simple words: the browser is a diary of what the user searched, visited, downloaded and logged into — it proves *intent*.

| Artifact | What it holds | What it proves |
|---|---|---|
| **History** | URLs, visit timestamps, visit count, transition type (typed / link / redirect) | Sites visited, when, how often, and *how* (typed = deliberate) |
| **Cache** | Copies of pages/images actually rendered | The **content** the user saw, even if the site later changed or went offline |
| **Cookies** | Small name–value files set by sites | Visits, logged-in sessions, tracking |
| **Favourites / Bookmarks** | Saved URLs (Chrome: JSON file) | Deliberate, durable interest — strong intent indicator |
| **Downloads list** | Source URL, save path, time, size | Ties a file on disk to its origin |
| **Session / tabs data** | Tabs open at last close or crash | Current activity at seizure |
| **Autofill, form data, saved logins** | Names, emails, addresses, credentials | Attribution |
| **Typed URLs / search terms** | What the user actively typed | Highest-intent artifact |
| **Plugins / extensions** | VPNs, ad-blockers, crypto wallets, malicious add-ons | Capability or anti-forensic indicator |
| **Sync data** | Activity from the user's other devices | Widens scope beyond the seized machine |

**Cookie types:**

| Type | Meaning |
|---|---|
| **Session cookie** | Held only while the browser runs; deleted on close — proves an *active* login session |
| **Persistent cookie** | Has an expiry date; stored on disk — survives restarts, proves repeat visits |
| **First-party** | Set by the site in the address bar (login, preferences) |
| **Third-party** | Set by another domain embedded in the page (ads/trackers) — shows cross-site tracking |

Exam tip: history = *where & when*, cache = *what was seen*, bookmarks/typed URLs = *intent*.

## Where browser data lives (Chrome / Firefox)

**Importance 8/10 · likely** · location + SQLite format is a standard short question and MCQ

In simple words: modern browsers keep their records in small SQLite database files inside the user's profile folder.

| Browser | Artifact files | Typical location (Windows) |
|---|---|---|
| **Chrome / Edge (Chromium)** | `History` (SQLite: URLs, visits, downloads), `Cookies` (SQLite), `Web Data` (autofill), `Login Data` (saved logins), `Bookmarks` (JSON), `Cache\` folder | `%LocalAppData%\Google\Chrome\User Data\Default\` |
| **Firefox** | `places.sqlite` (history + bookmarks), `cookies.sqlite`, `formhistory.sqlite`, `logins.json`, `cache2\` | `%AppData%\Mozilla\Firefox\Profiles\<random>.default\` |
| **Internet Explorer (legacy)** | `index.dat` (older), `WebCacheV*.dat` (ESE), registry `TypedURLs` | `%LocalAppData%\Microsoft\Windows\WebCache\` |
| **Safari** | `History.db` (SQLite) + plists | `~/Library/Safari/` (macOS) |

- Examiner works on a **copy**, runs SQL queries on `History` / `places.sqlite` to build a timeline.
- Timestamp trap: Chrome uses **WebKit time = microseconds since 1 Jan 1601**; Firefox uses **microseconds since the Unix epoch (1970)** — must be converted.

## Private / Incognito mode — limits

**Importance 8/10 · likely** · favourite "what survives?" question

In simple words: private mode stops the browser *saving* history/cookies/cache to disk — it does not make the user invisible.

| Still recoverable | Why |
|---|---|
| **RAM** (while open), pagefile, hiberfil | Private-session data lives in memory → **live memory capture** |
| **DNS cache** (`ipconfig /displaydns`) | OS still resolved the domains |
| **Router / proxy / firewall / ISP logs** | Network still carried the traffic |
| **Downloaded files** | Files kept on disk even though the download *list* is not |
| **OS artifacts** — prefetch, jump lists, temp files, crash dumps | Execution and activity traces |
| **Server-side / account records** | Websites and synced accounts log activity |

Exam tip: end with "the correct response to a suspected private session is **live RAM acquisition** before shutdown."

## Email — types, protocols and ports

**Importance 9/10 · very likely** · SMTP/POP3/IMAP with ports is a guaranteed MCQ and 5-mark

In simple words: SMTP pushes mail between servers; POP3 and IMAP let the user's client fetch it.

**Types of email:**

| Webmail (browser-based) | Client-based (desktop/mobile client) |
|---|---|
| Gmail, Outlook.com, Yahoo used in a browser over HTTPS | Outlook, Thunderbird, Apple Mail using POP3/IMAP/SMTP or Exchange |
| Mail stays on the provider's server | Mail may be downloaded to local files (`.pst/.ost` Outlook, `mbox` Thunderbird) |
| Evidence: provider records via legal request + browser artifacts/cache on the PC | Evidence: local mailbox files can be imaged and parsed directly |

| Protocol | Role | Port(s) |
|---|---|---|
| **SMTP** | **Send / relay** mail; each server adds a `Received:` header | **25** relay, **587** submission (STARTTLS), **465** SMTPS |
| **POP3** | Client **downloads** mail, usually **deletes** from server (one device) | **110**, **995** (TLS) |
| **IMAP** | Client **syncs**; mail **stays on server** (multi-device) | **143**, **993** (TLS) |
| **MIME** | Not a transport — format for **attachments / non-ASCII** (Base64) | — |

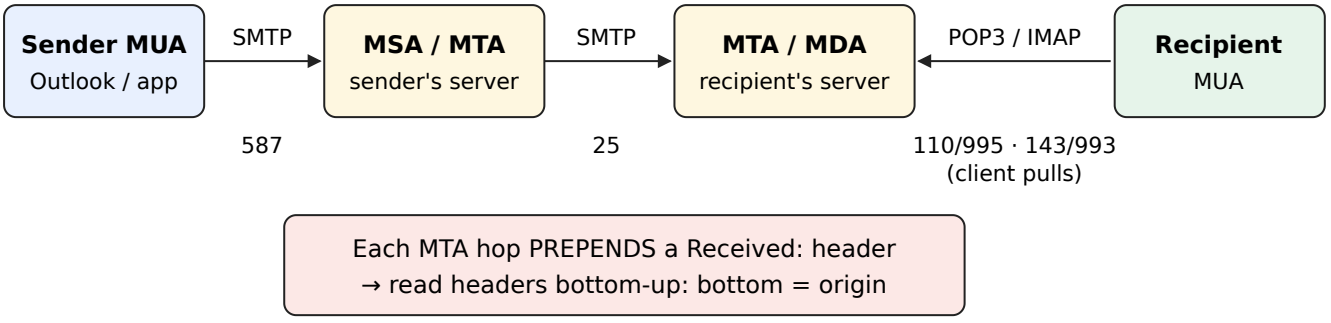



*Fig: Email flow — MUA → MSA/MTA → MTA/MDA → MUA, with protocols and ports*

## Analysing email headers and tracking email to its source

**Importance 10/10 · sure to come** · "email header tracing" is the set-piece forensic question

In simple words: every server a mail passes through stamps a `Received:` line on top; reading them bottom-up leads back to the sender's IP.

| Field | Meaning | Forensic use |
|---|---|---|
| **Received:** (one per hop) | `from <claimed> (<real host> [IP]) by <server>; date` | Trace backbone — **bottom-most = origin**; the bracketed IP is the receiving server's own observation (hard to forge) |
| **From:** | Displayed sender | Easily spoofed |
| **Return-Path:** | Envelope sender (SMTP `MAIL FROM`), where bounces go | Mismatch with `From:` = red flag |
| **Reply-To:** | Where replies go | Phishers point it at an attacker mailbox |
| **Message-ID:** | `<unique@sending-domain>` | Domain should match the real sending server |
| **X-Originating-IP:** | Client IP recorded by some webmail | May reveal the composing machine (not always present/trustworthy) |
| **Authentication-Results:** | Receiving server's SPF / DKIM / DMARC verdict | `fail` / `none` = spoof indicators |
| **Date:**, **X-Mailer:** | Compose time, client software | Timeline; missing/odd mailer suggests a script |

**Short annotated sample (illustrative):**

```
Received: from mail.trusted-bank.co ([203.0.113.9]) by mx.google.com  [3] last hop
(recipient side)
Received: from webmail.cheap-host.ru ([198.51.100.77])                [2] relay
        by mail.trusted-bank.co
Received: from [45.86.xx.xx] by webmail.cheap-host.ru with HTTP       [1] ORIGIN (read
bottom first)
Return-Path: <billing@cheap-host.ru>                  ≠ From domain  -> RED FLAG
From: "Trusted Bank" <security@trusted-bank.co>       displayed (spoofed)
Reply-To: <recover@secure-verify.info>                attacker inbox -> RED FLAG
Message-ID: <9f2c@cheap-host.ru>                      ≠ trusted-bank -> RED FLAG
Authentication-Results: spf=fail; dkim=none; dmarc=fail  -> SPOOFED
```

**Tracking to source — steps:** (1) open full/original headers (Gmail "Show original", Outlook "Internet headers"); (2) read `Received:` bottom-up; (3) take the originating IP from the bottom hop / `X-Originating-IP`; (4) WHOIS + geolocate the IP → ISP/host; (5) check From vs Return-Path vs Message-ID and SPF/DKIM/DMARC; (6) legal request to the ISP/mail provider for subscriber details and logs for that IP and time; (7) preserve the `.eml` with hash + §65B IEA / §63 BSA certificate (⚠️ verify sub-section before quoting).

## Spoofed mails and SPF / DKIM / DMARC

**Importance 9/10 · very likely** · spoofing indicators + the trio are a direct 5-mark

In simple words: a spoofed mail fakes the `From:`; SPF, DKIM and DMARC let the receiving server check whether the claim is genuine.

| Mechanism | Checks | Passes when |
|---|---|---|
| **SPF** (Sender Policy Framework) | Is the sending server's **IP authorised** for the envelope domain? (DNS TXT record) | Connecting IP is listed in the domain's SPF record |
| **DKIM** (DomainKeys Identified Mail) | Is the message **digitally signed** by the domain and **unaltered**? (public key in DNS) | Signature verifies |
| **DMARC** | Do SPF/DKIM **align with the visible From: domain**; policy `none` / `quarantine` / `reject` | Aligned SPF **or** DKIM pass |

**Spoof indicators:** From ≠ Return-Path / Message-ID domain · `spf=fail`, `dkim=none/fail`, `dmarc=fail` · Reply-To to an unrelated domain · originating IP not in the claimed sender's network · time-zone/date inconsistencies between hops · missing X-Mailer · urgency/lure in the subject.

## Virtual machine forensics

**Importance 7/10 · good chance** · syllabus names "virtual machine" — moderate 5/10-mark

In simple words: a VM is just a set of files on the host, so its disk and even its RAM can be copied as files.

| File | Platform | Contents / forensic value |
|---|---|---|
| **.vmdk** | VMware | Virtual disk — mount/image like a physical disk |
| **.vhd / .vhdx** | Hyper-V | Virtual disk |
| **.vdi** | VirtualBox | Virtual disk |
| **.qcow2** | QEMU/KVM | Virtual disk |
| **.vmx** | VMware | Config: hardware, MAC address, paths — ties VM to host |
| **.vmem** | VMware | VM **RAM** when suspended/snapshotted — a ready-made memory image (Volatility) |
| **.vmsn** + delta disks | VMware | **Snapshot** state — earlier states, deleted data, timeline points |
| **vmware.log** | VMware | Power-on/off, snapshot events |

**Acquisition approach:** (1) if the VM is running, **suspend or snapshot** it to freeze RAM into `.vmem`/`.vmsn` (or capture host RAM); (2) hash and copy all VM files (disk, config, snapshots, logs); (3) **image the host too** — it may hold deleted VMs and host-side traces; (4) mount the virtual disk read-only and examine as a normal disk; (5) document per chain of custody.
**Pitfalls:** VMs make anti-forensics easy (delete the whole VM, run from USB); VM-aware malware changes behaviour inside VMs.

## Cloud technology forensics

Importance 8/10 · likely · "cloud forensics challenges" is a classic 10-mark discussion
In simple words: the data sits on someone else's shared, constantly changing servers, often abroad — you cannot just seize the box.

| Challenge | Explanation |
|---|---|
| **Loss of physical control** | Cannot seize hardware; depend on the CSP's tools/logs |
| **Multi-tenancy** | One physical server holds many customers → privacy limits, collateral exposure |
| **Jurisdiction** | Data and replicas across countries → MLAT, conflicting laws, data localisation |
| **Volatility / elasticity** | Instances created/destroyed, storage reallocated → evidence vanishes |
| **Provider dependence** | Log availability, format and retention decided by the CSP |
| **Chain of custody / provenance** | Hard to prove where data was and that it is unaltered |
| **Encryption** | Keys held by provider or customer |

**Approaches:** legal requests/preservation notices to the provider · provider logs (access, audit, IAM, API activity) · CSP-native **volume/instance snapshots** (IaaS) · **client-side artifacts** on the suspect's device (sync folders, cached files, browser artifacts, app tokens) · hash everything, document shared-responsibility boundary · SLAs should pre-provide forensic access. ⚠️ verify any NIST document number before quoting.

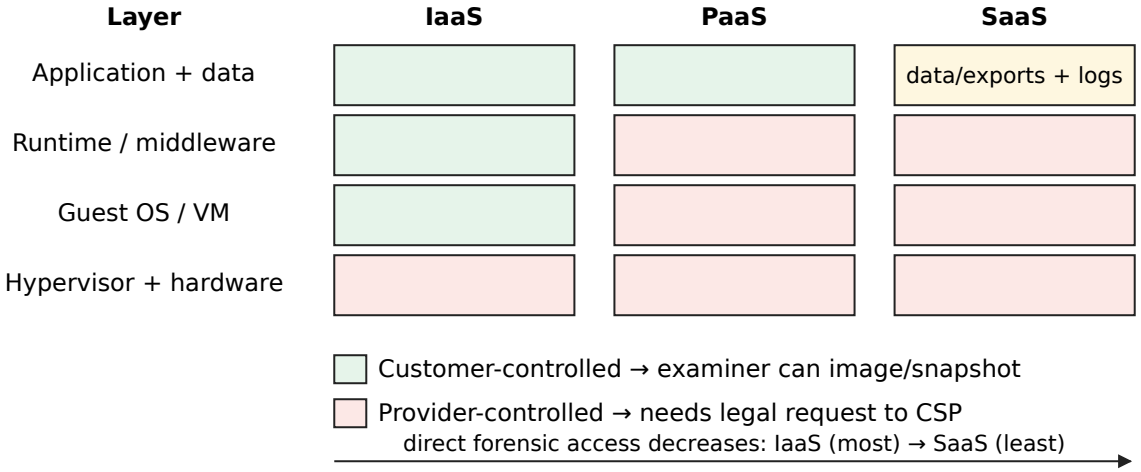



*Fig: Cloud service models — who controls each layer, and therefore where evidence must come from*

## Likely questions with model answers — Unit 2

**Q. List the important web-browser artifacts and state the forensic value of each.** *(5 marks · Likelihood 9/10)*
Browser artifacts record a user's online activity and, crucially, intent. (1) **History** — URLs with visit time, count and transition type; proves which sites were visited and whether the URL was typed. (2) **Cache** — stored copies of pages/images; proves the actual content viewed. (3) **Cookies** — session

cookies show an active login, persistent cookies show repeat visits, third-party cookies show tracking. (4) **Bookmarks/Favourites** — deliberate saved interest. (5) **Downloads** — links a file on disk to its source URL and time. (6) **Session data** — tabs open at last close. (7) **Autofill/saved logins** — attribution to a person. (8) **Extensions/plugins** — VPNs, wallets or wiping tools. In Chrome these sit in SQLite files (`History`, `Cookies`) and in Firefox in `places.sqlite` and `cookies.sqlite`, analysed on a hashed working copy.

**Q. Distinguish SMTP, POP3 and IMAP with their ports.** *(5 marks · Likelihood 9/10)*
**SMTP** (Simple Mail Transfer Protocol) is the push protocol for sending and relaying mail between servers; ports **25** (server relay), **587** (client submission) and **465** (SMTPS). Each SMTP server adds a `Received:` header, which is what makes tracing possible. **POP3** (Post Office Protocol 3) lets a client download mail from the server, usually deleting the server copy, so mail ends up on one device; ports **110** / **995** (TLS). Forensically, evidence then lies mainly on the local machine. **IMAP** (Internet Message Access Protocol) keeps mail on the server and synchronises folders across devices; ports **143** / **993** (TLS). Forensically, the authoritative copy is on the server, so a legal request to the provider is required. Summary: SMTP sends; POP3 and IMAP receive.

**Q. What are SPF, DKIM and DMARC? How do they help detect spoofed mail?** *(5 marks · Likelihood 8/10)*
**SPF** (Sender Policy Framework) is a DNS TXT record listing IP addresses allowed to send mail for a domain; the receiver checks the connecting IP — `spf=fail` means an unauthorised server. **DKIM** (DomainKeys Identified Mail) adds a digital signature over selected headers and body; the receiver verifies it with the public key in DNS — a pass proves the domain signed it and it was not altered. **DMARC** checks that SPF or DKIM passes **and aligns with the visible From: domain**, and publishes a policy (none/quarantine/reject) plus reporting. In a spoofed mail the `Authentication-Results:` header typically shows `spf=fail`, `dkim=none/fail`, `dmarc=fail`, along with a From: domain different from Return-Path and Message-ID.

**Q. What survives a private/incognito browsing session?** *(5 marks · Likelihood 7/10)*
Private mode only stops the browser saving history, cookies and cache to its normal profile files after the window closes. It is not anonymity. Recoverable traces: (1) **RAM** while the window is open, plus pagefile/hiberfil — captured by live memory acquisition; (2) the **OS DNS cache**; (3) **router, proxy, firewall and ISP logs**; (4) **downloaded files** kept on disk; (5) temp files, crash dumps, prefetch and jump lists of the browser; (6) **server-side and synced-account records**. Therefore, a running private session must be captured live before shutdown.

**Q. Describe virtual machine forensics.** *(5 marks · Likelihood 6/10)*
A VM exists as files on a host. Its **virtual disk** (`.vmdk` VMware, `.vhd/.vhdx` Hyper-V, `.vdi` VirtualBox, `.qcow2` KVM) can be hashed, mounted read-only and examined like a physical disk. The **`.vmx`** config gives hardware details, MAC address and paths. When suspended or snapshotted, VMware writes **RAM to `.vmem`** — a ready-made memory image for Volatility — and **snapshots (`.vmsn` + delta disks)** preserve earlier states, letting deleted data be recovered. **Logs** give a power/snapshot timeline. Approach: snapshot/suspend a running VM, copy and hash all VM files, and also image the host, which may hold deleted VMs. Pitfalls: whole-VM deletion and VM-aware malware.

**Q. State the challenges of cloud forensics.** *(5 marks · Likelihood 7/10)*
(1) **No physical access** — servers cannot be seized; the examiner depends on the CSP. (2) **Multi-tenancy** — many customers share hardware, so imaging risks other users' privacy. (3) **Jurisdiction** — data and replicas may be stored in several countries; MLAT and conflicting laws apply. (4) **Volatility/elasticity** — instances and storage are created and destroyed, so evidence vanishes. (5) **Provider dependence** — log availability, format and retention are set by the CSP. (6) **Chain of custody** — hard to prove provenance and integrity. (7) **Encryption** and key control. The service model matters: IaaS gives most access (VM snapshots), SaaS the least (exports and audit logs).

**Q. Explain browser forensics: artifacts, storage locations and private-mode limitations.** *(10 marks · Likelihood 8/10)*

**Concept:** browser forensics recovers and interprets the records a web browser keeps, to reconstruct a user's online activity and prove knowledge and intent.

**Artifacts:** history (URL, visit time, count, transition type), cache (actual content viewed), cookies (session vs persistent; first-party vs third-party), bookmarks/favourites, downloads list, session/tab restore data, autofill and saved logins, typed URLs and search terms, extensions/plugins, and sync data from other devices.

**Storage:** Chromium browsers keep SQLite databases under `%LocalAppData%\Google\Chrome\User Data\Default\` — `History` (visits, downloads), `Cookies`, `Web Data` (autofill), `Login Data`, plus `Bookmarks` as JSON and a `Cache` folder. Firefox uses `places.sqlite` (history + bookmarks), `cookies.sqlite`, `formhistory.sqlite` and `logins.json` in `%AppData%\Mozilla\Firefox\Profiles\`. Legacy IE used `index.dat` / `WebCacheV*.dat`.

**Method:** image the disk with a write blocker, hash it, extract the profile folders, query the SQLite files on a copy (or use a parser), convert timestamps (Chrome: microseconds since 1601; Firefox: Unix epoch microseconds) and build a timeline correlated with downloads, file-system and registry artifacts.

**Private mode:** prevents local persistence in the browser's stores, but RAM, pagefile/hiberfil, DNS cache, ISP/proxy logs, downloaded files, OS artifacts and server-side records remain. Hence live RAM capture before shutdown is essential.

**Conclusion:** typed URLs, bookmarks and repeated visits are the strongest evidence of intent, while cache proves what was actually displayed.

**Q. Discuss cloud forensics: service models, challenges and investigative approaches.** *(10 marks · Likelihood 7/10)*

**Concept:** cloud forensics applies forensic science to data and activity in cloud environments, where the data is held and operated by a third-party provider on shared infrastructure.

**Service models and access:** in **IaaS** the customer controls the VM, OS and data, so the examiner can take volume/instance snapshots — the most access. In **PaaS** the customer controls the application and its data/logs; the OS and runtime are the provider's. In **SaaS** only application data, exports and audit logs are reachable — the least access.

**Challenges:** loss of physical control; multi-tenancy (shared hardware, privacy of other tenants); jurisdiction (data across countries, MLAT, data localisation); volatility/elasticity (instances and storage are recycled); dependence on the provider for logs and cooperation; chain-of-custody and provenance difficulties; encryption and key management.

**Approaches:** (1) **legal requests** — preservation notices and orders to the CSP for data and logs; (2) **provider logs** — access, authentication, IAM and API activity logs, billing records; (3) **CSP-native snapshots** of volumes/instances in IaaS, hashed on acquisition; (4) **client-side artifacts** — sync folders, cached files, browser history, app tokens and credentials on the suspect's device; (5) document the shared-responsibility boundary and jurisdiction; (6) contracts/SLAs that guarantee log retention and forensic access in advance.

**Conclusion:** cloud forensics shifts from "seize and image" to "preserve, request and correlate", with the client device and provider logs as the main evidence sources.

**Q. Explain email forensics: header analysis, tracing the source and detection of spoofed mail.** *(15 marks · Likelihood 9/10)*

**1. Concept.** Email forensics establishes whether a message is genuine, where it really originated and who sent it. Email is the main vehicle for phishing, business email compromise, threats and extortion. The evidence is mostly in the **headers**, not the body.

**2. Email system.** The sender's **MUA** (Outlook, Gmail app) submits via **SMTP (587)** to the **MSA/MTA**; MTAs relay via **SMTP (25)**; the recipient's **MDA** stores the mail; the recipient's MUA fetches it by **POP3 (110/995)** or **IMAP (143/993)**. Webmail mail stays on the provider's servers,

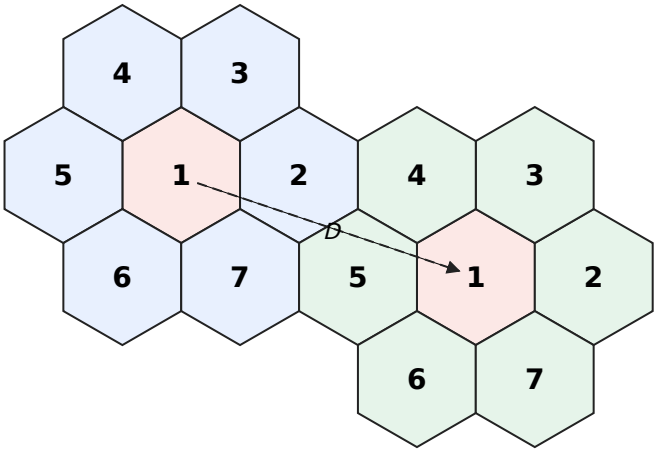
while client-based mail can sit in local `.pst/mbox` files. Every MTA **prepends a `Received:` header**, so headers form a stack.
**3. Header fields.** `Received:` (per hop — trace backbone), `From:` (easily forged), `Return-Path:` (envelope sender), `Reply-To:`, `Message-ID:` (`<id@sending-domain>`), `X-Originating-IP:` (client IP in some webmail), `Authentication-Results:` (SPF/DKIM/DMARC verdict), `Date:`, `X-Mailer:`.
**4. Tracing procedure.** (a) Obtain the full original headers (`.eml` "Show original"); hash the file. (b) Read `Received:` lines **bottom to top**; the bottom line is the origin, and the bracketed `[IP]` recorded by the receiving server is more reliable than the sender's claim. (c) Extract the originating IP (or X-Originating-IP). (d) WHOIS and geolocation → owning ISP or mail host. (e) Check time consistency across hops. (f) Send legal requests to the ISP/provider for subscriber details and logs for that IP and timestamp. (g) Correlate with the suspect's devices — sent items, browser webmail artifacts.
**5. Annotated example.** With the bottom hop `from [45.86.xx.xx] by webmail.cheap-host.ru`, a middle relay, and the top hop at the recipient's server, the origin is 45.86.xx.xx. `From: security@trusted-bank.co` but `Return-Path: billing@cheap-host.ru` and `Message-ID: <…@cheap-host.ru>`, `Reply-To` an unrelated domain, and `spf=fail; dkim=none; dmarc=fail`.
**6. Spoof detection.** Indicators: From ≠ Return-Path / Message-ID domain; SPF fail (sending IP not authorised in the domain's DNS record); DKIM missing or failing (no valid domain signature, or altered content); DMARC fail (no aligned pass with the visible From domain); Reply-To pointing elsewhere; origin IP outside the claimed organisation's network; inconsistent hop times; missing X-Mailer; urgency lures and look-alike domains.
**7. Tools.** Header analysers, mailbox parsers for `.pst/mbox`, WHOIS/IP lookup tools.
**8. Preservation and law.** Keep the original `.eml` with hash values, maintain chain of custody, and produce the electronic-record certificate under §65B IEA / §63 BSA (⚠️ verify sub-section). Spoofing and phishing attract IT Act §66C/§66D (⚠️ verify).
**Conclusion:** bottom-up `Received:` tracing locates the source, SPF/DKIM/DMARC plus field mismatches prove spoofing, and ISP legal process turns an IP into a person.

# Unit 7 — Mobile Computing

## Cells and frequency reuse

**Importance 9/10 · very likely** · "cells and frequency reuse" is a syllabus phrase — high-probability 5-mark
In simple words: split the area into small low-power cells so the same frequencies can be reused in cells far apart.

- **Cell:** area served by one base station (BTS / NodeB / eNodeB / gNodeB). Drawn as a **hexagon** — hexagons tile without gaps or overlaps and approximate a circular coverage area.
- **Frequency reuse:** the total channels are split among the **N cells of a cluster**; the same set is reused in the next cluster. Cluster size $N = i^2 + ij + j^2$ (1, 3, 4, 7, 12…).
- **Co-channel reuse distance:** $D = R \cdot \sqrt{(3N)}$ (R = cell radius). Larger N → less interference but fewer channels per cell.
- **Capacity growth:** cell splitting (smaller cells) and sectoring (e.g. 3 × 120° antennas).
- **Framework:** MS ↔ base station ↔ controller ↔ switching centre (MSC) ↔ PSTN/Internet, with HLR/VLR tracking location; **roaming** = service outside the home network.




**Cluster size N = 7 (i=2, j=1)**

Blue cluster and green cluster each use channel sets 1–7 once.

Co-channel cells (both "1")

Reuse distance D = R·√(3N)
For N = 7: D = √21·R ≈ 4.58R

Bigger N → less co-channel interference, fewer channels/cell

*Fig: Hexagonal cells with a 7-cell reuse cluster — the same channel set (1) is reused at distance D*

## Wireless delivery technology, switching and handoff

Importance 7/10 · good chance · "switching methods" syllabus phrase; handoff is a frequent compare

In simple words: older networks reserved a whole line per call (circuit); newer ones break everything into packets; handoff keeps a call alive while you move between cells.

| Circuit switching | Packet switching |
|---|---|
| Dedicated channel reserved for the whole call | Data split into packets sharing the channel |
| Guaranteed bandwidth, wasted when idle | Efficient, bandwidth on demand; variable delay |
| 1G/2G voice (GSM) | GPRS/EDGE overlay, 3G data; LTE/5G are all-IP (voice as **VoLTE**) |

| Hard handoff | Soft handoff |
|---|---|
| **Break-before-make** — old link dropped, then new one set up | **Make-before-break** — connected to two or more base stations at once |
| GSM (FDMA/TDMA, different frequencies in neighbouring cells) | CDMA / WCDMA (same frequency, different codes) |
| Simpler, brief interruption, risk of drop | Smoother, fewer drops, uses more network resources |

**Access methods (delivery technology):** FDMA (frequency slots, 1G), TDMA (time slots, GSM), CDMA (unique codes, IS-95/3G), OFDMA (orthogonal sub-carriers, LTE/5G).

## Generations 1G – 5G

Importance 8/10 · likely · evolution table is a standard 10-mark and MCQ source

| Gen | Era | Technology | Switching | Data / feature |
|---|---|---|---|---|
| **1G** | 1980s | **Analog**, AMPS, FDMA | Circuit | Voice only |
| **2G** | 1990s | **Digital** — GSM (TDMA/FDMA), CDMA IS-95 | Circuit | Voice + **SMS** |
| **2.5G / 2.75G** | ~2000 | **GPRS / EDGE** | Packet overlay | ~114 / ~384 kbps (approx.) |
| **3G** | 2000s | **UMTS/WCDMA**, CDMA2000, HSPA/HSPA+ | Circuit + packet | Mobile broadband (Mbps), video calls |
| **4G** | 2010s | **LTE / LTE-Advanced**, OFDMA, MIMO | **All-IP** | Tens–hundreds of Mbps, VoLTE |
| **5G** | 2019+ | **NR** (New Radio), mmWave, massive MIMO, network slicing | All-IP | Gbps, ultra-low latency, massive IoT |

## Mobile information access devices and internetworking standards

Importance 6/10 · moderate · syllabus phrases, mostly list answers

In simple words: the gadgets that access mobile data, and the standards that carry it.

- **Devices:** smartphones, tablets, feature phones, PDAs (historical), laptops with data cards, wearables, in-vehicle units, IoT/M2M modules. Constraints: small screen, limited battery/CPU/storage, intermittent connectivity.

| Standard | Type | Note |
|---|---|---|
| **WAP** | Application stack | Early web for phones: WML markup, **WAP gateway** translates WAP ↔ HTTP; layers WAE/WSP/WTP/WTLS/WDP (⚠️ verify layer names before writing all five); now historical |
| **GPRS / EDGE** | 2.5G / 2.75G | Packet data over GSM |
| **UMTS / HSPA** | 3G | WCDMA radio, mobile broadband |
| **LTE** | 4G | All-IP, OFDMA |
| **Wi-Fi (IEEE 802.11)** | WLAN | Local high-speed access, hotspots |
| **Bluetooth (IEEE 802.15.1)** | WPAN | Short-range device pairing |
| **NFC**, **Zigbee** | Short-range / IoT | Contactless payment; sensor networks |

## Cellular data communication protocols — GSM architecture

Importance 9/10 · very likely · GSM architecture diagram is the strongest 10/15-mark candidate in this unit

In simple words: phone → tower → tower controller → switching centre, which checks databases to find, authenticate and bill the subscriber.

| Element | Subsystem | Role |
|---|---|---|
| **MS** (Mobile Station) | — | Handset (identified by **IMEI**) + **SIM** (holds **IMSI**, key Ki) |
| **BTS** (Base Transceiver Station) | BSS | Radio transceiver/antenna of one cell; Um air interface |
| **BSC** (Base Station Controller) | BSS | Controls many BTSs: channel allocation, power control, handoffs between its BTSs |
| **MSC** (Mobile Switching Centre) | NSS | Call switching, routing, handoffs between BSCs, gateway to PSTN (GMSC) |
| **HLR** (Home Location Register) | NSS | Permanent subscriber database: IMSI, MSISDN, services, current VLR |
| **VLR** (Visitor Location Register) | NSS | Temporary copy for subscribers currently in the MSC's area |
| **AuC** (Authentication Centre) | NSS | Holds Ki; generates challenge/response for authentication and cipher keys |
| **EIR** (Equipment Identity Register) | NSS | IMEI lists — white/grey/**black** (stolen handsets) |
| **OMC** | OSS | Operation and maintenance |

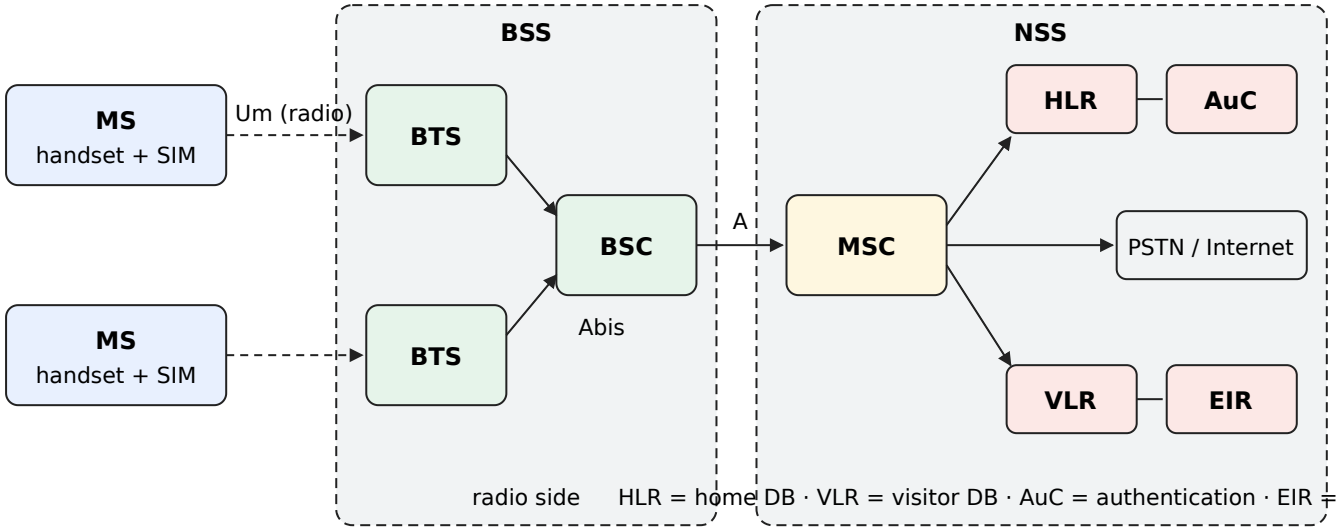



*Fig: GSM architecture — MS → BTS → BSC (BSS) → MSC with HLR, VLR, AuC, EIR (NSS)*

**GSM vs CDMA:** GSM uses TDMA+FDMA, removable SIM, hard handoff; CDMA uses code division (all users share the band), historically handset-bound with no SIM, soft handoff.

## Mobile computing applications

Importance 5/10 · possible · list answer

In simple words: anything useful you do on the move with a connected device.

- Communication (voice, SMS, messaging, email), mobile web, social media.
- Location-based services (navigation, ride-hailing, delivery tracking).
- M-commerce / m-banking / m-payments; m-ticketing.
- m-health (telemedicine, wearables), m-learning, m-governance (e-services on phones).
- Field workforce, logistics, emergency services, IoT/M2M monitoring.

## Mobile databases — protocols, scope, tools and technology

Importance 7/10 · good chance · syllabus names "mobile databases" — likely 5-mark

In simple words: a database that lives on (or syncs with) a phone and must keep working when the

network drops.

| Aspect | Details |
|---|---|
| **Scope / needs** | Intermittent connectivity, limited battery/storage, mobility, easily lost device → local storage, sync, conflict resolution, encryption |
| **Replication & synchronisation** | Local copy syncs with the central server when connected; conflict resolution (last-writer-wins, merge) |
| **Offline-first** | App reads/writes the local DB first; changes queued and synced later |
| **Caching** | Keep recently used data locally |
| **Hoarding** | **Pre-fetch** data likely to be needed *before* disconnection |
| **Transactions** | Long-lived / disconnected transactions; **optimistic** concurrency suits mobility |
| **Tools** | **SQLite** (default on Android & iOS), **Realm**; server-side sync to the enterprise DB |
| **Forensic payoff** | SMS, call logs, contacts, chat apps and browser history live in **SQLite** files; deleted rows often survive in free pages / WAL / journal files |

## M-business

**Importance 6/10 · moderate** · syllabus phrase; short note

In simple words: doing business — buying, banking, paying — through mobile devices.

- **M-commerce:** shopping, m-ticketing, location-based offers, m-marketing.
- **M-banking:** balance checks, transfers, statements via app/SMS/USSD.
- **M-payments: UPI**, mobile wallets, NFC/QR payments.
- **Enablers:** ubiquity, personalisation, location awareness, secure payment.
- **Risks / forensic relevance:** SIM-swap fraud, fake UPI apps, vishing; payment-app databases, UPI transaction records and location history are key evidence in fraud cases.

## Mobile forensic relevance

**Importance 9/10 · very likely** · IMEI vs IMSI and acquisition levels are high-value forensic items

In simple words: a phone is the richest exhibit — isolate it, identify it, then extract data at the right level.

| Identifier | Identifies | Where |
|---|---|---|
| **IMEI** | The **handset** — 15 digits, display with `*#06#` | Phone hardware |
| **IMSI** | The **subscriber** — up to 15 digits = MCC + MNC + MSIN | SIM / USIM |
| **ICCID** | The **SIM card** serial — 19–20 digits | Printed on/stored in SIM |
| **MSISDN** | The phone number | Network (HLR) |

- **SIM evidence:** IMSI, ICCID, phonebook (ADN), some SMS, last dialled numbers, location info (LAI/TMSI).
- **Isolation: Faraday bag** / airplane mode to stop remote wipe and new data; keep it charged.
- **BFU vs AFU:** *Before First Unlock* — keys not derived, little accessible; *After First Unlock* — keys in memory, much more recoverable. Keep an AFU phone powered.
- **Acquisition levels (least → most invasive):** manual → **logical** (API/backup; usually no deleted data) → file system (app SQLite DBs) → **physical** (bit-for-bit flash, incl. deleted) → JTAG/ISP → **chip-off** (desolder memory; destructive). On encrypted phones physical/chip-off give ciphertext without keys.
- **CDR** (Call Detail Record) from the operator: A/B party numbers, date/time/duration, IMEI, IMSI, **cell ID / tower location** — places a suspect near a scene (approximate, not GPS); same IMEI + different IMSI = SIM swap. Content needs lawful interception (IT Act §69 — ⚠️ verify).

## Likely questions with model answers — Unit 7

**Q. What is frequency reuse? Why are cells hexagonal?** *(5 marks · Likelihood 9/10)*
Radio spectrum is limited, so the service area is divided into small **cells**, each served by a low-power base station. The available channels are divided among the **N cells of a cluster**, and the same channel sets are **reused** in other clusters far enough away to keep co-channel interference acceptable. The cluster size is $N = i^2 + ij + j^2$ (e.g. 3, 4, 7), and the co-channel reuse distance is $D = R\sqrt{(3N)}$; for N = 7, D ≈ 4.58R. Smaller N gives more channels per cell (more capacity) but more interference. Cells are drawn as **hexagons** because hexagons tile a plane without gaps or overlaps and come closest to the circular radiation pattern of an antenna, so fewer cells cover an area. Capacity is increased further by cell splitting and sectoring.

**Q. Distinguish hard handoff and soft handoff.** *(5 marks · Likelihood 8/10)*
**Handoff** transfers an active call from one cell/base station to another as the user moves. **Hard handoff** is **break-before-make**: the link to the old base station is released before the new one is set up. It is used in GSM, where neighbouring cells use different frequencies; it is simpler but causes a brief interruption and may drop calls. **Soft handoff** is **make-before-break**: the mobile is connected to two or more base stations at once and drops the old one only after the new one is stable. It is used in CDMA/WCDMA, where all cells share the same frequency and users are separated by codes. It gives smoother calls and fewer drops, but uses more network resources.

**Q. Differentiate IMEI and IMSI. Why does the difference matter in investigations?** *(5 marks · Likelihood 9/10)*
**IMEI** (International Mobile Equipment Identity) identifies the **handset**; it is 15 digits (TAC + serial + check digit), stored in the phone hardware and shown by dialling `*#06#`. **IMSI** (International Mobile Subscriber Identity) identifies the **subscriber**; it is up to 15 digits (MCC + MNC + MSIN) and stored on the SIM. ICCID is the SIM's own serial and MSISDN is the phone number. Forensic value: when a criminal changes SIMs in the same phone, the **IMEI stays the same** while the IMSI changes, so CDRs searched by IMEI still trace the handset. Same IMSI with a different IMEI shows a handset swap. The EIR blacklists stolen IMEIs.

**Q. What is a mobile database? Explain caching, hoarding and synchronisation.** *(5 marks · Likelihood 7/10)*
A mobile database runs on or synchronises with a mobile device under intermittent connectivity, limited battery and storage, and mobility. **Caching** keeps recently used data on the device. **Hoarding** pre-fetches data the user is likely to need *before* disconnection, so the user can keep working offline. **Replication and synchronisation** keep a local copy that is reconciled with the central server when connected; conflicting updates are resolved by rules such as last-writer-wins or merging. The **offline-first** design writes locally and syncs later, with optimistic concurrency. Common tools: **SQLite** (default on Android and iOS) and **Realm**. Forensically, SMS, call logs and chat apps sit in SQLite files, and deleted rows can often be recovered.

**Q. Explain the mobile data internetworking standards.** *(5 marks · Likelihood 6/10)*
**WAP** (Wireless Application Protocol) delivered web content to early phones using WML markup through a WAP gateway that translates WAP to HTTP; it is now historical. **GPRS** (2.5G) and **EDGE** (2.75G) added packet-switched data to GSM. **UMTS/WCDMA** (3G) and **HSPA/HSPA+** gave mobile broadband. **LTE** (4G) is an all-IP OFDMA network carrying voice as VoLTE; **5G NR** adds Gbps speeds, low latency and network slicing. Local standards: **Wi-Fi (IEEE 802.11)** for WLAN, **Bluetooth (802.15.1)** for personal-area pairing, and **NFC** for contactless payments.

**Q. Explain the levels of mobile data acquisition.** *(5 marks · Likelihood 8/10)*
From least to most invasive: (1) **Manual** — operate the phone and photograph screens; only visible data. (2) **Logical** — extract via the OS/backup API (contacts, SMS, call logs, media); fast and safe, usually no deleted data. (3) **File system** — copy the file system including app SQLite databases;

may recover deleted rows. (4) **Physical** — bit-for-bit image of flash memory including unallocated space. (5) **JTAG/ISP** — read raw memory through test points or eMMC pads. (6) **Chip-off** — desolder the memory chip and read it directly; destructive and expert-only. Before any method, isolate the phone in a **Faraday bag** and note BFU/AFU state. On encrypted phones, physical and chip-off images are ciphertext without the key.

**Q. Trace the evolution of mobile networks from 1G to 5G.** *(10 marks · Likelihood 8/10)*
**1G (1980s):** analog cellular such as AMPS, FDMA, circuit-switched voice only; poor security (easy eavesdropping, cloning) and quality.
**2G (1990s):** digital — **GSM** (TDMA+FDMA, SIM card) and **CDMA IS-95**; digital encryption, **SMS**, better capacity; circuit-switched.
**2.5G/2.75G: GPRS** and **EDGE** overlay packet-switched data on GSM (roughly 114 kbps and 384 kbps peak); always-on data, WAP browsing, MMS.
**3G (2000s): UMTS/WCDMA** and CDMA2000, later **HSPA/HSPA+**; mobile broadband in Mbps, video calling, soft handoff in WCDMA; both circuit (voice) and packet (data) domains.
**4G (2010s): LTE / LTE-Advanced** — OFDMA, MIMO, flat **all-IP** packet core; tens to hundreds of Mbps; voice as VoLTE; HD video streaming.
**5G (2019+): NR (New Radio)** with mmWave and sub-6 GHz bands, massive MIMO, beamforming, **network slicing**; Gbps rates, ultra-low latency and massive IoT.
**Trends:** analog → digital; circuit → packet → all-IP; FDMA → TDMA → CDMA → OFDMA; growing data rates and lower latency. A summary table (generation, technology, switching, data) earns marks.
**Forensic note:** each generation leaves operator records (CDR, IMEI/IMSI, cell ID) used to place and link suspects.

**Q. Discuss mobile databases: scope, protocols, tools and technology.** *(10 marks · Likelihood 6/10)*
**Concept:** a mobile database stores and manages data on mobile devices and synchronises it with a central server, so the application works despite mobility and disconnection.
**Scope / constraints:** intermittent and low-bandwidth links, limited battery, CPU and storage, frequent location changes, small screens, and devices that are easily lost (so encryption is needed).
**Architecture:** mobile clients with an embedded DB ↔ wireless network ↔ sync/middleware server ↔ enterprise database.
**Protocols and techniques:** (1) **replication** — full or partial copies on the device; (2) **synchronisation** — two-way reconciliation on reconnection, with **conflict resolution** (timestamps, last-writer-wins, merge rules); (3) **caching** of recent data; (4) **hoarding** — pre-fetching data before disconnection; (5) **offline-first** — local writes queued and synced later; (6) transaction models for long-lived or disconnected transactions with **optimistic concurrency** and later validation; (7) location-dependent queries.
**Tools and technology: SQLite** (embedded, serverless, default on Android/iOS), **Realm** (object database for mobile), and server-side sync services connected to enterprise RDBMS.
**Forensic significance:** messaging apps, call logs, SMS, contacts and browser data are SQLite files; deleted records often survive in free pages, WAL and journal files and can be carved after file-system acquisition.

**Q. Describe the GSM architecture with a neat diagram.** *(10 marks · Likelihood 8/10)*
Draw the GSM diagram (MS → BTS → BSC → MSC with HLR, VLR, AuC, EIR) and explain three subsystems.
**(1) Mobile Station (MS):** the handset (IMEI) plus the SIM (IMSI, secret key Ki, phonebook/SMS). It talks over the **Um** radio interface.
**(2) Base Station Subsystem (BSS):** the **BTS** has the radio transceivers and antenna for a cell and handles the radio link. The **BSC** controls many BTSs over the **Abis** interface — radio channel allocation, frequency hopping, power control and handoffs between its own BTSs.
**(3) Network Switching Subsystem (NSS):** the **MSC** (over the **A** interface) switches and routes

calls, handles handoffs between BSCs, and connects to the PSTN/other networks through a gateway MSC. The **HLR** is the permanent subscriber database (IMSI, MSISDN, subscribed services, current location area/VLR). The **VLR** holds temporary data for subscribers currently in the MSC's area, reducing HLR queries. The **AuC** stores Ki and generates challenge–response values and cipher keys for authentication and encryption. The **EIR** checks IMEIs against white/grey/black lists to block stolen handsets.
**(4) OSS:** operation and maintenance centre for network management.
**Call flow (brief):** the MS registers → VLR fetches its profile from the HLR → AuC authenticates → for an incoming call the GMSC asks the HLR, which points to the current VLR/MSC → MSC pages the MS through BSC/BTS.
**Forensic relevance:** HLR/VLR location data, EIR, and the MSC's CDRs (IMEI, IMSI, cell ID) support suspect location and handset tracing.

**Q. Describe the architecture of a cellular mobile network (cells, frequency reuse, GSM elements, handoff) and explain its forensic significance.** *(15 marks · Likelihood 7/10)*
**1. Cellular concept.** The service area is divided into hexagonal **cells**, each with a low-power base station. Hexagons tile without gaps. Channels are split among the **N cells of a cluster** ($N = i^2 + ij + j^2$, e.g. 7) and reused in other clusters at distance **$D = R\sqrt{(3N)}$**. This frequency reuse multiplies capacity; cell splitting and sectoring add more. (Draw the 7-cell cluster.)
**2. Access and switching.** FDMA (1G), TDMA+FDMA (GSM), CDMA (IS-95, WCDMA), OFDMA (LTE/5G). Voice was **circuit-switched** (a dedicated channel per call); data moved to **packet switching** with GPRS/EDGE; LTE and 5G are **all-IP**, with voice as VoLTE.
**3. GSM architecture. MS** (handset with IMEI + SIM with IMSI/Ki) — **Um** — **BTS** (radio of one cell) — **Abis** — **BSC** (controls BTSs, channel allocation, intra-BSC handoff) — **A** — **MSC** (switching, routing, inter-BSC handoff, gateway to PSTN). Databases: **HLR** (permanent subscriber profile and location), **VLR** (temporary profile for visitors in the MSC area), **AuC** (authentication and cipher keys), **EIR** (IMEI white/grey/black lists). OSS handles maintenance. (Draw the GSM diagram.)
**4. Mobility management.** Location updates as the MS moves between location areas update the VLR/HLR; **roaming** uses the visited network's VLR. **Handoff:** hard (break-before-make, GSM) versus soft (make-before-break, CDMA/WCDMA).
**5. Generations.** 1G analog → 2G digital GSM/CDMA + SMS → 2.5G GPRS/EDGE packet data → 3G UMTS/HSPA → 4G LTE all-IP → 5G NR (mmWave, massive MIMO, slicing).
**6. Forensic significance.** (a) **IMEI vs IMSI** — handset vs subscriber; SIM-swap and handset-swap detection; EIR blocks stolen phones. (b) **CDRs** at the MSC record A/B party, time, duration, IMEI, IMSI and **cell ID / tower** — placing a suspect near a scene (approximate) and mapping contacts; **tower dumps** list all phones on a cell in a time window. (c) The SIM holds IMSI, ICCID, contacts, SMS and last location info. (d) Seized phones are isolated in a **Faraday bag** to stop network contact and remote wiping; BFU/AFU state is recorded; data is acquired logically, by file system, physically, or by chip-off as needed, with hashing and chain of custody.
**Conclusion:** the cellular design (cells, reuse, MSC/HLR/VLR) both delivers mobility and creates the operator records that make mobile phones the most valuable evidence source in modern investigations.

# Unit 3 — Software Engineering

## 3.1 SDLC — Steps

**Importance 8/10 · likely** · first syllabus phrase of the unit; classic 5-mark "explain the steps of SDLC"

In simple words: SDLC is the ordered set of stages a software product passes through from idea to retirement.

| Step | Activity | Output |
|---|---|---|
| 1. Feasibility study | Technical, economic, operational, legal viability | Feasibility report |
| 2. Requirement analysis | Gather & analyse user needs | SRS document |
| 3. Design | System (architectural) + detailed design | Design document (DFD, structure chart, UML) |
| 4. Coding | Translate design into code; unit test | Source code |
| 5. Testing | Integration, system, acceptance testing | Test reports, defect log |
| 6. Deployment | Install, train users | Running system |
| 7. Maintenance | Corrective, adaptive, perfective, preventive changes | New releases |

Exam tip: name every step with its deliverable — examiners mark "phase + output".

## 3.2 Waterfall Model

**Importance 9/10 · very likely** · syllabus names it explicitly; diagram question

In simple words: phases flow one after another like a waterfall; a phase starts only after the previous one is signed off (Royce, 1970).

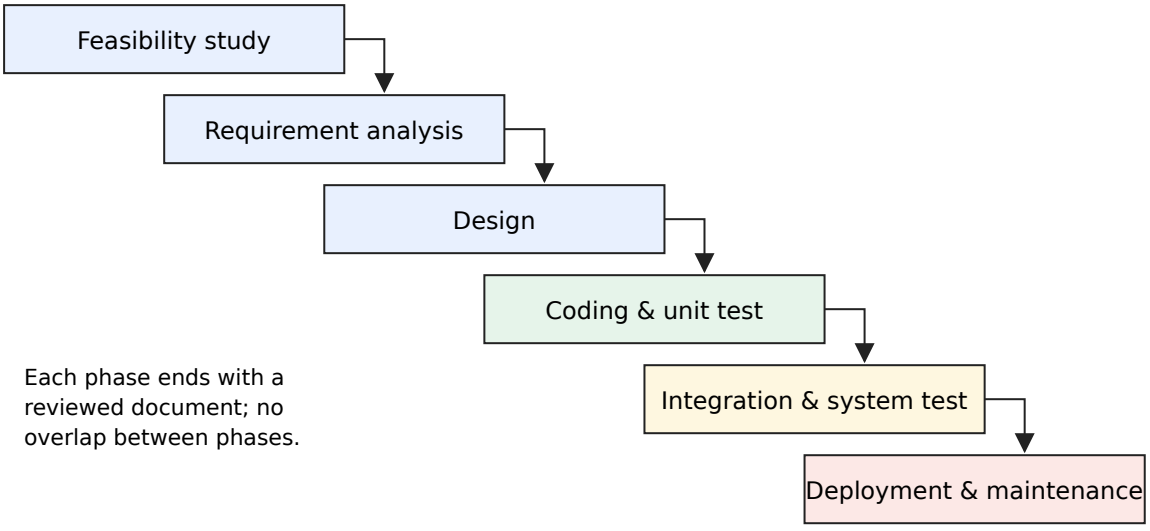



*Fig: Waterfall model — strictly sequential phases cascading downwards*

- **Advantages:** simple, easy to manage, well documented, good for small projects with stable requirements.
- **Disadvantages:** no working software until late; changes costly; errors found late; high risk for large projects.
- *Iterative waterfall* adds feedback paths to earlier phases.

## 3.3 Prototyping Model

**Importance 7/10 · good chance** · syllabus word "prototypes"

In simple words: build a quick working model, show it to the customer, refine until requirements are clear, then build the real system.

- Steps: requirement gathering → quick design → build prototype → customer evaluation → refine → engineer product.
- **Throwaway** prototype (discarded) vs **evolutionary** prototype (grows into the product).
- Use when requirements are unclear or UI is critical. Risk: customer treats prototype as final; quick-and-dirty choices retained.

## 3.4 Spiral Model (Boehm, 1986)

**Importance 9/10 · very likely** · syllabus names it; risk-driven model is a favourite 10-mark question

In simple words: development goes round in loops; each loop does planning, risk analysis, building and review.

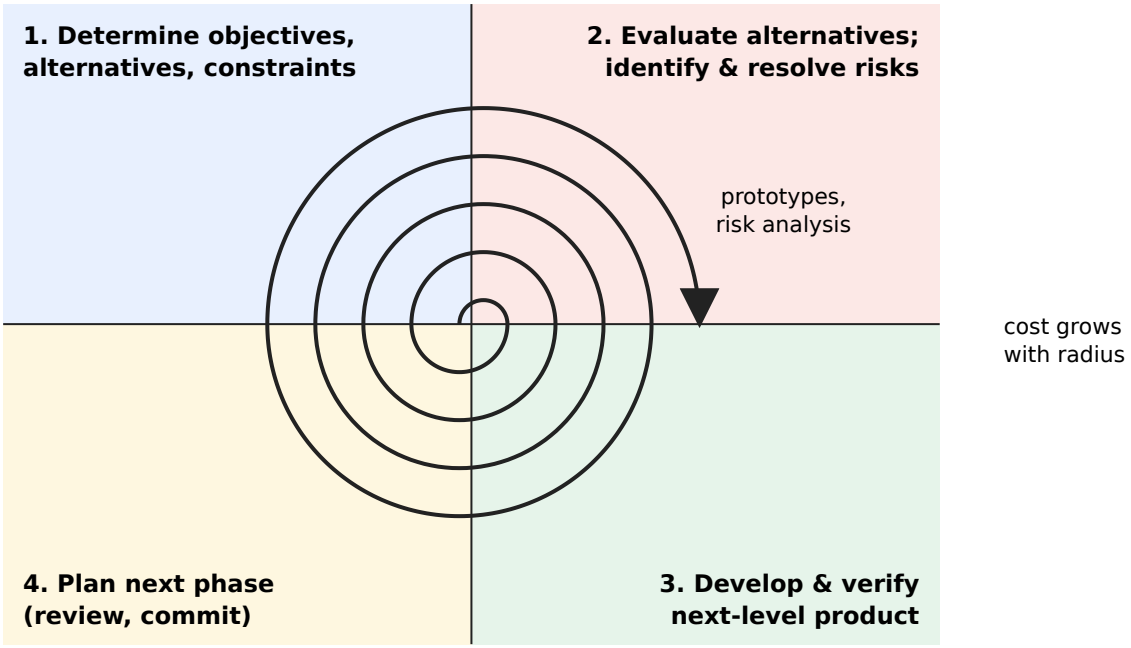



*Fig: Boehm's spiral model — every loop passes through the four quadrants; radius = cumulative cost*

- **Advantages:** explicit risk analysis; suits large, high-risk projects; absorbs change; early prototypes.
- **Disadvantages:** costly; needs risk expertise; milestones hard to fix; overkill for small projects.
- Called a **meta-model** — it contains waterfall, prototyping and incremental models.

| Feature | Waterfall | Prototyping | Spiral |
|---|---|---|---|
| Requirements | Fixed, clear | Unclear | Evolving, risky |
| Risk handling | None | Low | Explicit each loop |
| Customer involvement | Start & end | High | Every loop |
| Best for | Small, stable | UI-heavy, vague | Large, high-risk |

## 3.5 Software Metrics

**Importance 7/10 · good chance** · syllabus phrase; definitions + examples

In simple words: a metric is a quantitative measure of a product, process or project attribute.

- **Product metrics:** size (LOC/KLOC, Function Points), complexity (McCabe cyclomatic V(G) = E − N + 2P = decisions + 1), quality (defect density).
- **Process metrics:** defect removal efficiency DRE = E/(E+D) (E = errors found before delivery, D = defects after).

- **Project metrics:** effort (person-months), cost, schedule; productivity = KLOC / person-month.
- **Halstead:** n1, n2 (distinct operators/operands), N1, N2 (totals); length N = N1+N2, volume V = N log2 n.

## 3.6 Software Project Management

**Importance 7/10 · good chance** · syllabus phrase

In simple words: planning, organising, staffing, monitoring and controlling a software project to deliver on time, within cost and quality.

- **Activities:** project planning, size/cost estimation (COCOMO: Effort = a·(KLOC)^b; organic a=2.4, b=1.05), scheduling (Gantt, PERT/CPM), staffing, risk management, configuration management, monitoring.
- **The 4 P's:** People, Product, Process, Project.
- Output: Software Project Management Plan (SPMP).

## 3.7 Software Design — System Design and Detailed Design

**Importance 7/10 · good chance** · syllabus phrase; "differentiate" style

In simple words: system (high-level/architectural) design decides the modules and their interfaces; detailed (low-level) design decides the internal logic and data structures of each module.

| System design (HLD) | Detailed design (LLD) |
|---|---|
| Modules, interfaces, architecture, database | Algorithms, data structures, pseudocode of each module |
| Tools: structure chart, architecture diagram | Tools: pseudocode, flowchart, decision tables |
| Done by architects | Done by designers/programmers |

## 3.8 Function-Oriented Design (DFD, Structure Charts)

**Importance 8/10 · likely** · syllabus phrase; DFD is a sure diagram question

In simple words: the system is viewed as a set of functions that transform data; decomposed top-down (Structured Analysis/Structured Design).

- **DFD symbols:** process (circle/bubble), data flow (arrow), data store (open rectangle / parallel lines), external entity (rectangle).
- **Levels:** Level-0 = context diagram (whole system as one bubble); Level-1, Level-2 … refine it. **Balancing:** inputs/outputs of a child DFD must match its parent bubble.
- **Structure chart:** hierarchy of modules (rectangles) with calls; arrows show data couples (open circle) and control flags (filled circle). Derived from DFD by **transform analysis** (input–process–output) or **transaction analysis**.
- **Data dictionary** defines every data item in the DFD.

## 3.9 Object-Oriented Design (UML — Class and Use-case)

**Importance 6/10 · moderate** · syllabus phrase; brief coverage enough

In simple words: the system is modelled as interacting objects (data + behaviour) using UML.

- **Class diagram:** box with 3 compartments (name, attributes, operations); relationships — association, aggregation (hollow diamond), composition (filled diamond), inheritance/generalisation (hollow triangle), dependency (dashed arrow); multiplicity 1, 0..*, 1...*
- **Use-case diagram:** actors (stick figures), use cases (ovals), system boundary (rectangle); relationships «include», «extend», generalisation.

- OO principles: abstraction, encapsulation, inheritance, polymorphism.

## 3.10 User Interface Design — Principles

**Importance 6/10 · moderate** · syllabus phrase

In simple words: rules that make a UI easy, safe and pleasant to use.

- Mandel's golden rules: (1) **place the user in control**, (2) **reduce the user's memory load**, (3) **make the interface consistent**.
- Others: feedback for every action, error prevention & simple recovery (undo), shortcuts for experts, visibility of system status, accessibility, minimal design.

## 3.11 Design-Level Metrics — Coupling, Cohesion, Fan-in/Fan-out

**Importance 9/10 · very likely** · classic question "explain coupling and cohesion with types"

In simple words: good design = **low coupling** (modules independent) + **high cohesion** (each module does one job).

| Cohesion (worst → best) | Meaning |
|---|---|
| Coincidental | Unrelated tasks lumped together |
| Logical | Logically similar tasks, chosen by a flag |
| Temporal | Tasks executed at the same time (e.g. init) |
| Procedural | Steps of one procedure in sequence |
| Communicational | Tasks operate on the same data |
| Sequential | Output of one is input to the next |
| Functional (best) | Single well-defined function |

| Coupling (best → worst) | Meaning |
|---|---|
| Data (best) | Pass only needed elementary data |
| Stamp | Pass a whole structure, part used |
| Control | Pass a flag that controls the other's logic |
| External | Share external format/device/protocol |
| Common | Share global data |
| Content (worst) | One module modifies/jumps into another's internals |

- **Fan-in** = number of modules that call a given module (high = good reuse). **Fan-out** = number of modules a module calls (high = complex; keep low, ≈ ≤ 7).
- Good design: high fan-in at low levels, moderate fan-out.

## 3.12 Coding and Testing; Testing-Level Metrics

**Importance 8/10 · likely** · syllabus phrase; testing always asked

In simple words: coding converts design to programs following standards; testing executes the program to find defects.

- **Coding standards:** naming, indentation, comments, no globals, limited function length; **code review** = walkthrough, inspection.
- **Levels:** unit → integration (top-down/bottom-up/big-bang, stubs & drivers) → system → acceptance (alpha/beta). Regression testing after changes.
- **Black-box:** equivalence partitioning, boundary value analysis, cause-effect graph. **White-box:** statement, branch, path coverage; cyclomatic complexity = number of independent paths.

- **Testing metrics: code coverage** = (statements/branches executed ÷ total) × 100; **defect density** = defects ÷ KLOC; DRE; test-case pass rate; mean time to find defect.

## 3.13 Software Quality and Reliability (MTTF, MTBF, Availability)

**Importance 8/10 · likely** · syllabus phrase with formulas — easy numerical

In simple words: quality = conformance to requirements and fitness for use; reliability = probability of failure-free operation for a given time in a given environment.

- Quality models: **McCall** (product operation, revision, transition), ISO 9126 / ISO 25010 (functionality, reliability, usability, efficiency, maintainability, portability). Process quality: ISO 9001, **CMM levels 1–5** (Initial, Repeatable, Defined, Managed, Optimizing).
- **MTTF** = mean time to failure; **MTTR** = mean time to repair; **MTBF = MTTF + MTTR**.
- **Availability = MTTF / (MTTF + MTTR) = MTTF / MTBF**.
- Example: MTTF = 98 h, MTTR = 2 h → MTBF = 100 h, availability = 98/100 = **98%**.
- ROCOF = rate of occurrence of failures; POFOD = probability of failure on demand.

## 3.14 Clean Room Approach

**Importance 7/10 · good chance** · unusual syllabus phrase — likely 5-mark

In simple words: prevent defects instead of removing them — like a semiconductor "clean room" (developed at IBM by Harlan Mills).

- Formal specification (box-structure: black box → state box → clear box).
- **Correctness verification** by mathematical proof/team review instead of unit testing — developers do **not** execute their code.
- Incremental development by small teams.
- **Statistical usage testing** by a separate team, based on an operational usage profile → certifies reliability (MTTF).
- Result: very low defect density; high cost of formal methods and skills.

## 3.15 Software Re-engineering and Reverse Engineering (forensic relevance)

**Importance 9/10 · very likely** · forensic examiner will love reverse engineering → malware analysis

In simple words: reverse engineering = analysing a system to recover its design/specification without changing it; re-engineering = reverse engineering + restructuring + forward engineering to produce an improved system.

- **Re-engineering process model (Pressman):** inventory analysis → document restructuring → **reverse engineering** → code restructuring → data restructuring → forward engineering.
- **Reverse engineering levels:** abstraction level (procedural, program, data, system), completeness, interactivity, directionality.
- **Forward vs reverse:** forward = requirements → code; reverse = code → design/requirements.
- **Forensic relevance — malware analysis:** investigators reverse engineer suspicious binaries to learn what they do.
- *Static analysis:* hashing (MD5/SHA-256) and VirusTotal lookup, `strings`, PE header/imports, packer detection, disassembly (IDA Pro, Ghidra), decompilation.
- *Dynamic analysis:* run in an isolated sandbox/VM (Cuckoo); monitor processes, registry, files, network (Procmon, Wireshark).
- Outputs: IOCs (hashes, C2 domains/IPs, mutexes, registry keys), attribution, YARA rules.
- Also used for: legacy-code recovery, interoperability, recovering deleted-app data formats, examining firmware of seized devices.

## Likely questions with model answers

**Q. Explain the spiral model of software development with a diagram.** *(5 marks · Likelihood 9/10)*
The spiral model (Boehm, 1986) is a risk-driven, iterative model combining the controlled steps of waterfall with the iteration of prototyping. Development proceeds in loops; each loop is a phase and passes through four quadrants: (1) determine objectives, alternatives and constraints; (2) evaluate alternatives, identify and resolve risks — usually by building prototypes; (3) develop and verify the next-level product; (4) review and plan the next phase. The radius represents cumulative cost and the angle represents progress. *(Draw the spiral figure above.)* Advantages: explicit risk management, suits large and high-risk projects, accommodates changing requirements, customer sees early prototypes. Disadvantages: expensive, depends on risk-assessment expertise, hard to fix deadlines, not suited to small projects. It is a meta-model since it can include waterfall or prototyping in individual loops.

**Q. Explain coupling and cohesion. List their types.** *(5 marks · Likelihood 9/10)*
**Cohesion** is the degree to which elements inside one module belong together; **coupling** is the degree of interdependence between modules. A good design has **high cohesion and low coupling**, which makes modules understandable, testable, reusable and easy to maintain, and limits ripple effects of change. Cohesion types from worst to best: coincidental, logical, temporal, procedural, communicational, sequential, functional. Coupling types from best to worst: data, stamp, control, external, common, content. Example: a module `computeTax(income)` that only computes tax is functionally cohesive and data-coupled; a module that reads a global flag set elsewhere is common-coupled. Related metrics: fan-in (callers of a module — high is good) and fan-out (modules called — keep low).

**Q. Define MTTF, MTTR, MTBF and availability. A system has MTTF 490 h and MTTR 10 h; find availability.** *(5 marks · Likelihood 7/10)*
MTTF is the mean operating time before a failure occurs; MTTR is the mean time taken to repair and restore after failure; MTBF = MTTF + MTTR is the mean time between consecutive failures. Availability is the fraction of time the system is operational: A = MTTF / (MTTF + MTTR). Here MTBF = 490 + 10 = 500 h and A = 490/500 = 0.98 = **98%**. Reliability is the probability of failure-free operation over a time period; a system can be highly available but not reliable if it fails often but is repaired very fast.

**Q. What is the clean room approach to software engineering?** *(5 marks · Likelihood 7/10)*
Clean room software engineering (Harlan Mills, IBM) aims to **prevent** defects rather than find and fix them. Key ideas: (i) formal box-structure specification (black box, state box, clear box); (ii) incremental development by small teams; (iii) **correctness verification** — developers prove code correct through formal reasoning and team review and do not unit-test or execute it; (iv) **statistical usage testing** by an independent team, using test cases drawn from an operational usage profile; (v) certification of reliability as an MTTF estimate. Benefits: very low defect rates and measurable reliability. Limitations: needs mathematical skill, high initial cost and is less suited to rapidly changing requirements.

**Q. Differentiate reverse engineering and re-engineering. How is reverse engineering used in forensic malware analysis?** *(10 marks · Likelihood 9/10)*
**Reverse engineering** is the process of analysing an existing system to identify its components and their relationships and to create representations of it at a higher level of abstraction (design, specification) — it does **not** change the system. **Re-engineering** is the examination and alteration of a system to reconstitute it in a new form: it includes reverse engineering, followed by restructuring and forward engineering.

| Reverse engineering | Re-engineering |
|---|---|
| Code → design/specification | Old system → improved new system |
| Analysis only; no change | Changes structure/code/data |
| Output: documentation, models | Output: new working system |
| Part of re-engineering | Whole process |

Re-engineering process: inventory analysis (decide which applications to re-engineer), document restructuring, reverse engineering (processing, data and UI), code restructuring, data restructuring, and forward engineering. It is used for legacy systems that are business-critical but hard to maintain.

**Forensic use — malware analysis:** a forensic examiner often receives a suspicious executable (from a seized system or phishing mail) without source code. Reverse engineering reveals its behaviour:
1. *Identification:* compute MD5/SHA-256 hashes for integrity and lookup; identify file type by signature (MZ/PE, ELF).
2. *Static analysis:* extract strings (URLs, IPs, file paths), inspect PE headers and imported APIs (e.g. `CreateRemoteThread`, `InternetOpenUrl`), detect packers (UPX), disassemble/decompile with Ghidra or IDA Pro.
3. *Dynamic analysis:* execute in an isolated sandbox VM; record process creation, registry changes (Run keys for persistence), dropped files and network traffic (C2 servers) with Procmon and Wireshark.
4. *Reporting:* produce indicators of compromise (hashes, domains, IPs, mutexes), YARA signatures, timeline, and the malware's capabilities (keylogging, ransomware encryption, data exfiltration) — evidence that links the malware to the offence and suspect.

Precautions: work on copies, preserve hash and chain of custody, never run samples on networked production machines. Thus reverse engineering supports both software maintenance and cyber-crime investigation.

**Q. Explain function-oriented design using DFD and structure charts.** *(10 marks · Likelihood 7/10)*
Function-oriented design views the system as a set of functions transforming inputs into outputs, decomposed top-down (Structured Analysis/Structured Design, Yourdon–DeMarco). **Structured analysis** uses DFDs: external entities (rectangles) are sources/sinks; processes (circles) transform data; data stores (parallel lines) hold data at rest; arrows carry data flows. The **context diagram (Level 0)** shows the whole system as one process with external entities; Level-1 decomposes it into major processes; further levels refine each bubble. Child diagrams must be **balanced** with parents. A **data dictionary** defines every flow and store (e.g. `order = customer_id + {item} + date`). **Structured design** maps the DFD to a **structure chart** — a tree of modules showing which module calls which, with data couples (arrow with open circle) and control flags (filled circle), plus symbols for loops and decisions. Mapping is by **transform analysis** (split DFD into afferent/input, central transform, efferent/output branches) or **transaction analysis** (a dispatcher module selects one of many transaction modules). Good structure charts show high cohesion, low coupling, high fan-in for utility modules and limited fan-out. Example: a library system — Level-0: Member/Librarian ↔ "Library System"; Level-1: Issue book, Return book, Calculate fine, Maintain catalogue with stores BOOKS and MEMBERS.

# Unit 5 — Database and SQL

## 5.1 E-R Diagrams

**Importance 9/10 · very likely** · first syllabus phrase; drawing an ER diagram is a sure question

In simple words: an E-R diagram models real-world entities, their attributes and the relationships between them (Chen, 1976).

- Symbols: entity = rectangle (weak entity = double rectangle); attribute = oval (key underlined, multivalued = double oval, derived = dashed oval, composite = oval with sub-ovals); relationship = diamond (identifying = double diamond); total participation = double line.
- Cardinality: 1:1, 1:N, M:N. Extended ER: specialisation/generalisation (ISA), aggregation.

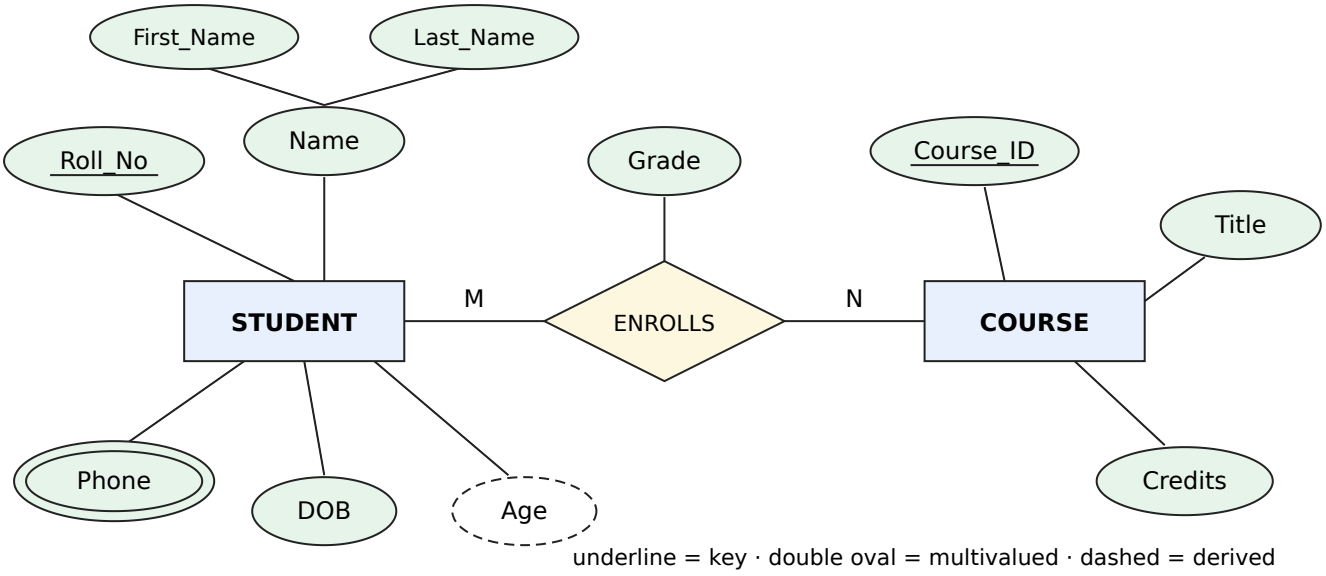



*Fig: ER diagram (Chen notation) — STUDENT M:N COURSE via ENROLLS, with key, composite, multivalued and derived attributes*

## 5.2 Transformation of ER to Relational Design

**Importance 9/10 · very likely** · syllabus phrase "transformation to relational design"

In simple words: fixed rules convert each ER construct into tables, keys and foreign keys.

| ER construct | Relational mapping |
|---|---|
| Strong entity | Table; key attribute → primary key |
| Composite attribute | Store only the simple component attributes |
| Multivalued attribute | Separate table (owner PK + value), composite PK |
| Derived attribute | Not stored (computed, e.g. Age from DOB) |
| Weak entity | Table with owner's PK (as FK) + partial key = PK |
| 1:1 relationship | FK in either table (prefer total-participation side) |
| 1:N relationship | FK on the N side |
| M:N relationship | New table: PKs of both sides (composite PK) + relationship attributes |
| n-ary relationship | New table with PKs of all participants |
| Specialisation (ISA) | Table per subclass with superclass PK, or single table with type column |

Applied to the figure: STUDENT(**roll_no**, first_name, last_name, dob); STUDENT_PHONE(**roll_no, phone**); COURSE(**course_id**, title, credits); ENROLLS(**roll_no, course_id**, grade).

## 5.3 Normalisation 1NF – BCNF

**Importance 9/10 · very likely** · core descriptive question; worked example earns full marks

In simple words: decomposing tables to remove redundancy and update/insert/delete anomalies using functional dependencies (FDs).

| Form | Condition |
|---|---|
| 1NF | All attributes atomic (no repeating groups / multivalued cells) |
| 2NF | 1NF + no **partial dependency** (non-prime attribute depending on part of a candidate key) |
| 3NF | 2NF + no **transitive dependency**; for every FD X→A: X is superkey or A is prime |
| BCNF | For every non-trivial FD X→A, **X is a superkey** (stricter than 3NF) |

**Worked example:** R(StudID, CourseID, StudName, CourseName, Instructor, InstrPhone, Grade) FDs: StudID → StudName; CourseID → CourseName, Instructor; Instructor → InstrPhone; {StudID, CourseID} → Grade. Key = {StudID, CourseID}.

- **1NF:** assume atomic values — R is in 1NF.
- **2NF:** StudName depends only on StudID, CourseName/Instructor only on CourseID (partial) → split: STUDENT(StudID, StudName), COURSE(CourseID, CourseName, Instructor, InstrPhone), ENROL(StudID, CourseID, Grade).
- **3NF:** in COURSE, CourseID → Instructor → InstrPhone (transitive) → split: COURSE(CourseID, CourseName, Instructor), INSTRUCTOR(Instructor, InstrPhone).
- **BCNF:** every determinant now is a key → in BCNF. (Classic 3NF-not-BCNF case: R(Student, Subject, Teacher) with Teacher → Subject, where Teacher is not a superkey.)
- Decomposition should be **lossless-join** and ideally **dependency-preserving** (BCNF may lose dependency preservation; 3NF never does).

## 5.4 SQL — DDL and Data Types

**Importance 7/10 · good chance** · syllabus phrase; write-the-SQL questions

In simple words: DDL defines and changes schema objects; DML manipulates rows; DCL controls access; TCL controls transactions.

- DDL: `CREATE, ALTER, DROP, TRUNCATE, RENAME`. DML: `SELECT, INSERT, UPDATE, DELETE`. DCL: `GRANT, REVOKE`. TCL: `COMMIT, ROLLBACK, SAVEPOINT`.
- Data types: `INT, SMALLINT, DECIMAL(p,s)/NUMERIC, FLOAT, CHAR(n)` (fixed), `VARCHAR(n)` (variable), `DATE, TIME, TIMESTAMP, BOOLEAN, BLOB, CLOB/TEXT`.
- DELETE (DML, row-wise, WHERE allowed, can roll back, fires triggers) vs TRUNCATE (DDL, removes all rows, faster) vs DROP (removes table + structure).

## 5.5 Constraints, Queries, INSERT/UPDATE/DELETE

**Importance 8/10 · likely** · syllabus phrases; likely "write SQL" questions

In simple words: constraints enforce integrity; queries retrieve data.

```
CREATE TABLE dept (dept_id INT PRIMARY KEY, dname VARCHAR(30) NOT NULL UNIQUE);
CREATE TABLE emp (
  emp_id  INT PRIMARY KEY,
  name    VARCHAR(40) NOT NULL,
  salary  DECIMAL(10,2) CHECK (salary > 0),
  city    VARCHAR(20) DEFAULT 'Kohima',
  dept_id INT REFERENCES dept(dept_id) ON DELETE SET NULL
);
INSERT INTO emp VALUES (1, 'Asha', 50000, 'Dimapur', 10);
UPDATE emp SET salary = salary * 1.10 WHERE dept_id = 10;
DELETE FROM emp WHERE emp_id = 1;
-- Query: departments with average salary above 40000
SELECT d.dname, AVG(e.salary) AS avg_sal
FROM emp e JOIN dept d ON e.dept_id = d.dept_id
GROUP BY d.dname HAVING AVG(e.salary) > 40000 ORDER BY avg_sal DESC;
```

- Constraints: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `DEFAULT`. Entity integrity (PK not null), referential integrity (FK matches PK or is null).
- Query clauses order: SELECT–FROM–WHERE–GROUP BY–HAVING–ORDER BY; WHERE filters rows, HAVING filters groups. Subqueries, `IN`, `EXISTS`, `LIKE`, `BETWEEN`; joins (inner, left/right/full outer).

## 5.6 Views

**Importance 7/10 · good chance** · syllabus phrase; security use

In simple words: a view is a virtual table defined by a stored query; it stores no data (unless materialised).

```
CREATE VIEW emp_public AS SELECT emp_id, name, dept_id FROM emp;   -- hides salary
GRANT SELECT ON emp_public TO clerk;
```

- Uses: security (hide columns/rows), simplicity, logical data independence. Simple single-table views are updatable; views with GROUP BY/DISTINCT/joins usually are not. **Materialised view** stores the result physically.

## 5.7 Stored Procedures and Functions

**Importance 7/10 · good chance** · syllabus phrase with "syntax example"

In simple words: named PL/SQL (or T-SQL) blocks stored in the database and executed on the server.

```
-- Procedure (MySQL syntax): raise salary of a department
DELIMITER //
CREATE PROCEDURE raise_salary(IN p_dept INT, IN p_pct DECIMAL(5,2))
BEGIN
  UPDATE emp SET salary = salary * (1 + p_pct/100) WHERE dept_id = p_dept;
END //
-- Function: returns annual salary
CREATE FUNCTION annual_sal(p_id INT) RETURNS DECIMAL(12,2) DETERMINISTIC
BEGIN
  DECLARE s DECIMAL(10,2);
  SELECT salary INTO s FROM emp WHERE emp_id = p_id;
  RETURN s * 12;
END //
DELIMITER ;
CALL raise_salary(10, 5);
SELECT name, annual_sal(emp_id) FROM emp;
```

| Procedure | Function |
|---|---|
| Called with CALL/EXEC | Used inside SQL expressions |
| May return 0..n values via OUT params | Must return exactly one value (RETURN) |
| Can do DML, transactions | Usually no DML side-effects when used in SELECT |

Benefits: less network traffic, reuse, security (grant EXECUTE only), precompiled.

## 5.8 Database Triggers (incl. Audit Trigger)

**Importance 9/10 · very likely** · syllabus phrase + direct forensic link (audit trails)

In simple words: a trigger is a stored block that fires **automatically** on an event (INSERT/UPDATE/DELETE) on a table — Event-Condition-Action rule.

- Timing: BEFORE / AFTER (INSTEAD OF for views); granularity: row-level (`FOR EACH ROW`, uses OLD/NEW) or statement-level.

```
CREATE TABLE emp_audit (
  audit_id INT AUTO_INCREMENT PRIMARY KEY, emp_id INT,
  old_salary DECIMAL(10,2), new_salary DECIMAL(10,2),
  action VARCHAR(10), changed_by VARCHAR(50), changed_at TIMESTAMP);
CREATE TRIGGER trg_emp_audit
AFTER UPDATE ON emp FOR EACH ROW
INSERT INTO emp_audit(emp_id, old_salary, new_salary, action, changed_by, changed_at)
VALUES (OLD.emp_id, OLD.salary, NEW.salary, 'UPDATE', CURRENT_USER(), NOW());
```

- Uses: auditing, enforcing complex business rules, derived columns, replication.
- **Forensic relevance:** audit tables give *who, what, when* (old vs new value, DB user, timestamp) — reconstruct fraud such as an insider altering salary or bank balances. Protect audit tables (insert-only, separate privileges, hash-chain or export to WORM/SIEM) so a suspect cannot erase traces.

## 5.9 SQL Injection — Attack, Prevention, Forensic Traces

**Importance 10/10 · sure to come** · overlap of DBMS + cyber-crime; examiner is a forensic scientist

In simple words: attacker inserts SQL code into an input field so that the application builds and executes a malicious query.

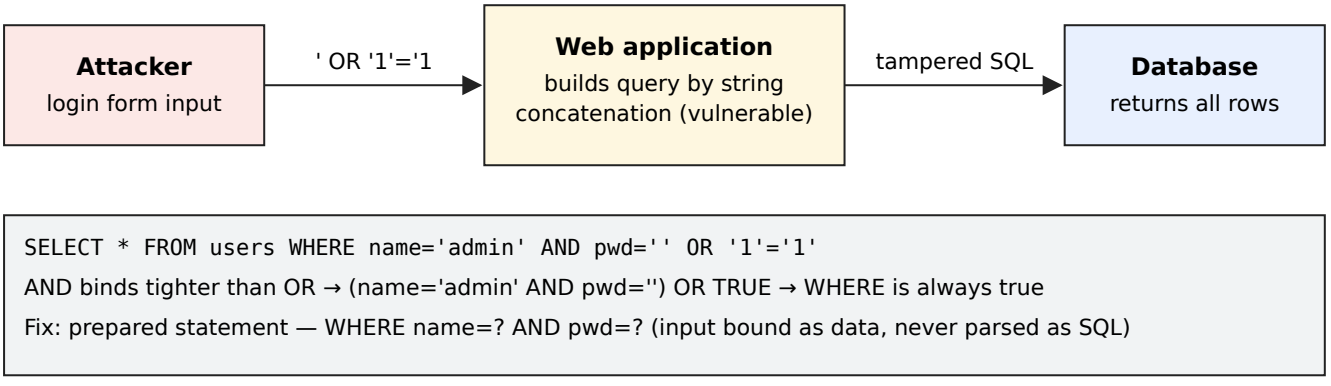



*Fig: SQL injection — malicious input changes the logic of a dynamically built query*

- **Types:** in-band (error-based, UNION-based: `' UNION SELECT username, password FROM users--`), blind (boolean / time-based: `' AND SLEEP(5)--`), out-of-band; second-order (stored then executed later). Stacked: `'; DROP TABLE users;--`.

- **Prevention:** parameterised queries / **prepared statements** (primary defence); stored procedures without dynamic SQL; input validation (whitelist); escaping; **least-privilege** DB account; WAF; hide detailed error messages; ORM.

```
// Vulnerable
$q = "SELECT * FROM users WHERE name='$u' AND pwd='$p'";
// Safe (PDO prepared statement)
$st = $pdo->prepare("SELECT * FROM users WHERE name = ? AND pwd = ?");
$st->execute([$u, $p]);
```

- **Forensic traces:** web-server access logs (Apache/IIS) show payloads in URLs — `%27%20OR%20%271%27%3D%271`, `UNION SELECT`, `information_schema`, `SLEEP(`; many requests from one IP; tool user-agents (e.g. `sqlmap`); HTTP 500 bursts; DB general/query logs, error logs, audit logs; WAF/IDS alerts; unusual large responses (data exfiltration size in log byte column). Correlate timestamps with DB transaction logs to show what data was read/modified.

## 5.10 Transaction Processing — ACID

**Importance 9/10 · very likely** · sure question

In simple words: a transaction is a logical unit of work (sequence of reads/writes) that must run completely or not at all.

- **Atomicity** — all or nothing (undo via log) · **Consistency** — DB moves from one valid state to another · **Isolation** — concurrent transactions do not see each other's partial results · **Durability** — committed changes survive failures (redo via log).
- Responsible component: A → recovery manager; C → programmer + integrity constraints; I → concurrency-control manager; D → recovery manager.

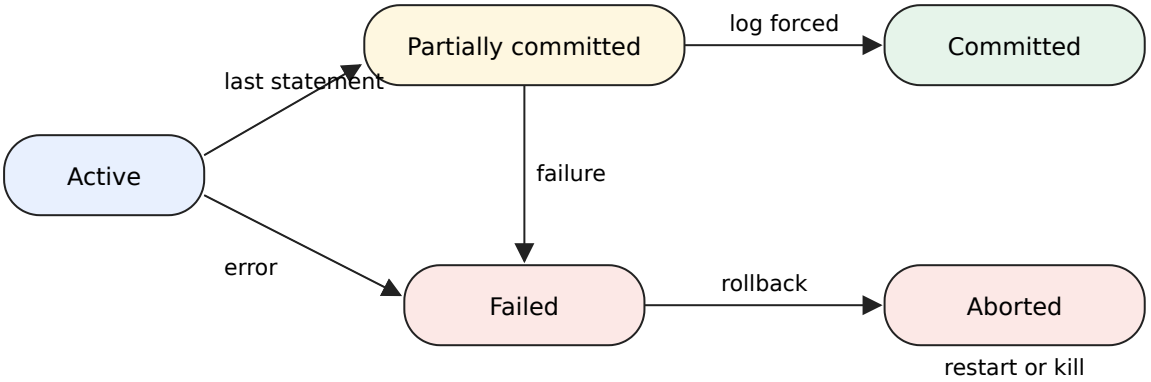



*Fig: Transaction state diagram (Korth) — Active → Partially committed → Committed, or → Failed → Aborted*

- **Schedules:** serial vs concurrent; **conflict-serializable** if the precedence graph is acyclic; recoverable, cascadeless, strict schedules.
- Concurrency problems: lost update, dirty read (uncommitted dependency), unrepeatable read, phantom.

## 5.11 Concurrency Control Techniques

**Importance 9/10 · very likely** · syllabus phrase; 10/15-mark candidate

In simple words: techniques that let transactions run concurrently while keeping the result equivalent to some serial order.

- **Locking:** shared (S, read) and exclusive (X, write) locks; S compatible with S only.

- **Two-Phase Locking (2PL):** growing phase (acquire, no release) then shrinking phase (release, no acquire). Guarantees conflict-serializability; can deadlock. **Strict 2PL:** hold X locks till commit (avoids cascading rollback). **Rigorous 2PL:** hold all locks till commit.
- Deadlock handling: wait-for graph detection; prevention by **wait-die** (older waits, younger dies) and **wound-wait** (older wounds/preempts younger, younger waits); timeouts.
- **Timestamp ordering:** each Ti gets TS(Ti); each item keeps W-TS and R-TS. Ti read(X): reject & roll back if TS(Ti) < W-TS(X). Ti write(X): reject if TS(Ti) < R-TS(X) or TS(Ti) < W-TS(X). Deadlock-free; **Thomas write rule** ignores obsolete writes.
- **MVCC (multiversion):** writes create new versions; readers read the version valid at their start timestamp → readers never block writers (PostgreSQL, Oracle, MySQL InnoDB snapshot isolation).
- **Optimistic (validation):** read → validate → write phases; good for low conflict.
- Granularity: multiple-granularity locking with intention locks (IS, IX, SIX).

## 5.12 Database Recovery Techniques

**Importance 8/10 · likely** · syllabus phrase listing several techniques

In simple words: restoring the database to the last consistent state after a failure (transaction error, system crash, disk failure).

- **Log-based recovery:** write-ahead logging (WAL) — log record `<Ti, X, old, new>` written to stable storage **before** data. Records: `<Ti start>`, `<Ti commit>`, `<Ti abort>`.
- **Deferred update (NO-UNDO/REDO):** writes go to log only, applied to DB after commit; on crash, redo committed transactions, ignore uncommitted.
- **Immediate update (UNDO/REDO):** DB may be updated before commit; on crash, **undo** transactions with start but no commit (backwards), **redo** those with commit (forwards).
- **Checkpoints:** periodically flush log and dirty buffers and write `<checkpoint L>`; recovery scans only after the last checkpoint → faster.
- **Shadow paging:** keep a shadow page table (unchanged) and a current page table; on commit, atomically switch the pointer; crash → shadow table still valid. No log needed; drawbacks — data fragmentation, garbage collection, hard with concurrency.
- Media failure: restore backup + redo from archived log. ARIES = industrial WAL algorithm (analysis, redo, undo).
- Forensic note: transaction logs (e.g. SQL Server .ldf, MySQL binlog, Oracle redo logs) can reconstruct deleted/modified records and timelines.

## 5.13 Object and Object-Relational Databases

**Importance 6/10 · moderate** · syllabus phrase; short note

In simple words: OODBMS stores objects directly (with identity, methods, inheritance); ORDBMS extends a relational DB with object features.

| OODBMS | ORDBMS |
|---|---|
| Objects persist as in OO language (db4o, ObjectStore) | Relational + user-defined types, methods, inheritance (PostgreSQL, Oracle) |
| Query: OQL (ODMG standard) | Query: SQL:1999 extensions |
| Object identity (OID), navigation by references | Tables of typed rows, REF types, nested tables, arrays |
| Suits CAD, multimedia, complex objects | Suits existing SQL apps needing complex types |

Features: complex objects, encapsulation, inheritance, polymorphism, collections; solves impedance mismatch.

## 5.14 Database Security and Authorization

Importance 9/10 · very likely · syllabus phrase + forensic auditing

In simple words: protecting data against unauthorised disclosure, alteration and destruction (CIA).

- **Discretionary access control (DAC)** via privileges:

```
GRANT SELECT, UPDATE(salary) ON emp TO hr_user WITH GRANT OPTION;
REVOKE UPDATE ON emp FROM hr_user CASCADE;
CREATE ROLE auditor;  GRANT SELECT ON emp_audit TO auditor;  GRANT auditor TO ravi;
```

- **Roles (RBAC):** privileges bundled into roles granted to users — easier administration, least privilege.
- **Views** restrict columns/rows (see 5.6). **MAC** (mandatory, Bell–LaPadula: no read up, no write down) for classified data.
- Other measures: authentication, encryption at rest (TDE) and in transit (TLS), statistical-DB inference control, backups, SQL-injection prevention.
- **Auditing:** audit trail of logins, privilege changes, DML on sensitive tables (Oracle `AUDIT`, SQL Server Audit, MySQL audit plugin, triggers) → accountability and forensic evidence.

## 5.15 Data Dictionary

Importance 6/10 · moderate · syllabus phrase; short note

In simple words: "data about data" — the system catalogue storing metadata.

- Contents: table/column names, data types, constraints, indexes, views, users, privileges, storage details, statistics.
- **Active** (auto-updated by DBMS, e.g. Oracle `USER_TABLES`, `information_schema`) vs **passive** (separate documentation).
- Forensic note: attackers query `information_schema` during SQL injection; examiners use the catalogue to map schema of a seized DB.

## Likely questions with model answers

**Q. Explain the rules for transforming an ER diagram into relational tables.** *(5 marks · Likelihood 8/10)*

Each **strong entity** becomes a table with its key as primary key; composite attributes are flattened into simple ones, derived attributes are not stored. A **multivalued attribute** becomes a separate table containing the owner's PK and the value, both forming the PK. A **weak entity** becomes a table whose PK = owner's PK (also FK) + partial key. For a **1:1** relationship, put the FK in one side (preferably the total-participation side); for **1:N**, put the PK of the "1" side as FK in the "N" side table; for **M:N**, create a new table holding both PKs (composite PK) plus relationship attributes. Example: STUDENT(roll_no, name), COURSE(course_id, title), ENROLLS(roll_no, course_id, grade). Specialisation maps to subclass tables sharing the superclass PK.

**Q. What is a database trigger? Write a trigger to audit salary changes.** *(5 marks · Likelihood 8/10)*

A trigger is a stored program that the DBMS executes automatically when a specified event (INSERT, UPDATE, DELETE) occurs on a table, following the event–condition–action model. It can be BEFORE/AFTER and row-level (with OLD and NEW values) or statement-level. Uses: auditing, enforcing complex constraints, maintaining derived data. *(Write the `trg_emp_audit` trigger from 5.8.)* After any UPDATE on `emp`, a row recording employee id, old salary, new salary, DB user and

timestamp is inserted into `emp_audit`. In an investigation this audit trail shows who changed what and when; it should be protected by separate privileges so the suspect cannot delete it.

**Q. Explain the ACID properties of a transaction.** *(5 marks · Likelihood 9/10)*
A transaction is a logical unit of database work. **Atomicity:** all operations happen or none — e.g. a transfer debit without credit is undone using the log. **Consistency:** a correct transaction takes the DB from one consistent state to another (total of balances unchanged). **Isolation:** concurrent transactions behave as if executed serially; intermediate results are invisible to others — ensured by concurrency control (locking, timestamps). **Durability:** once committed, changes survive crashes — ensured by write-ahead log and redo. Atomicity and durability are the recovery manager's job; isolation is the concurrency manager's; consistency is the programmer's along with integrity constraints.

**Q. Differentiate deferred and immediate update recovery.** *(5 marks · Likelihood 7/10)*
In **deferred update**, a transaction's writes are recorded only in the log until it commits; the database is modified after commit. On crash, transactions with a commit record are **redone**; others are simply ignored — **NO-UNDO/REDO**. In **immediate update**, the database may be modified before commit, provided the log record (with old and new values) is written first (WAL). On crash, uncommitted transactions must be **undone** using old values and committed ones **redone** using new values — **UNDO/REDO**. Deferred needs more buffer space; immediate allows earlier writes but needs undo. Both use **checkpoints** to limit the log scan.

**Q. Explain 2NF, 3NF and BCNF with an example.** *(5 marks · Likelihood 8/10)*
2NF: relation in 1NF with no non-prime attribute partially dependent on a candidate key. 3NF: in 2NF and no transitive dependency of a non-prime attribute on a key. BCNF: for every non-trivial FD X→Y, X is a superkey. Example R(StudID, CourseID, StudName, Instructor, InstrPhone, Grade), key {StudID, CourseID}. StudID → StudName is partial → split STUDENT(StudID, StudName). CourseID → Instructor → InstrPhone is transitive → COURSE(CourseID, Instructor), INSTRUCTOR(Instructor, InstrPhone). ENROL(StudID, CourseID, Grade) remains. All determinants are now keys → BCNF. Every BCNF relation is in 3NF but not conversely.

**Q. Write short notes on views and the data dictionary.** *(5 marks · Likelihood 6/10)*
A **view** is a virtual table defined by a query, e.g. `CREATE VIEW emp_public AS SELECT emp_id, name FROM emp;`. It stores no data; each access re-runs the query (a materialised view stores results). Views give security (hide sensitive columns like salary, grant access to the view only), simplify complex joins and provide logical data independence. Simple views are updatable. The **data dictionary** (system catalogue) is the DBMS's metadata repository: names and types of tables and columns, constraints, indexes, views, users, privileges and statistics used by the optimiser. An active dictionary is maintained automatically (e.g. `information_schema`). DBAs and investigators query it to understand the schema and privileges.

**Q. Explain concurrency control techniques: two-phase locking and timestamp ordering.** *(10 marks · Likelihood 8/10)*
Concurrency control ensures that interleaved execution of transactions is serializable, avoiding lost updates, dirty reads and unrepeatable reads.
**Lock-based:** a transaction must obtain a **shared (S)** lock to read and an **exclusive (X)** lock to write; S is compatible only with S. Simple locking alone does not guarantee serializability, so **Two-Phase Locking** is used: in the *growing phase* a transaction may acquire but not release locks; once it releases any lock it enters the *shrinking phase* and may not acquire new ones. The point of last lock acquisition is the *lock point*; transactions serialize in lock-point order, so 2PL guarantees **conflict-serializability**. Basic 2PL can cause cascading rollback and deadlock. **Strict 2PL** holds X-locks until commit/abort → cascadeless, strict schedules; **rigorous 2PL** holds all locks to commit. Deadlocks are detected by a cycle in the wait-for graph (victim rolled back) or prevented by wait-die / wound-wait.

**Timestamp ordering:** each transaction gets a unique timestamp TS(Ti) at start; each data item X has W-timestamp and R-timestamp. For read(X): if TS(Ti) < W-TS(X), Ti would read an overwritten value → roll back Ti; else read and set R-TS = max(R-TS, TS(Ti)). For write(X): if TS(Ti) < R-TS(X) or TS(Ti) < W-TS(X) → roll back; else write and set W-TS = TS(Ti). Under the **Thomas write rule**, an obsolete write (TS(Ti) < W-TS(X)) is simply ignored. Timestamp protocols are deadlock-free (no waiting) but may cause starvation and many restarts.
**MVCC:** keeps multiple versions so readers get a consistent snapshot without blocking writers — used by PostgreSQL and Oracle. **Optimistic** validation suits low-conflict workloads.

| | 2PL | Timestamp ordering |
|---|---|---|
| Order decided by | Lock point (runtime) | Start timestamp |
| Deadlock | Possible | Not possible |
| Conflict handling | Wait | Rollback/restart |

**Q. Describe database recovery techniques.** *(10 marks · Likelihood 7/10)*
Failures: transaction (logical error, deadlock victim), system crash (memory lost, disk intact) and media failure (disk lost). Recovery restores atomicity and durability.
**Log-based recovery** keeps a log on stable storage with `<Ti start>`, `<Ti, X, Vold, Vnew>`, `<Ti commit>`. Under **write-ahead logging**, the log record reaches stable storage before the data page. *Deferred update* writes the DB only after commit → on restart redo committed transactions only (no undo). *Immediate update* may write before commit → undo (backwards, using Vold) transactions without commit, redo (forwards, using Vnew) those with commit. Redo/undo are idempotent.
**Checkpoints:** the system periodically suspends updates, flushes log records and modified buffer blocks to disk, and writes `<checkpoint L>` (L = active transactions). At restart, only transactions active at or after the last checkpoint are examined, cutting recovery time.
**Shadow paging:** the DB is a set of pages accessed through a page table. At transaction start, the current page table is copied to a shadow page table on disk. Updates are written to new pages and only the current table points to them. Commit = atomically switch the root pointer to the current table; abort/crash = discard current and keep shadow. No undo/redo needed, but causes fragmentation, garbage pages and expensive commits; hard to extend to concurrent transactions.
**Media recovery:** periodic full dump/backup + archived log; restore dump and redo committed transactions. Modern DBMSs use **ARIES** (analysis, redo, undo, with LSNs and compensation log records). Transaction logs are also valuable forensic artifacts for reconstructing deleted or altered records.

**Q. What is SQL injection? Explain with an example, how it can be prevented, how it is detected forensically, and discuss database security and authorization mechanisms.** *(15 marks · Likelihood 9/10)*
**SQL injection (SQLi)** is a code-injection attack in which an attacker supplies input containing SQL syntax to an application that builds queries by string concatenation, so the input is executed as part of the query. It is in the OWASP Top 10 (Injection) and a punishable offence under the IT Act 2000 (Sec 43 and 66 — unauthorised access/damage to data).
**Example:** login code `"SELECT * FROM users WHERE name='" + u + "' AND pwd='" + p + "'"`. Attacker enters user `admin` and password `' OR '1'='1`. The query becomes `SELECT * FROM users WHERE name='admin' AND pwd='' OR '1'='1'`. Since AND has higher precedence, the condition is (name='admin' AND pwd='') OR TRUE, which is always true → rows returned and the attacker logs in without a password. *(Draw SQLi figure.)*
**Variants:** UNION-based (`' UNION SELECT username,password FROM users--`) to extract data; error-based; blind boolean/time-based (`' AND SLEEP(5)--`); stacked queries (`'; DROP TABLE users;--`); second-order injection; reading `information_schema` to map the database.
**Prevention:** (1) **prepared statements / parameterised queries** — the SQL structure is compiled first and input bound as data (`prepare("... WHERE name=? AND pwd=?")`); (2) stored

procedures without dynamic SQL; (3) whitelist input validation and type checking; (4) escaping as a last resort; (5) **least privilege** — the application account should not own tables or have DROP/GRANT; (6) generic error messages; (7) web application firewall; (8) store password hashes (bcrypt), not plaintext; (9) regular code review and penetration testing.
**Forensic detection:** (i) web server access logs show encoded payloads such as `%27%20OR%20%271%27%3D%271`, `UNION%20SELECT`, `information_schema`, `SLEEP(`, comment sequences `--` and `#`; (ii) many rapid requests from one IP and tool user-agents like `sqlmap`; (iii) spikes of HTTP 500 errors and abnormally large response sizes (data exfiltration); (iv) database query/general logs, error logs, audit trails and transaction logs showing unusual queries, new accounts or modified rows; (v) WAF/IDS alerts. The examiner preserves logs with hashes and chain of custody, builds a timeline correlating IP, timestamp, payload and DB changes, and maps the IP to the subscriber via the ISP.
**Database security and authorization:** goals are confidentiality, integrity and availability. *Authentication* verifies users (passwords, MFA). *Discretionary access control* uses privileges: `GRANT SELECT, UPDATE(salary) ON emp TO hr_user WITH GRANT OPTION;` and `REVOKE UPDATE ON emp FROM hr_user CASCADE;` (CASCADE also revokes privileges that user granted onward). *Roles* (RBAC): `CREATE ROLE clerk; GRANT SELECT ON orders TO clerk; GRANT clerk TO asha;` simplify administration and enforce least privilege. *Views* restrict access to selected rows/columns (e.g. a view without salary). *Mandatory access control* labels data and users (Top Secret … Unclassified) with Bell–LaPadula rules. *Encryption* at rest (TDE) and in transit (TLS). *Inference control* for statistical databases. *Auditing* — DBMS audit features and audit triggers record logins, failed attempts, privilege changes and DML on sensitive tables, giving accountability and admissible evidence. Together, secure coding (prepared statements) plus least-privilege authorization and auditing both prevent SQLi and ensure that any attack leaves traceable evidence.

# Unit 4 — Programming in C, OOP, C++ & Web Programming

## C tokens and identifiers

**Importance 7/10 · good chance** · basic syllabus phrase; easy 5-mark definition
In simple words: tokens are the smallest meaningful units the C compiler reads.

- **Six token types:** keywords (`int`, `while`, 32 in C89), identifiers (`sum`, `main`), constants (`10`, `3.5`, `'a'`), string literals (`"hi"`), operators (`+ ++ == &&`), special symbols/punctuators (`; , { } [ ] ( )`).
- **Identifier rules:** letters, digits, underscore only; must not start with a digit; case-sensitive (`Sum` ≠ `sum`); cannot be a keyword; no spaces or special characters. Valid: `_total`, `rate2`. Invalid: `2rate`, `int`, `my-var`.

## Data types and sizes

**Importance 7/10 · good chance** · MCQ favourite; size tables asked in 5-mark questions

| Type | Typical size (64-bit gcc) | Format | Range / note |
|---|---|---|---|
| `char` | 1 byte | `%c` | −128 to 127 (signed) |
| `short` | 2 bytes | `%hd` | −32768 to 32767 |
| `int` | 4 bytes | `%d` | about ±2.1 × $10^9$ |
| `long` | 8 bytes (Linux 64-bit) | `%ld` | 4 bytes on Windows |
| `float` | 4 bytes | `%f` | ~6–7 significant digits |
| `double` | 8 bytes | `%lf` | ~15–16 significant digits |
| `void` | — | — | no value |

- Categories: **primary** (int, char, float, double, void), **derived** (array, pointer, function), **user-defined** (struct, union, enum, typedef).
- Exam tip: sizes are implementation-dependent; the standard only guarantees `char` = 1 and `short` ≤ `int` ≤ `long`.

## Control structures (sequence, selection, iteration)

**Importance 6/10 · moderate** · standard theory; pair with a small flowchart in answers

| Structure | Meaning | C constructs |
|---|---|---|
| Sequence | statements run one after another | ordinary statements |
| Selection (decision) | choose a path based on a condition | `if`, `if-else`, `else-if` ladder, `switch` |
| Iteration (loop) | repeat while condition holds | `for`, `while` (entry-controlled), `do-while` (exit-controlled, runs at least once) |
| Jump | transfer control | `break`, `continue`, `goto`, `return` |

## Arrays and strings

**Importance 6/10 · moderate** · definitions + string functions are MCQ material

- **Array:** collection of same-type elements in contiguous memory, index from 0. `int a[5];` Address of `a[i]` = base + i × sizeof(element). Array name decays to pointer to first element.
- 2-D array stored in **row-major** order in C: address of `a[i][j]` = base + (i × cols + j) × size.
- **String:** array of `char` terminated by the null character `'\0'`. `char s[] = "CSE";` uses 4 bytes.
- `<string.h>`: `strlen` (excludes `'\0'`), `strcpy`, `strcat`, `strcmp` (returns 0 if equal), `strrev` (non-standard).

## Structure vs union

**Importance 8/10 · likely** · classic "differentiate" question; sizeof output is MCQ bait

In simple words: a struct gives every member its own memory; a union makes all members share one memory area.

| Point | Structure | Union |
|---|---|---|
| Keyword | `struct` | `union` |
| Memory | each member has separate storage | all members share the same storage |
| Size | ≥ sum of members (plus padding) | size of largest member (plus padding) |
| Active members | all members valid at once | only the last-written member is meaningful |
| Use | records (student, employee) | saving memory, variant data, type punning |

```
#include <stdio.h>
struct S { char c; int i; double d; };
union  U { char c; int i; double d; };
int main(void){
    printf("sizeof(struct S) = %zu\n", sizeof(struct S));
    printf("sizeof(union U)  = %zu\n", sizeof(union U));
    union U u; u.i = 65; printf("u.c = %c\n", u.c);
    return 0;
}
```

Output (gcc, x86-64 Linux, verified):

```
sizeof(struct S) = 16
sizeof(union U)  = 8
u.c = A
```

Working: struct = c(1) + 3 padding + i(4) + d(8) = 16. Union = largest member, double = 8. `u.c` reads the low byte of 65 → `'A'` (on a little-endian machine).

## Pointers

**Importance 8/10 · likely** · pointer output questions are the commonest C MCQs

In simple words: a pointer is a variable that stores the address of another variable.

- `int *p = &x;` → `&` gives address, `*` dereferences. `p + 1` moves by `sizeof(int)` bytes (pointer arithmetic).
- `a[i]` ≡ `*(a + i)`. NULL pointer = points nowhere; dangling pointer = points to freed memory; void pointer = generic, must be cast before dereference.
- Uses: call by reference, dynamic memory (`malloc`/`calloc`/`realloc`/`free`), linked structures, arrays and strings.

```
#include <stdio.h>
void swap(int *a, int *b){ int t = *a; *a = *b; *b = t; }
int main(void){
    int arr[] = {10, 20, 30};
    int *p = arr;
    printf("%d %d %d\n", *p, *(p + 1), p[2]);
    int x = 5, y = 9;
    swap(&x, &y);
    printf("x=%d y=%d\n", x, y);
    return 0;
}
```

Output (verified with gcc): `10 20 30` then `x=9 y=5`.

## OOP concepts: class, object, instantiation

**Importance 9/10 · very likely** · syllabus phrase "OOP concepts" — near-certain 5- or 10-mark

In simple words: a class is a blueprint; an object is a real thing built from it.

- **Class:** user-defined type bundling data members and member functions, with access control (`private`, `protected`, `public`).
- **Object:** an instance of a class; occupies memory; has state (data values), behaviour (functions) and identity.
- **Instantiation:** creating an object from a class — `Student s1;` (stack) or `Student *p = new Student();` (heap). The constructor runs at instantiation.
- **Four pillars:** encapsulation (data + functions bound, data hidden), abstraction (show essentials, hide details), inheritance (reuse), polymorphism (one interface, many forms).
- Other terms: message passing, dynamic binding.

## Inheritance and its types

**Importance 8/10 · likely** · "explain types of inheritance with diagram" is a standard 10-mark

In simple words: a new (derived) class reuses and extends an existing (base) class.

| Type | Shape | Example |
|---|---|---|
| Single | A → B | `class B : public A` |
| Multiple | A, B → C | `class C : public A, public B` (C++ allows, Java does not for classes) |
| Multilevel | A → B → C | grandparent chain |
| Hierarchical | A → B, A → C | one base, many derived |
| Hybrid | combination, e.g. diamond A → B, C → D | solved by `virtual` base class |

Visibility: `public` inheritance keeps public/protected as is; `protected` makes them protected; `private` makes them private. Private members of base are never directly accessible in derived.

## Polymorphism and overloading

**Importance 9/10 · very likely** · compile-time vs run-time differentiation is a sure question

| Point | Compile-time (static) | Run-time (dynamic) |
|---|---|---|
| Mechanism | function overloading, operator overloading, templates | virtual functions + overriding via base pointer/reference |
| Binding | early binding, decided by compiler | late binding, via vtable at run time |
| Speed | faster | slight overhead |
| Example | `area(int)`, `area(int,int)` | `Base *p = new Derived; p->show();` |

- **Function overloading:** same name, different parameter list (number/type/order). Return type alone cannot distinguish.
- **Operator overloading:** give meaning to operators for user types — `Complex operator+(const Complex&)`. Cannot overload `::`, `.`, `.*`, `?:`, `sizeof`.
- **Overriding:** derived class redefines a base virtual function with the same signature.

```
int    area(int s)            { return s * s; }
int    area(int l, int b)     { return l * b; }
double area(double r)         { return 3.14 * r * r; }
// cout << area(4) << " " << area(3, 5) << " " << area(2.0);   → 16 15 12.56  (verified g++)
```

## C++ basics: tokens, variables, constants, operators, control statements

**Importance 5/10 · possible** · overlaps C; know the C++-only additions

- Tokens same as C plus extra keywords (`class`, `new`, `delete`, `this`, `virtual`, `template`, `try`, `catch`, `throw`, `namespace`, `bool`).
- Constants: `const int N = 10;`, `constexpr`, literals, `enum`. References: `int &r = x;` (alias, must be initialised).
- C++-specific operators: `::` scope resolution, `new`/`delete`, `<<`/`>>` stream insertion/extraction, `.*` and `->*` member pointers.
- Control statements identical to C (`if`, `switch`, `for`, `while`, `do-while`, `break`, `continue`), plus range-for in C++11.

## Functions and parameter passing

**Importance 8/10 · likely** · "call by value vs call by reference" is a perennial 5-mark

| Method | What is passed | Can change caller's variable? | Syntax |
|---|---|---|---|
| By value | copy of value | No | `void f(int a)` |
| By reference | alias of variable | Yes | `void f(int &a)` |
| By pointer (address) | address | Yes, through `*` | `void f(int *a)` |

```
#include <iostream>
using namespace std;
void byValue(int a)  { a = a + 10; }
void byRef(int &a)   { a = a + 10; }
void byPtr(int *a)   { *a = *a + 10; }
int main() {
    int x = 1;
    byValue(x); cout << "after byValue: " << x << '\n';
    byRef(x);   cout << "after byRef:   " << x << '\n';
    byPtr(&x);  cout << "after byPtr:   " << x << '\n';
}
```

Output (verified g++): `after byValue: 1`, `after byRef: 11`, `after byPtr: 21`.
Other function features: default arguments, inline functions, friend functions, static members.

## Constructors and destructors (order of calls)

**Importance 8/10 · likely** · order of execution is an MCQ and 5-mark favourite

In simple words: a constructor initialises an object when it is created; a destructor cleans up when it dies.

- Constructor: same name as class, no return type, called automatically, can be overloaded. Types: default, parameterised, copy (`X(const X&)`).
- Destructor: `~ClassName()`, no arguments, no return type, cannot be overloaded (one per class).
- **Order:** constructors run **base → derived**; destructors run **derived → base** (reverse). Make base destructor `virtual` when deleting through a base pointer.

```
class Base {
public:
    Base()  { cout << "Base ctor\n"; }
    virtual ~Base() { cout << "Base dtor\n"; }
    virtual void show() { cout << "Base::show\n"; }
};
class Derived : public Base {
public:
    Derived()  { cout << "Derived ctor\n"; }
    ~Derived() { cout << "Derived dtor\n"; }
    void show() override { cout << "Derived::show\n"; }
};
int main() { Base *p = new Derived(); p->show(); delete p; }
```

Output (verified g++): `Base ctor` · `Derived ctor` · `Derived::show` · `Derived dtor` · `Base dtor`.

## Templates and exception handling

**Importance 8/10 · likely** · both named in syllabus; example code earns full marks

In simple words: templates write code once for any type; exceptions handle run-time errors without crashing.

- **Template:** generic function/class; compiler generates a version per type used (compile-time polymorphism). Function template and class template (e.g. STL `vector<T>`).

- **Exception handling:** `try` block holds risky code, `throw` raises an exception object, `catch(type)` handles it. `catch(...)` catches all. Unhandled exception → `std::terminate`.

```
#include <iostream>
#include <stdexcept>
using namespace std;
template <typename T>
T maxOf(T a, T b) { return (a > b) ? a : b; }
double divide(int a, int b) {
    if (b == 0) throw runtime_error("divide by zero");
    return (double)a / b;
}
int main() {
    cout << maxOf(3, 7) << " " << maxOf(2.5, 1.5) << " " << maxOf('a', 'z') << '\n';
    try {
        cout << divide(10, 4) << '\n';
        cout << divide(1, 0) << '\n';
    } catch (const runtime_error &e) {
        cout << "Caught: " << e.what() << '\n';
    }
    cout << "Program continues\n";
}
```

Output (verified g++):

```
7 2.5 z
2.5
Caught: divide by zero
Program continues
```

## HTML vs DHTML

**Importance 7/10 · good chance** · "differentiate" phrase from syllabus

In simple words: HTML gives a fixed page; DHTML lets the page change in the browser without reloading.

| Point | HTML | DHTML |
|---|---|---|
| Nature | markup language | not a language — a combination: HTML + CSS + JavaScript + DOM |
| Content | static once loaded | dynamic; changes after load (show/hide, animate, restyle) |
| Server trips | change needs a new page from server | changes done client-side |
| Example | `<p>Hello</p>` | JS changes `document.getElementById('x').style.color` on click |

## XML: well-formed rules, DTD and Schema

**Importance 7/10 · good chance** · well-formed vs valid is a classic short question

In simple words: XML describes and carries data with user-defined tags; HTML displays data with fixed tags.

- **Well-formed rules:** exactly one root element; every start tag has an end tag (or is self-closed `<br/>`); tags properly nested; case-sensitive; attribute values in quotes; no duplicate attribute in one element; special chars escaped (`<`, `&`).
- **Valid** XML = well-formed + conforms to a DTD or XML Schema.
- **DTD:** older, non-XML syntax, no data types — `<!ELEMENT student (name, roll)>`.

- **XML Schema (XSD):** written in XML, supports data types (`xs:int`, `xs:date`), namespaces, min/max occurs.

## Scripting: client side vs server side

**Importance 6/10 · moderate** · standard comparison table

| Point | Client-side | Server-side |
|---|---|---|
| Runs on | user's browser | web server |
| Languages | JavaScript | PHP, JSP, Python, Node.js, ASP.NET |
| Source visible? | yes (view source) | no, user sees only output HTML |
| Uses | validation, DOM effects, UI | database access, login, sessions, dynamic pages |
| Security | cannot be trusted for checks | secure, controls data |

## Java features: platform independence, JVM and bytecode

**Importance 7/10 · good chance** · "write once, run anywhere" explanation expected

- Features: simple, object-oriented, **platform independent**, secure (no explicit pointers, bytecode verifier, sandbox), robust (garbage collection, exception handling), multithreaded, distributed, portable.
- **How independence works:** `javac` compiles `.java` → `.class` **bytecode** (not machine code). Any machine with a **JVM** runs it — the JVM interprets/JIT-compiles bytecode to native code. JVM is platform-specific; bytecode is not.
- JDK = JRE + dev tools; JRE = JVM + libraries.

## Servlets (life cycle)

**Importance 7/10 · good chance** · life-cycle diagram is an easy 5-mark

In simple words: a servlet is a Java class running inside a web server (container such as Tomcat) that answers HTTP requests.

- **init()** — called once when the servlet is first loaded; initialisation (e.g. DB connection).
- **service()** — called for every request; dispatches to `doGet()`/`doPost()`; each request on a new thread.
- **destroy()** — called once before unloading; release resources. Then garbage collected.
- Advantage over CGI: one instance, many threads (no new process per request), portable, secure.

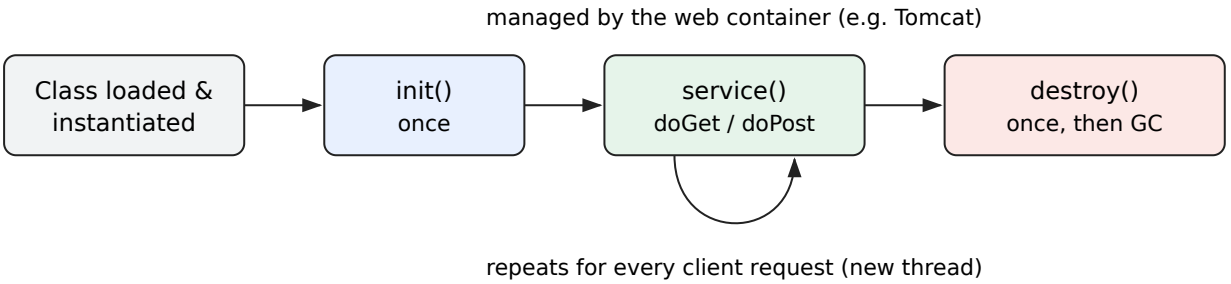



*Fig: Servlet life cycle — init() once, service() per request, destroy() once*

## Applets (life cycle)

**Importance 5/10 · possible** · syllabus phrase, but technology is obsolete

In simple words: an applet was a small Java program embedded in a web page and run by the browser's JVM plug-in.

- Life cycle: **init()** (once, set up) → **start()** (each time page becomes visible) → **paint(Graphics g)** (draw output) → **stop()** (page hidden/left) → **destroy()** (once, before unloading).
- Applet vs application: no `main()`, runs in browser sandbox, restricted file/network access.
- ⚠️ Applet API deprecated in Java 9 and marked for removal (JEP 398, Java 17); browsers dropped the plug-in. Mention this in answers but still write the life cycle.

# Unit 6 — Data and File Structures

## Data vs information; data structure definition

**Importance 6/10 · moderate** · opening definitions of the unit

In simple words: data is raw facts; information is processed, meaningful data. A data structure is a way of organising data in memory so operations are efficient.

| Data | Information |
|---|---|
| raw, unorganised facts (e.g. 72, 85, 90) | processed, organised, meaningful (e.g. "class average = 82") |
| input to processing | output of processing |
| may not help decisions alone | supports decisions |

- Operations on data structures: traversal, insertion, deletion, searching, sorting, merging.
- Abstract Data Type (ADT): logical description (values + operations) independent of implementation, e.g. Stack ADT with push/pop.

## Classification of data structures

**Importance 8/10 · likely** · "classify data structures with diagram" is a likely 5-mark

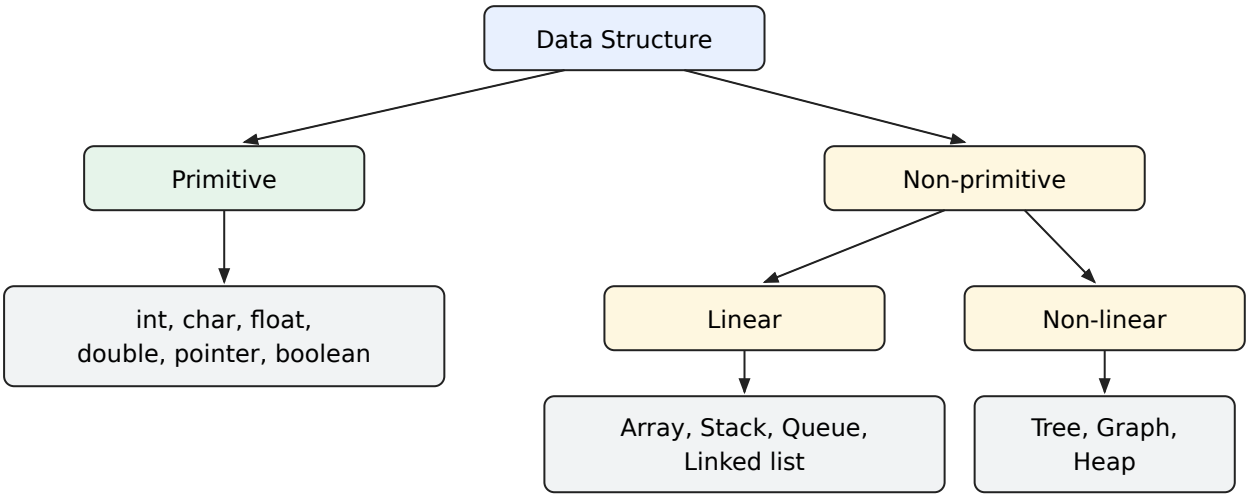



*Fig: Classification of data structures*

- **Primitive:** built into the language, directly operated by machine instructions.
- **Non-primitive linear:** elements in a sequence, each has one predecessor/successor.
- **Non-primitive non-linear:** hierarchical or network relationships (one element linked to many).
- Also: static (array, fixed size) vs dynamic (linked list, grows at run time).

## Arrays and stacks

**Importance 7/10 · good chance** · stack applications are standard 5-mark material

- **Array:** fixed size, contiguous, O(1) random access, O(n) insert/delete in middle.

- **Stack:** LIFO; operations `push`, `pop`, `peek`, `isEmpty`; all O(1). Overflow (push on full), underflow (pop on empty).
- **Applications:** function calls/recursion (call stack), expression evaluation and infix → postfix conversion, parenthesis matching, undo in editors, browser back button, DFS, backtracking.
- Example: infix `A + B * C` → postfix `A B C * +`.

## Queues: simple, circular, deque, priority

**Importance 7/10 · good chance** · circular queue condition formulas are MCQ favourites

- **Queue:** FIFO; `enqueue` at rear, `dequeue` at front. Uses: CPU scheduling, printer spooling, BFS, buffers.
- **Circular queue:** rear wraps to index 0 to reuse freed space. `rear = (rear + 1) % N`. Full when `(rear + 1) % N == front`; empty when `front == -1` (or `front == rear` in the one-slot-empty version).
- **Deque (double-ended):** insert/delete at both ends; input-restricted and output-restricted variants.
- **Priority queue:** element with highest priority served first; best implemented with a heap (insert and delete O(log n)).

## Linked lists (singly, doubly, circular)

**Importance 7/10 · good chance** · comparison table answers a 5-mark directly

| Type | Node | Traversal | Note |
|---|---|---|---|
| Singly | data + next | forward only | least memory |
| Doubly | prev + data + next | both directions | easy delete of a given node, more memory |
| Circular | last node's next → first | loop from any node | used in round-robin scheduling |

- vs array: dynamic size, O(1) insert/delete at known position, but no random access (O(n) search) and extra pointer memory.

## Trees: binary tree, BST, traversals (worked)

**Importance 9/10 · very likely** · traversal outputs appear in nearly every paper

- **Binary tree:** each node has at most 2 children. Max nodes at level i = $2^i$ (root level 0); max nodes in height h = $2^{h+1} - 1$.
- **BST:** left subtree keys < node < right subtree keys. Search/insert/delete O(log n) average, O(n) worst (skewed). **Inorder of a BST gives sorted order.**
- Worked example: insert 50, 30, 70, 20, 40, 60, 80 into an empty BST → root 50; 30 (children 20, 40) on left; 70 (children 60, 80) on right.

| Traversal | Rule | Output |
|---|---|---|
| Preorder | Root, Left, Right | 50 30 20 40 70 60 80 |
| Inorder | Left, Root, Right | 20 30 40 50 60 70 80 |
| Postorder | Left, Right, Root | 20 40 30 60 80 70 50 |
| Level order | level by level (queue) | 50 30 70 20 40 60 80 |

## Graphs: representations, BFS and DFS

**Importance 7/10 · good chance** · BFS vs DFS table is a standard question

- Graph G = (V, E); directed/undirected; weighted/unweighted.

- **Adjacency matrix:** V × V, O(V²) space, O(1) edge check — good for dense graphs. **Adjacency list:** O(V + E) space — good for sparse graphs.

| Point | BFS | DFS |
|---|---|---|
| Data structure | queue | stack / recursion |
| Order | level by level | goes deep first, then backtracks |
| Shortest path (unweighted) | yes | no |
| Time | O(V + E) | O(V + E) |
| Uses | shortest path, web crawling | topological sort, cycle detection, connected components |

## Heaps and priority queues

**Importance 6/10 · moderate** · heap property and heap sort are MCQ topics

- **Heap:** complete binary tree with heap property — max-heap (parent ≥ children) or min-heap (parent ≤ children). Stored in an array: children of i at 2i+1, 2i+2 (0-based); parent at ⌊(i−1)/2⌋.
- Insert / delete-max: O(log n); build-heap: O(n); heap sort: O(n log n), in place, not stable.

## File structures: fields, records, files

**Importance 7/10 · good chance** · basic definitions open every file-organisation answer

In simple words: a field is one data item, a record is a group of related fields, a file is a collection of related records.

- **Field:** smallest named unit of data (e.g. `RollNo`). **Key field** uniquely identifies a record (primary key); secondary keys are non-unique.
- **Record:** collection of related fields about one entity (one student). Fixed-length or variable-length (delimiters / length prefix).
- **File:** collection of related records stored on secondary storage (student file). Physical record = block read/written in one I/O; blocking factor = records per block.

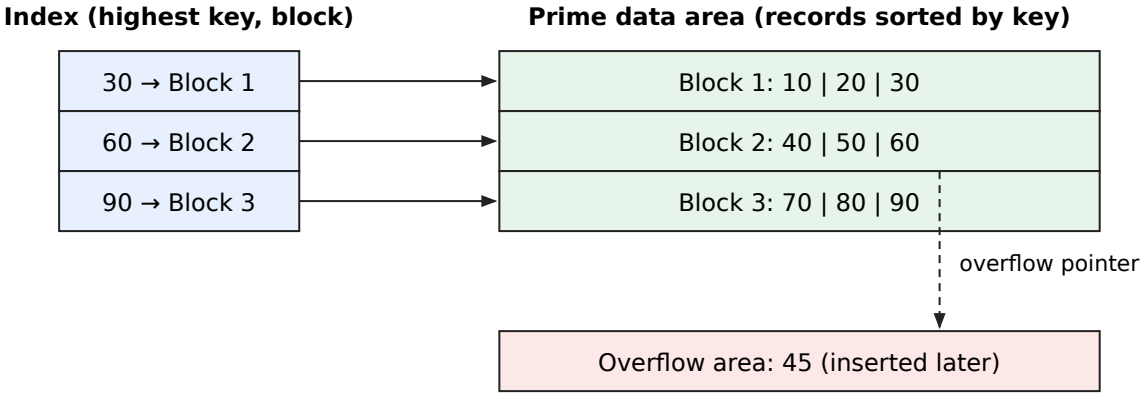
## File organisation: sequential, direct, index-sequential, relative

**Importance 9/10 · very likely** · exact syllabus phrase; strong 10/15-mark candidate

In simple words: file organisation decides how records are placed on disk, which decides how fast we can find them.

- **Sequential:** records stored one after another, usually sorted on a key. Read in order. Good for batch processing (payroll) where most records are processed; slow for a single lookup (O(n)); insertion needs rewriting the file. Suits tape.
- **Direct (random / hashed):** a hash function maps key → disk address; record read in about one access. Best for online, random lookups (reservations); no sequential order; collisions must be handled.
- **Index-sequential (ISAM):** records sorted on key **plus** an index (key → block). Supports both sequential processing and fast random access via index; new records go to **overflow area**, so periodic reorganisation is needed.
- **Relative:** record stored at a position given by its relative record number (1st, 2nd, …, nth slot); address = start + (n − 1) × record size. Fast direct access by number; key must map to a slot number; empty slots waste space.

| Point | Sequential | Direct (hashed) | Index-sequential | Relative |
|---|---|---|---|---|
| Placement | in key order | address = h(key) | key order + index | slot = record number |
| Random access | slow O(n) | fastest (~1 access) | fast (index + block) | fast (computed) |
| Sequential access | fastest | poor | good | possible by slot order |
| Extra storage | none | spare buckets | index + overflow area | empty slots |
| Insertion | rewrite file | easy (hash) | overflow area | into slot |
| Media | tape or disk | disk only | disk only | disk only |
| Use | payroll, billing | ATM, reservations | student/bank records | fixed numbered records |




Search 50: index says key ≤ 60 is in Block 2 → read Block 2 only → found

*Fig: Index-sequential file — index, prime area and overflow area*

## Hashing: hash functions and collision resolution

**Importance 9/10 · very likely** · worked linear-probing numerical is a likely 10-mark part

In simple words: hashing turns a key into a table address so a record can be found in about one step.

- **Hash functions:** division `h(k) = k mod m` (m prime), mid-square (square, take middle digits), folding (split key into parts and add), multiplication method. Good hash = fast, uniform spread, few collisions.
- **Collision:** two keys map to the same slot.
- **Open addressing (closed hashing):** linear probing `(h + i) mod m` (causes primary clustering), quadratic probing `(h + i²) mod m` (secondary clustering), double hashing `(h₁ + i·h₂) mod m`.
- **Chaining (open hashing):** each slot holds a linked list of all keys hashing there.
- Load factor α = n / m; keep α below about 0.7 for open addressing.

Worked example (verified with Python): insert 18, 41, 22, 44, 59, 32, 31, 73 into a table of size m = 11, h(k) = k mod 11, linear probing.

| Key | h(k) | Probing | Final slot |
|---|---|---|---|
| 18 | 7 | free | 7 |
| 41 | 8 | free | 8 |
| 22 | 0 | free | 0 |
| 44 | 0 | 0 full → 1 | 1 |
| 59 | 4 | free | 4 |
| 32 | 10 | free | 10 |
| 31 | 9 | free | 9 |
| 73 | 7 | 7, 8, 9, 10, 0, 1 full → 2 | 2 |

Final table (slot 0–10): `22, 44, 73, –, 59, –, –, 18, 41, 31, 32`. Key 73 needs 6 extra probes — this is primary clustering.

## Inverted lists and multi-lists

Importance 8/10 · likely · exact syllabus phrase, never asked yet; forensic link makes it likely

In simple words: both are ways to find records by secondary (non-key) attributes quickly.

- **Inverted list (inverted file/index):** for each value of a secondary key, keep a list of addresses (or IDs) of all records having that value. E.g. Dept = "CSE" → {R1, R4, R9}. Queries like "Dept = CSE AND City = Kohima" become intersections of lists without scanning the file. Cost: index must be updated on every insert/delete.
- **Multi-list:** the records themselves carry extra pointer fields; all records with the same secondary-key value are linked into a chain; the index stores only the head of each chain and its length. Saves index space, but a query must walk the chain on disk.

| Point | Inverted list | Multi-list |
|---|---|---|
| Where links are kept | in the index (address lists) | inside data records (pointer fields) |
| Index size | large | small (head + length) |
| Boolean queries | fast (list intersection, no record access) | slower (traverse chains) |
| Update | change index lists | adjust record pointers |

- **Forensic note:** keyword search in forensic suites (e.g. Autopsy with its Solr-based keyword index, FTK's dtSearch index) builds an **inverted index** of every word in the acquired image → list of files/offsets. That is why indexing takes long once, but each later keyword search returns hits almost instantly.

## B-trees and B+ trees

Importance 9/10 · very likely · syllabus phrase; core of DBMS indexing; likely 10/15-mark

In simple words: a B-tree is a short, wide, always-balanced search tree designed so each node fits one disk block.

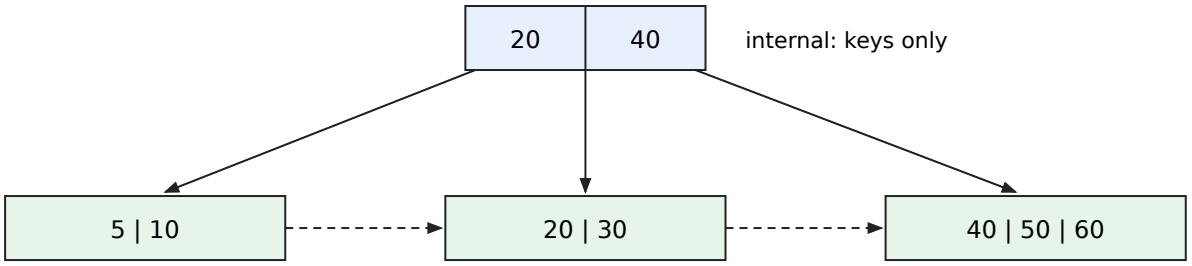
**B-tree of order m properties:**

- Every node has at most m children and m − 1 keys.
- Every internal node except root has at least ⌈m/2⌉ children (⌈m/2⌉ − 1 keys); root has at least 2 children if not a leaf.
- All leaves at the same level (perfectly height-balanced); keys in a node sorted.
- Insertion splits a full node and pushes the median key up; tree grows at the root.

Search/insert/delete O(log n).

**B+ tree:** variant where **all data (record pointers) are in leaves**; internal nodes hold only keys for routing; leaves are linked left-to-right for fast range and sequential scans. Used by almost all DBMS indexes and file systems (NTFS, ext4 directories via HTree, XFS).

| Point | B-tree | B+ tree |
|---|---|---|
| Data / record pointers | in internal and leaf nodes | only in leaf nodes |
| Key repetition | each key appears once | internal keys repeated in leaves |
| Leaf linking | no | leaves linked (sequential access) |
| Range query | slow (tree traversal) | fast (follow leaf chain) |
| Search ends | may stop at an internal node | always reaches a leaf |
| Fan-out | lower (internal nodes hold data) | higher, so shorter tree |




Leaves hold all keys + record pointers; dashed arrows = leaf chain for range scans
Order m = 4: max 4 children / 3 keys per node; keys < 20 left, 20–39 middle, ≥ 40 right

*Fig: A small B+ tree of order 4 with linked leaves*

## Likely questions with model answers

**Q. Differentiate between structure and union in C with an example.** *(5 marks · Likelihood 8/10)*
A structure is a user-defined type in which every member gets its own memory, so all members can hold values at the same time; its size is at least the sum of member sizes plus padding. A union is a user-defined type in which all members share one memory location, so only the most recently written member is meaningful; its size equals the largest member (plus padding). Structures model records such as a student; unions save memory when only one of several alternatives is needed (variant data, protocol fields).
Example: `struct S { char c; int i; double d; };` and `union U` with the same members. On gcc x86-64, `sizeof(struct S)` = 16 (1 + 3 padding + 4 + 8) while `sizeof(union U)` = 8 (the double). Writing `u.i = 65` and reading `u.c` prints `A`, showing the shared memory. Keyword: `struct` vs `union`; member access with `.` or `->` is the same for both.

**Q. Explain compile-time and run-time polymorphism in C++.** *(5 marks · Likelihood 9/10)*
Polymorphism means one name, many forms. **Compile-time (static) polymorphism** is resolved by the compiler (early binding) using function overloading, operator overloading and templates — e.g. `area(int)`, `area(int,int)` and `area(double)` are three functions chosen by argument types. It is fast because there is no run-time lookup. **Run-time (dynamic) polymorphism** is resolved during execution (late binding) using virtual functions and function overriding: a base-class pointer `Base *p = new Derived;` calling `p->show()` executes `Derived::show()` because `show` is declared `virtual`; the call goes through the object's vtable. It allows code written against a base class to work with any future derived class, at a small cost of an indirect call. Overloading needs a different parameter list; overriding needs the same signature in a derived class.

**Q. Explain the order of constructor and destructor calls in inheritance.** *(5 marks · Likelihood 7/10)*
When a derived-class object is created, the base-class constructor executes first, then member objects are constructed, then the derived-class constructor body runs — the object is built from the foundation upward. In multiple inheritance, base constructors run in the order the bases are listed in the class declaration. Destructors run in exactly the reverse order: derived destructor first, then members, then base. In the verified program with `Base` and `Derived`, creating and deleting a `Derived` through a `Base*` prints: Base ctor, Derived ctor, Derived::show, Derived dtor, Base dtor. The base destructor must be `virtual`; otherwise `delete p` on a base pointer calls only `~Base()` (undefined behaviour, derived resources leak). Constructors cannot be virtual; destructors cannot be overloaded.

**Q. What is a well-formed XML document? Differentiate DTD and XML Schema.** *(5 marks · Likelihood 6/10)*
An XML document is well-formed when it obeys XML syntax rules: exactly one root element; every

opening tag has a matching closing tag or is self-closing; elements are properly nested (no overlap); tag names are case-sensitive; attribute values are quoted; an element has no duplicate attribute; reserved characters are escaped (`<`, `&`). A well-formed document that also follows the rules in a DTD or Schema is called valid.
DTD (Document Type Definition) uses its own non-XML syntax (`<!ELEMENT student (name, roll)>`), has no data types (everything is text) and weak namespace support. XML Schema (XSD) is itself written in XML, supports built-in and user-defined data types (`xs:integer`, `xs:date`), occurrence limits (`minOccurs`, `maxOccurs`) and namespaces, so it validates data far more precisely. Schema is the modern choice.

**Q. Explain the life cycle of a servlet.** *(5 marks · Likelihood 6/10)*
A servlet is a Java class running inside a web container (e.g. Tomcat) that handles HTTP requests. Its life cycle is managed by the container through three methods of the `Servlet` interface. (1) Loading and instantiation: the container loads the class and creates one instance, on first request or at start-up. (2) `init(ServletConfig)` is called exactly once to initialise resources such as database connections. (3) `service(request, response)` is called for every client request, each on a separate thread; `HttpServlet` dispatches it to `doGet()`, `doPost()` etc. (4) `destroy()` is called once when the servlet is taken out of service, to release resources; the object is then garbage collected. Because one instance serves many requests with threads, servlets are faster than CGI, which starts a new process per request. (Draw the servlet life-cycle figure.)

**Q. What is hashing? Explain collision resolution techniques.** *(5 marks · Likelihood 8/10)*
Hashing stores and finds records by computing their address from the key with a hash function, giving near O(1) access. Common functions: division `k mod m` (m prime), mid-square, folding and multiplication. A collision occurs when two keys hash to the same slot. Resolution: (1) Chaining (open hashing) — each slot points to a linked list of all colliding keys; simple, table never fills. (2) Open addressing (closed hashing) — look for another free slot in the table: linear probing `(h+i) mod m` (simple but causes primary clustering), quadratic probing `(h+i²) mod m` (reduces primary clustering), double hashing `(h₁+i·h₂) mod m` (best spread). Example: with m = 11, keys 22 and 44 both hash to 0; using linear probing 22 goes to slot 0 and 44 to slot 1. Load factor n/m should be kept low to limit probes.

**Q. Explain the different types of inheritance in C++ with suitable diagrams and syntax.** *(10 marks · Likelihood 8/10)*
Inheritance is the mechanism by which a new class (derived/child) acquires the data members and member functions of an existing class (base/parent), supporting code reuse and an "is-a" relationship. Syntax: `class Derived : access Base { ... };` where access is `public`, `protected` or `private`.
**1. Single inheritance:** one base, one derived (A → B). `class Car : public Vehicle {};`
**2. Multiple inheritance:** one derived class from two or more bases (A, B → C). `class TA : public Student, public Employee {};` C++ supports it; Java allows it only through interfaces. Ambiguity arises if both bases have a member with the same name; resolved with the scope operator `Student::show()`.
**3. Multilevel inheritance:** a chain (A → B → C), e.g. `Person → Employee → Manager`; C gets members of both B and A.
**4. Hierarchical inheritance:** one base, several derived (A → B, A → C), e.g. `Shape → Circle, Rectangle`.
**5. Hybrid inheritance:** a combination of the above, typically the diamond: B and C derive from A, and D derives from both B and C. D would get two copies of A's members (ambiguity); this is solved by declaring `class B : virtual public A` and `class C : virtual public A`, so only one shared A sub-object exists.
**Access control:** with public inheritance, public members of the base stay public and protected stay protected; with protected inheritance both become protected; with private inheritance both become

private. Private members of the base are inherited but never directly accessible in the derived class.
**Constructor order:** base constructors run first (in declaration order), then the derived one; destructors run in reverse.
**Advantages:** reusability, extensibility, less redundancy, enables run-time polymorphism through virtual functions. **Disadvantages:** tight coupling between classes, deep hierarchies are hard to maintain.
(Draw five small box-and-arrow diagrams for the five types.)

**Q. Explain hashing and resolve collisions using linear probing for keys 18, 41, 22, 44, 59, 32, 31, 73 with table size 11.** *(10 marks · Likelihood 7/10)*
Hashing maps a key to a table address using a hash function h(k), so a search costs about one access instead of O(n) or O(log n). A good hash function is fast to compute, uses all of the key, and spreads keys uniformly. Methods: division (`k mod m`, m prime), mid-square, folding, multiplication. When two keys hash to the same address we have a collision, resolved by chaining or open addressing (linear, quadratic, double hashing).
Here m = 11, h(k) = k mod 11, linear probing h(k, i) = (h(k) + i) mod 11.
18 mod 11 = 7 → slot 7. 41 mod 11 = 8 → slot 8. 22 mod 11 = 0 → slot 0. 44 mod 11 = 0 → collision, try 1 → slot 1. 59 mod 11 = 4 → slot 4. 32 mod 11 = 10 → slot 10. 31 mod 11 = 9 → slot 9. 73 mod 11 = 7 → slots 7, 8, 9, 10, 0, 1 all occupied → slot 2 (6 extra probes).
Final table: 0:22, 1:44, 2:73, 3:–, 4:59, 5:–, 6:–, 7:18, 8:41, 9:31, 10:32. Load factor = 8/11 ≈ 0.73.
Observation: key 73 shows **primary clustering** — the run of filled slots 7 to 1 grows and every key hashing into it must walk to the end. Quadratic probing or double hashing would spread these keys; chaining would place 73 directly in the list at slot 7. Searching uses the same probe sequence and stops at an empty slot (unsuccessful search). Deletion in open addressing needs a "deleted" marker (tombstone) so later searches do not stop early.

**Q. What are inverted lists and multi-lists? Explain with an example and state where inverted indexes are used.** *(10 marks · Likelihood 7/10)*
Both are file structures for retrieving records on secondary keys (non-unique attributes) without scanning the whole file.
**Inverted list:** a separate index is kept for each secondary key; for every value of that key it stores the list of addresses of records having it. Example employee file R1 (CSE, Kohima), R2 (ECE, Dimapur), R3 (CSE, Dimapur), R4 (CSE, Kohima). Dept index: CSE → {R1, R3, R4}, ECE → {R2}; City index: Kohima → {R1, R4}, Dimapur → {R2, R3}. The query "Dept = CSE AND City = Kohima" is answered by intersecting {R1, R3, R4} ∩ {R1, R4} = {R1, R4} — only the matching records are read. Advantages: very fast Boolean queries, record file unchanged. Disadvantages: index storage can be large; every insert/delete must update all relevant lists.
**Multi-list:** instead of address lists, each record has an extra link field per secondary key, and all records sharing a value are chained. The index stores only the head pointer and the length of each chain: CSE → head R1, length 3 (R1 → R3 → R4). Advantages: small index, easy to add a record (insert at head). Disadvantages: a query must follow the chain on disk, one access per record; Boolean AND needs walking the shorter chain and testing each record. Knowing the chain length helps pick the shorter chain.
**Uses of inverted indexes:** search engines, full-text search in databases and document systems, and digital forensics — tools such as Autopsy (Solr keyword index) and FTK (dtSearch) index every word in the evidence image once, so later keyword searches for names, phone numbers or e-mail addresses return file hits immediately instead of rescanning terabytes.

**Q. Explain the different file organisations (sequential, direct, index-sequential, relative). Describe B-trees and B+ trees and differentiate them.** *(15 marks · Likelihood 8/10)*
**File organisation** is the way records of a file are arranged on secondary storage; it decides the speed of retrieval, insertion and update. A file is a collection of records; a record is a collection of related fields; one field (the primary key) identifies each record.

**1. Sequential organisation:** records are stored one after another in order of the key. Retrieval of a particular record requires reading from the start (average n/2 reads). Insertion and deletion require creating a new copy of the file, usually done in batches. It uses storage efficiently and is ideal for batch jobs where most records are processed, such as payroll and billing. It works on tape as well as disk.

**2. Direct (random/hashed) organisation:** a hash function converts the key into a disk address, so a record is read in about one access. It is ideal for online systems that need fast random access (reservations, ATM). Sequential processing is poor because records are scattered, and collisions need overflow handling. Disk only.

**3. Index-sequential organisation (ISAM):** records are kept sorted in a prime data area and an index stores the highest key of each block with its address. To find a record, the index is searched and then only one block is read. The file can still be processed sequentially. New records that do not fit are placed in an overflow area linked from their block, so performance falls as overflow grows and the file must be reorganised periodically. (Draw the index-sequential figure: index → prime blocks → overflow area.)

**4. Relative organisation:** the file is a series of fixed-size slots numbered 1 to n; a record is stored in the slot given by its relative record number, so address = start + (n − 1) × record size. Access by number is direct and fast; empty slots waste space; a key must be converted to a slot number. (Give the comparison table: placement, random access, sequential access, insertion, media, typical use.)

**B-tree:** a balanced multi-way search tree designed for disks, where each node is one block. A B-tree of order m: every node has at most m children and m − 1 sorted keys; every internal node except the root has at least ⌈m/2⌉ children; the root has at least two children unless it is a leaf; all leaves are at the same level. Insertion goes into a leaf; a full node is split and its median key moves up to the parent, so the tree grows at the root and stays balanced. Deletion borrows from a sibling or merges nodes. Height is O(log_m n), so very few disk reads are needed even for millions of keys.

**B+ tree:** a B-tree variant used by most DBMS indexes and file systems. Internal nodes contain only keys used for routing; all keys with their record pointers are stored in the leaf level, and leaves are linked left to right. Every search ends at a leaf, and range queries ("roll 20 to 50") find the first leaf and then follow the leaf chain. Because internal nodes hold no data, more keys fit per node, giving higher fan-out and a shorter tree. (Draw the B+ tree figure of order 4: root [20 | 40], leaves [5 10] → [20 30] → [40 50 60].)

**Differences:** in a B-tree data pointers are in all nodes, each key appears once, leaves are not linked, a search may end at an internal node, and range queries require traversal; in a B+ tree data pointers are only in leaves, internal keys are repeated in leaves, leaves are linked, every search reaches a leaf, range and sequential access are fast, and fan-out is higher. B+ trees therefore combine the random-access speed of an index with the sequential speed of a sorted file — which is exactly the goal of index-sequential organisation, without its overflow problem.